

Confluo:

Distributed Monitoring and Diagnosis Stack for High-speed Networks

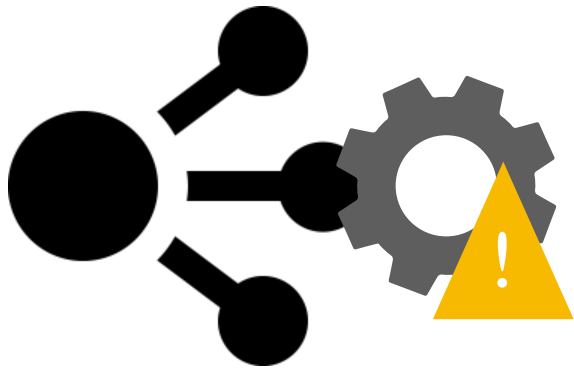
Anurag Khandelwal, Rachit Agarwal, Ion Stoica



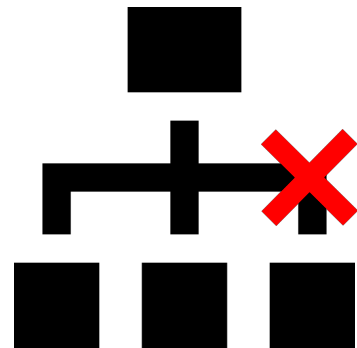
Motivation

Motivation

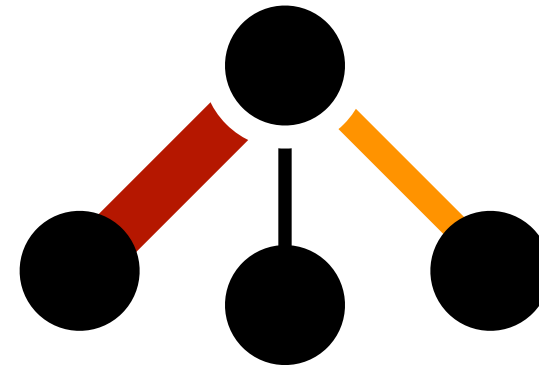
- Managing large scale networks is increasingly **complex**



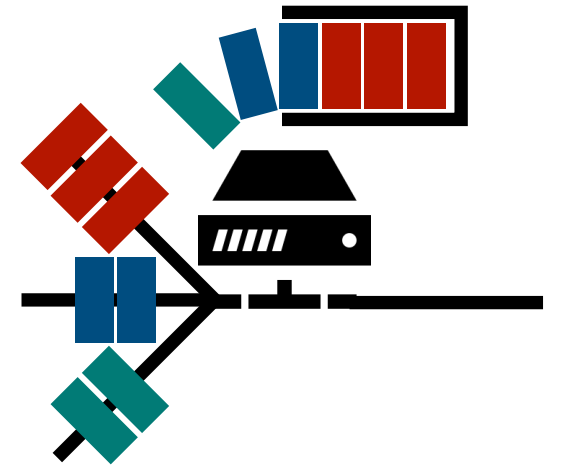
Network Misconfigurations



Network Failures



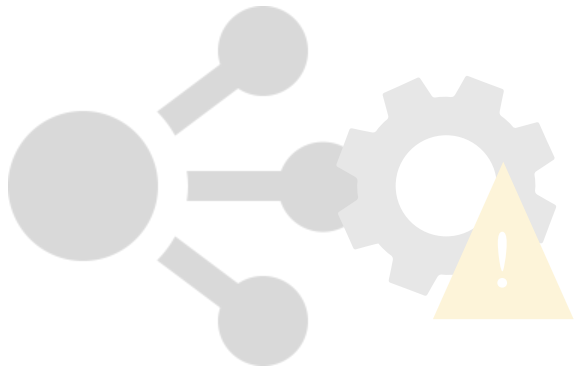
Load Imbalance



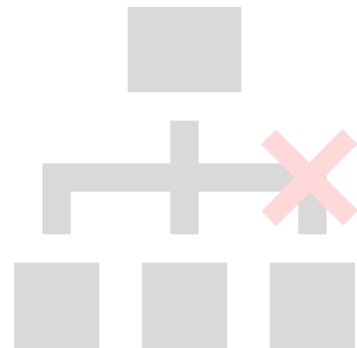
Network Congestion

Motivation

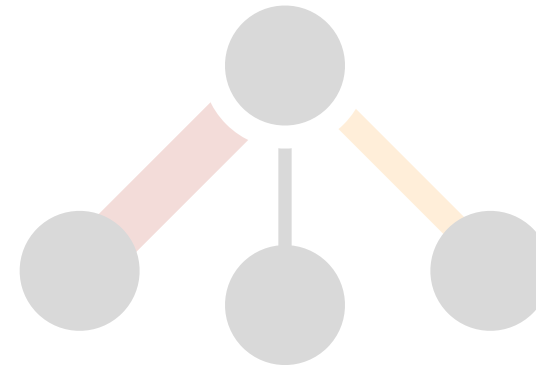
- Managing large scale networks is increasingly **complex**



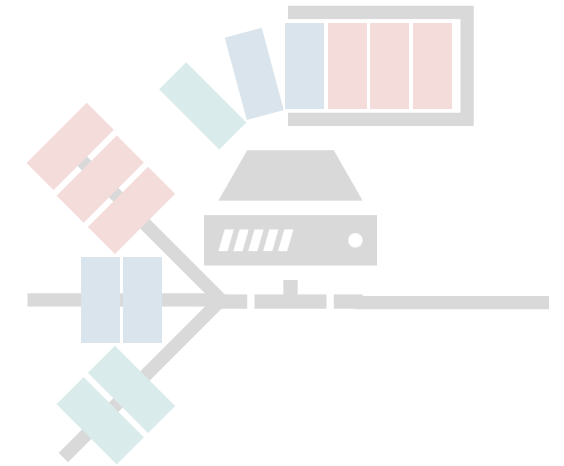
Network Misconfigurations



Network Failures



Load Imbalance



Network Congestion

- Network issues $\Rightarrow$ Performance degradation, loss in revenue

NETWORKWORLD
FROM IDG

Amazon EBS failure brings down Reddit, Imgur, others

NETWORKWORLD
FROM IDG

UPDATE: Amazon.com suffers outage: Nearly \$5M down the drain?

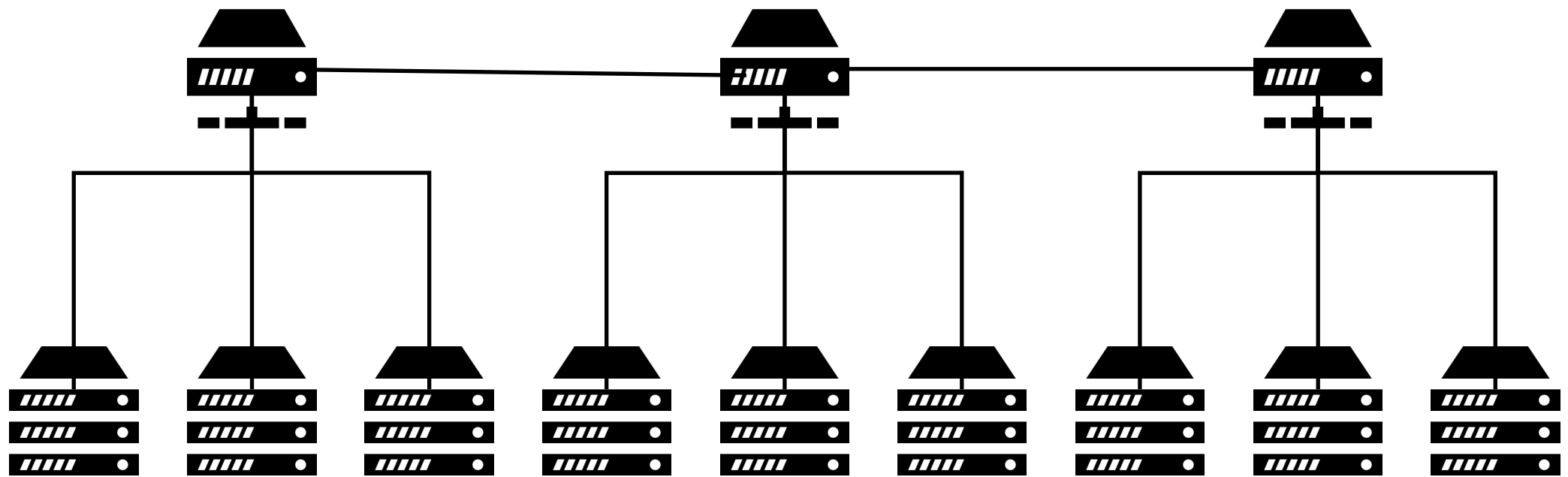
sky news

Google Outage: Internet Traffic Plunges 40%

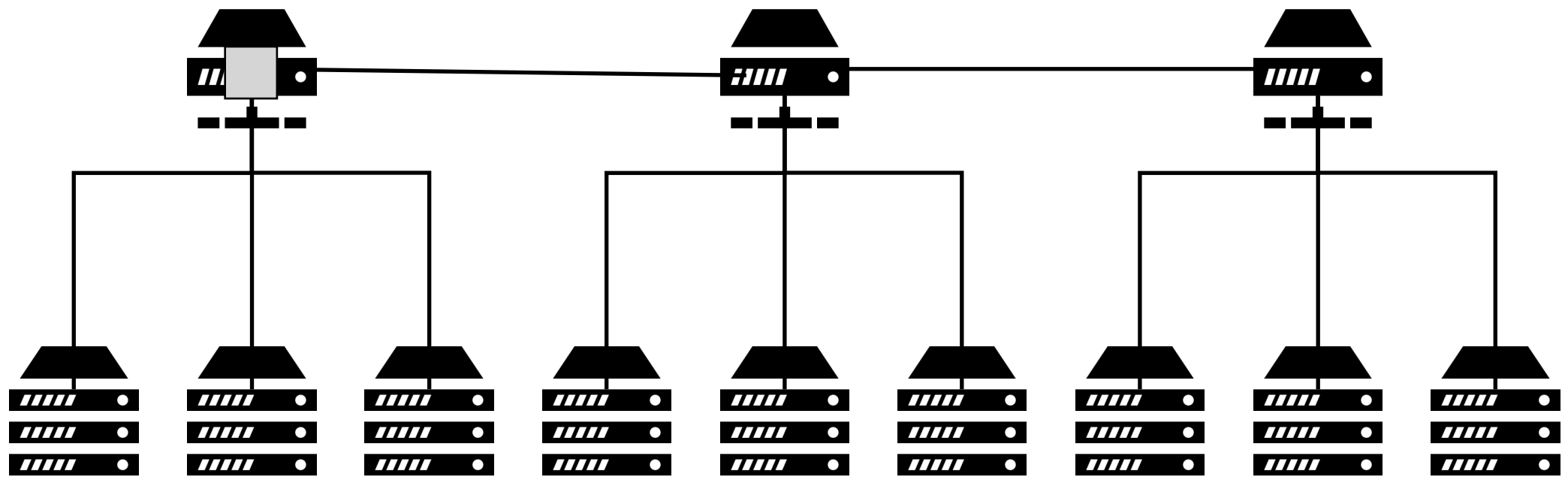
**PUGET SOUND
BUSINESS JOURNAL**

Microsoft Azure goes down for five hours, raises questions about cloud reliance

Opportunity: Networks can capture a lot of data...

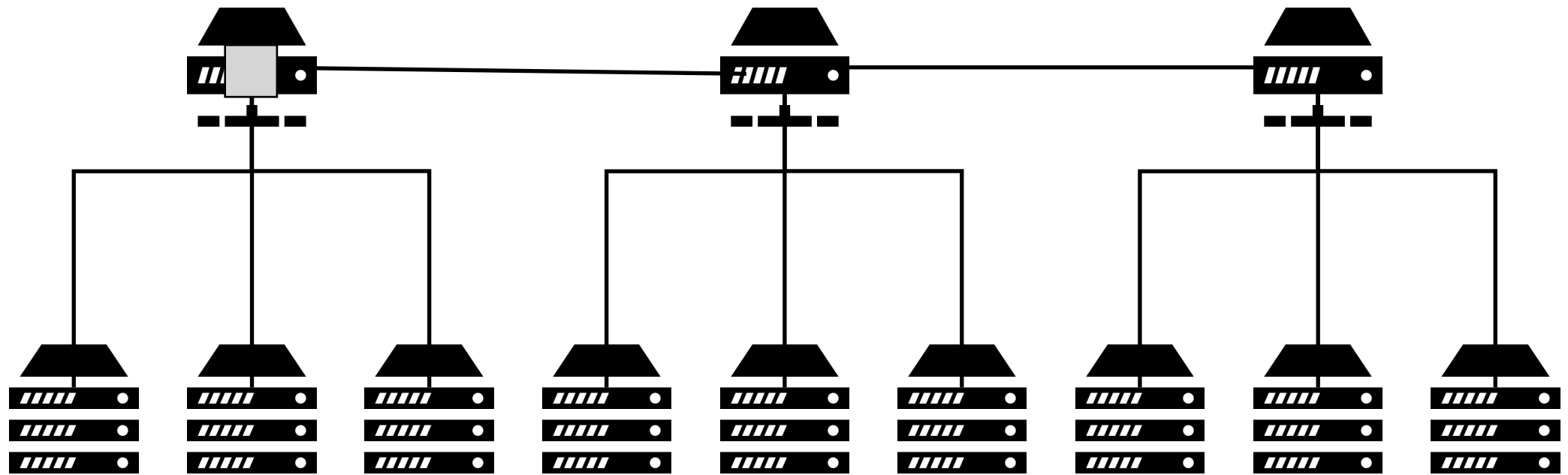


Opportunity: Networks can capture a lot of data...



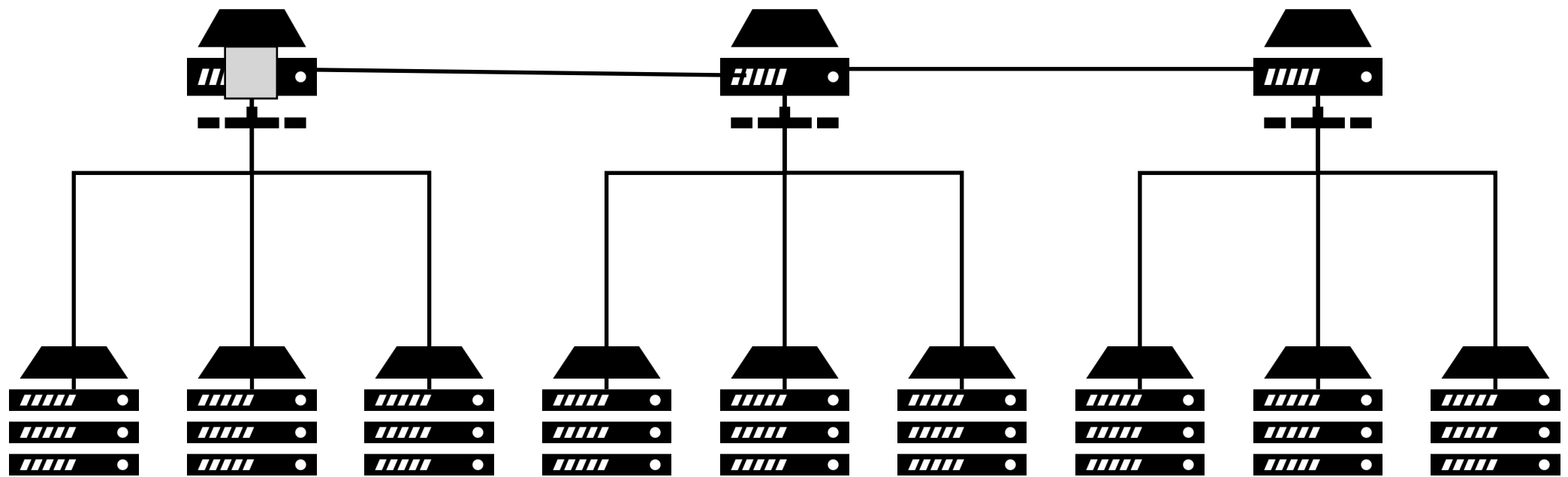
Opportunity: Networks can capture a lot of data...

In-network techniques, e.g., Marple [SIGCOMM'19],
FlowRadar [NSDI'16], UnivMon [SIGCOMM'16]



Opportunity: Networks can capture a lot of data...

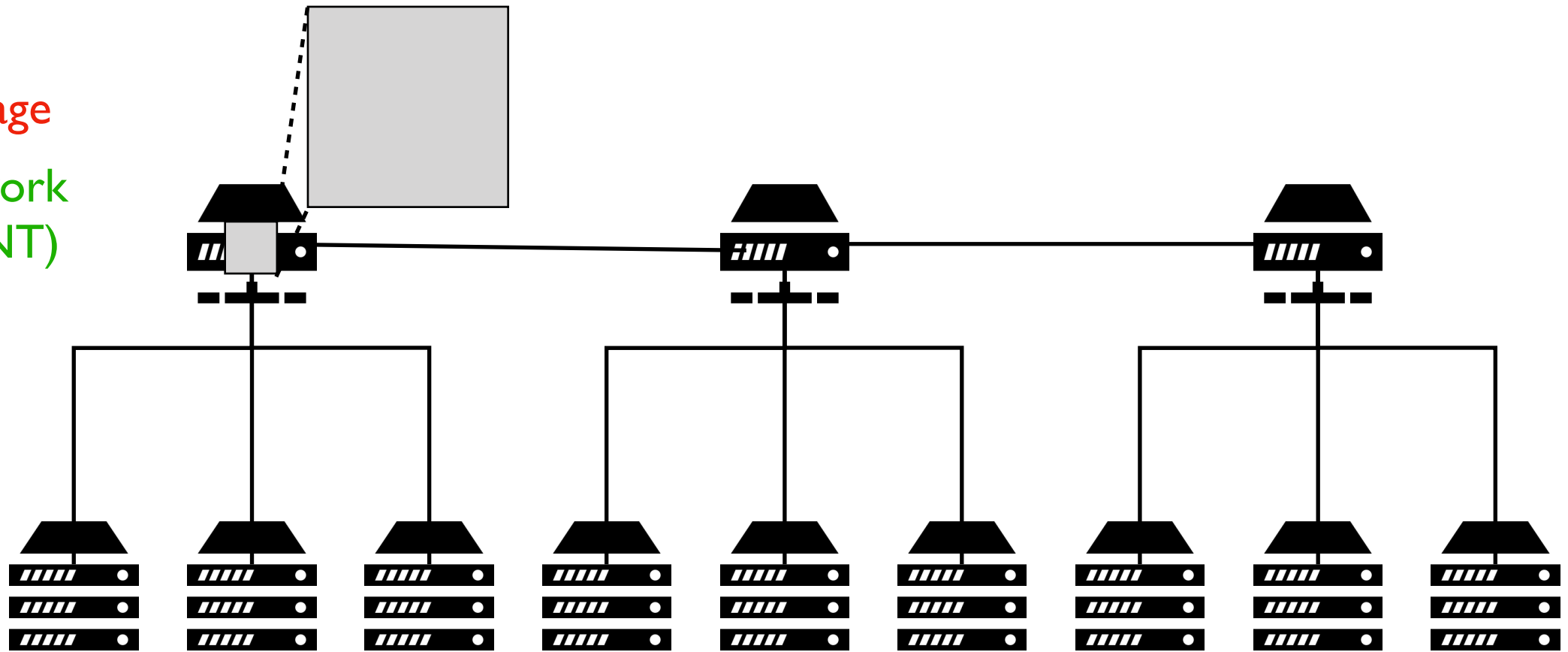
Limited storage



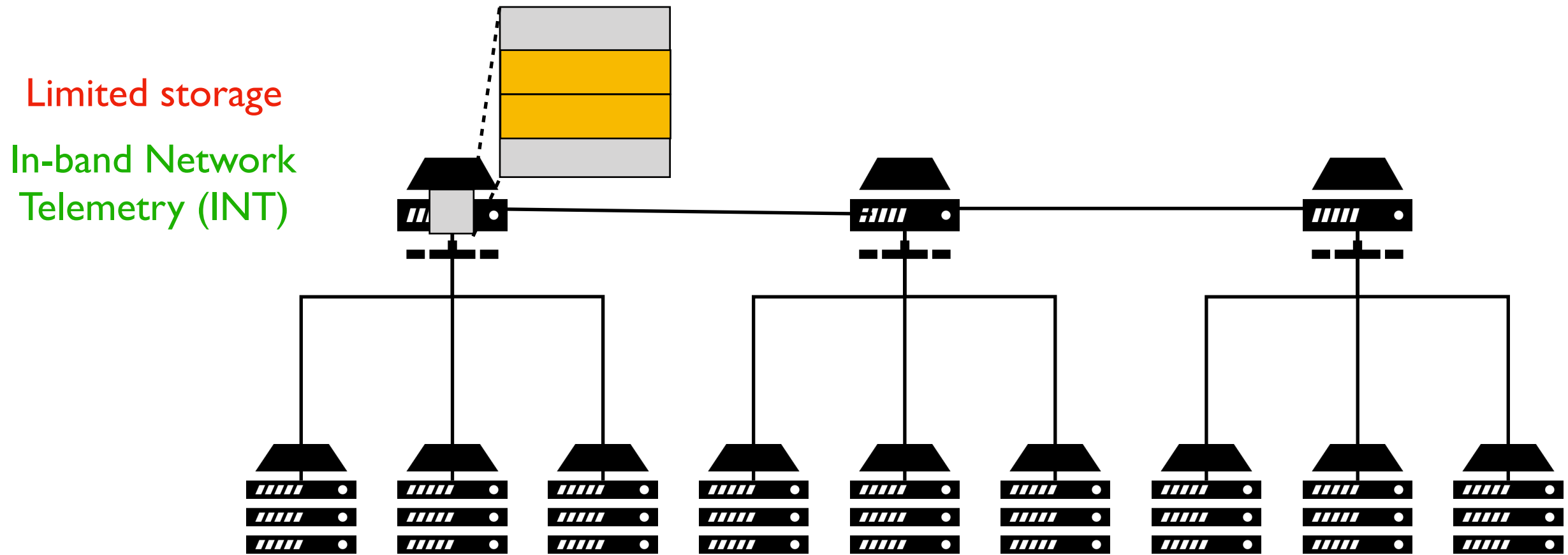
Opportunity: Networks can capture a lot of data...

Limited storage

In-band Network
Telemetry (INT)



Opportunity: Networks can capture a lot of data...

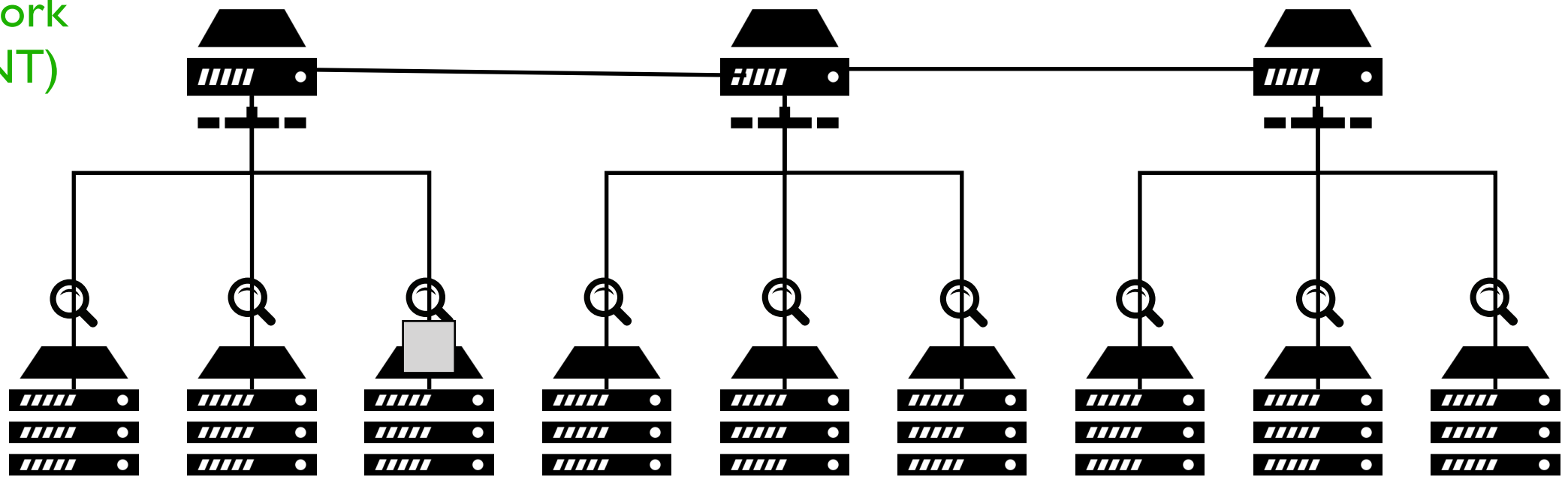


- Embed wide range of telemetry data within packet headers:
 - ▶ Packet trajectory
 - ▶ Queue lengths
 - ▶ Hop latency
 - ▶ Link utilization, and many more...

Opportunity: Networks can capture a lot of data...

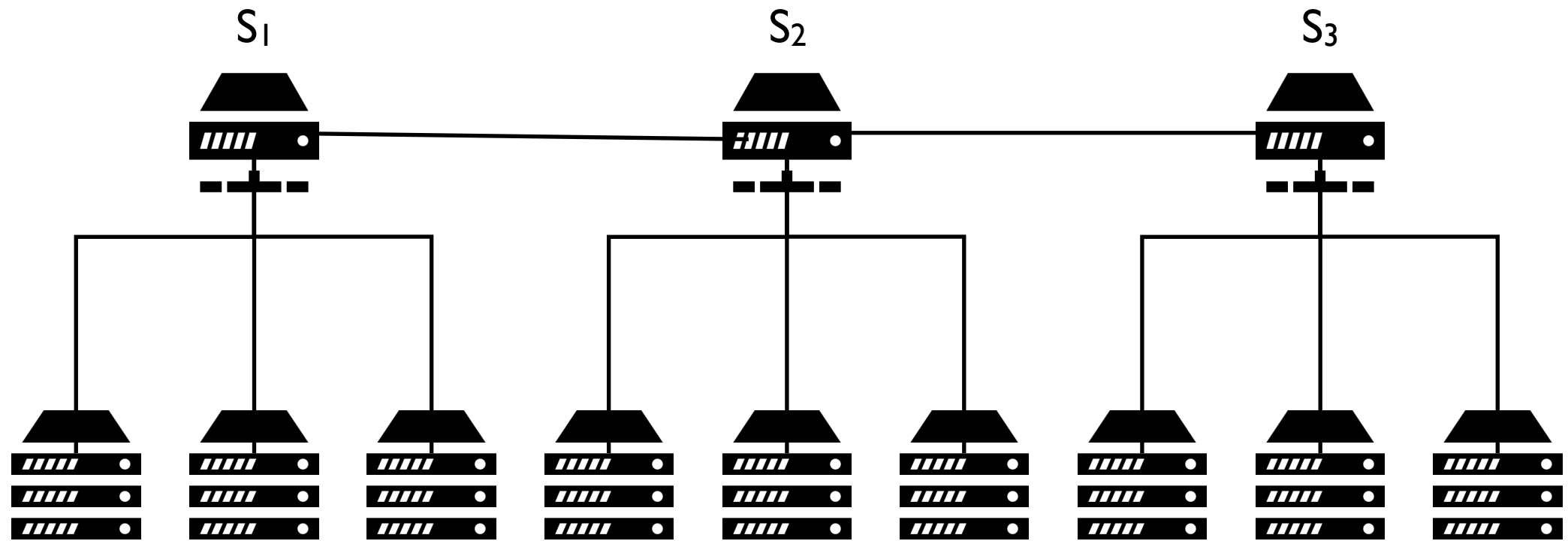
Limited storage

In-band Network
Telemetry (INT)



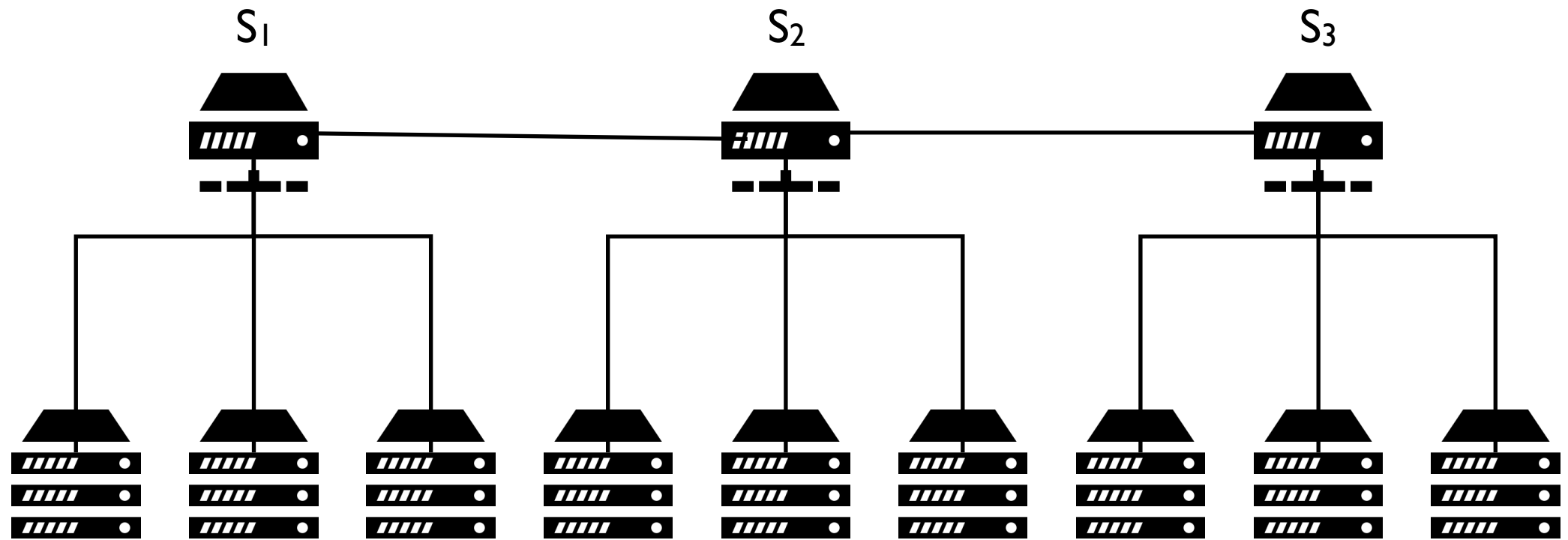
- Embed wide range of telemetry data within packet headers:
 - ▶ Packet trajectory
 - ▶ Queue lengths
 - ▶ Hop latency
 - ▶ Link utilization, and many more...
- Analyze telemetry data **at end-hosts**
e.g., Trumpet [SIGCOMM'16], PathDump [OSDI'16], SwitchPointer [NSDI'18]

Example: Checking Path Conformance



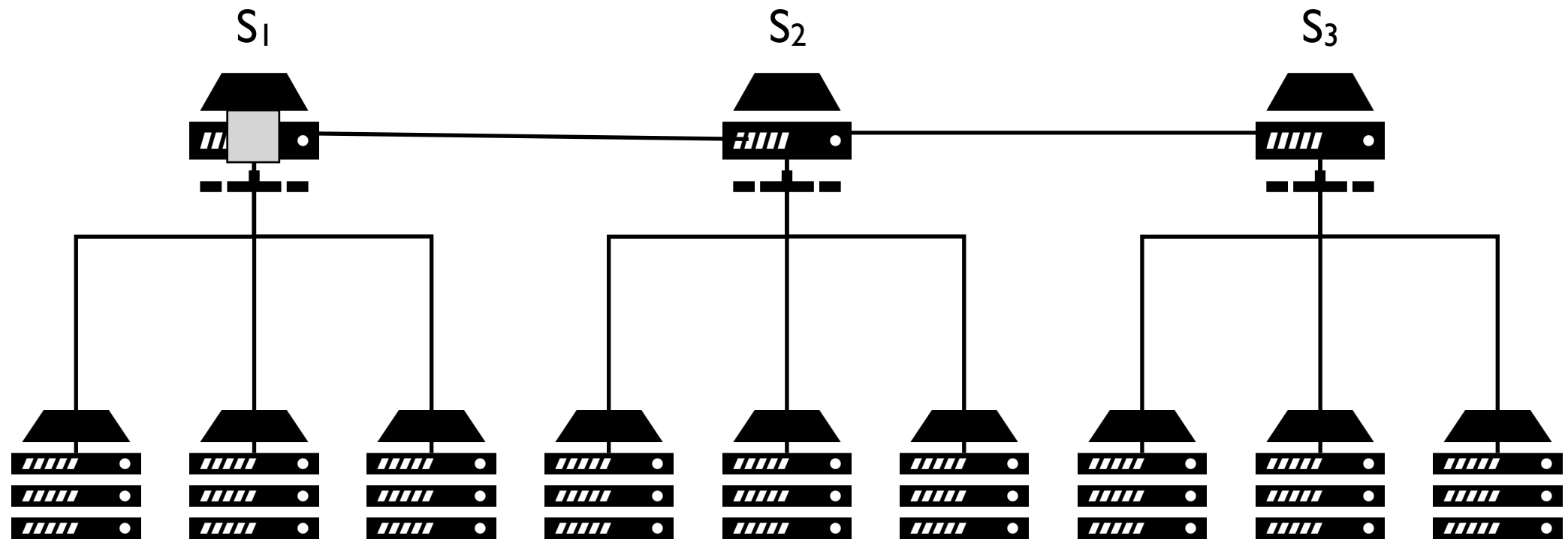
- Embed wide range of telemetry data within packet headers:
 - ▶ **Packet trajectory**
 - ▶ Hop latency
 - ▶ Queue lengths
 - ▶ Link utilization, and many more...

Example: Checking Path Conformance



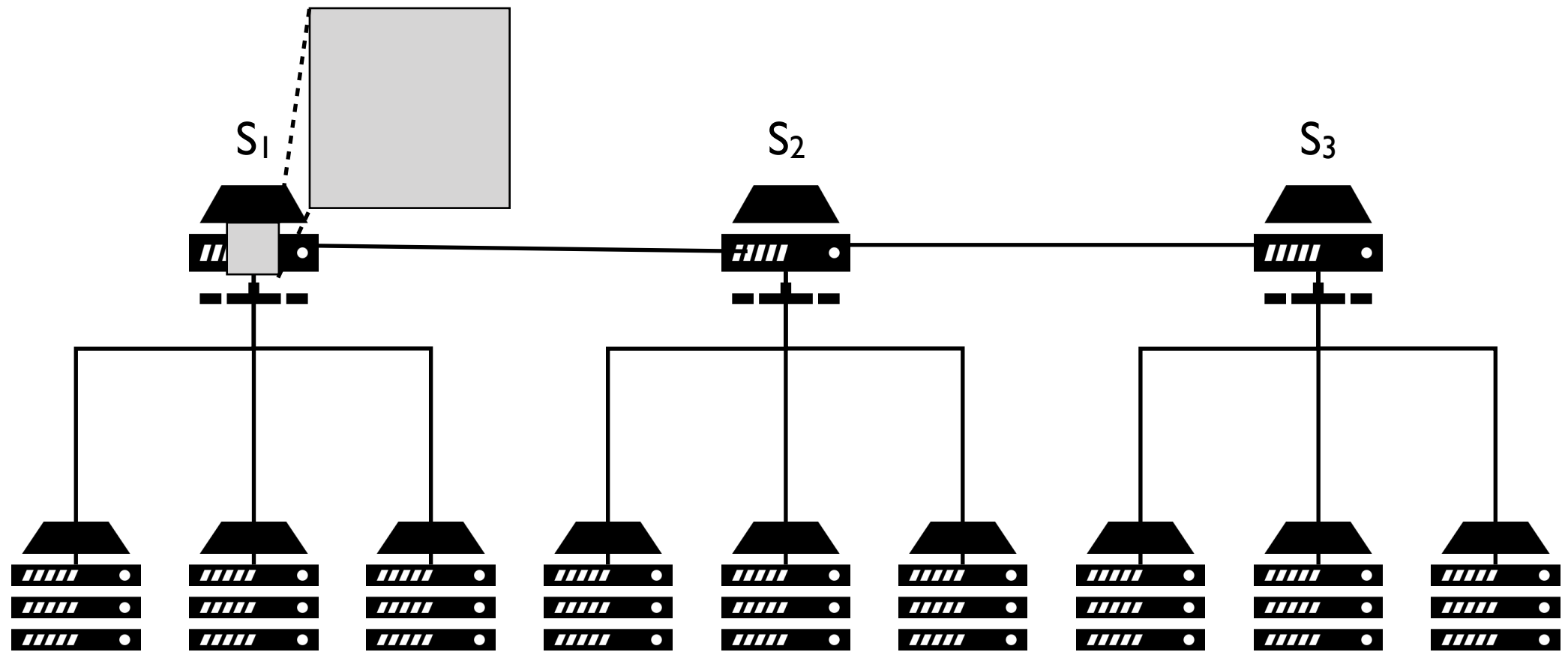
- Embed wide range of telemetry data within packet headers:
 - ▶ **Packet trajectory**
 - ▶ Hop latency
 - ▶ Queue lengths
 - ▶ Link utilization, and many more...
- *Does the packet pass through switch S_1 ?* **[PathDump, OSDI'16]**

Example: Checking Path Conformance



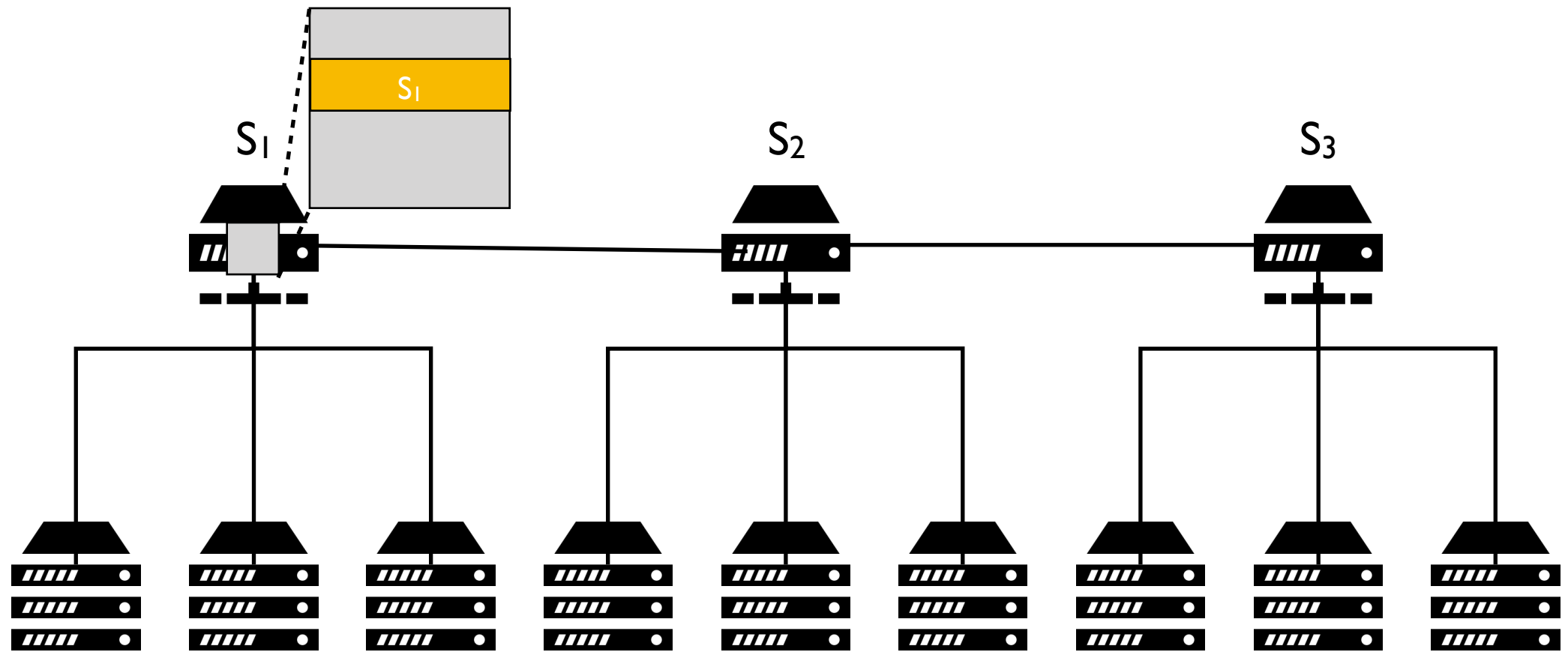
- Embed wide range of telemetry data within packet headers:
 - ▶ **Packet trajectory**
 - ▶ Hop latency
 - ▶ Queue lengths
 - ▶ Link utilization, and many more...
- *Does the packet pass through switch S_1 ?* **[PathDump, OSDI'16]**

Example: Checking Path Conformance



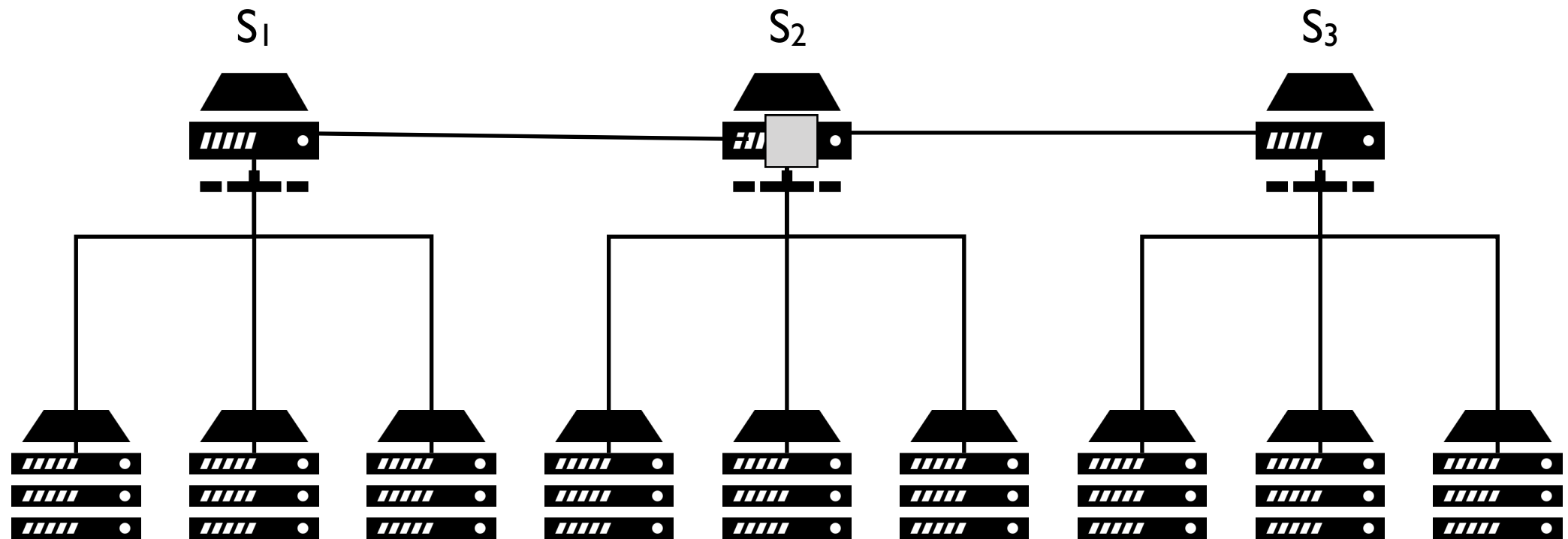
- Embed wide range of telemetry data within packet headers:
 - ▶ **Packet trajectory**
 - ▶ Hop latency
 - ▶ Queue lengths
 - ▶ Link utilization, and many more...
- *Does the packet pass through switch S_1 ?* **[PathDump, OSDI'16]**

Example: Checking Path Conformance



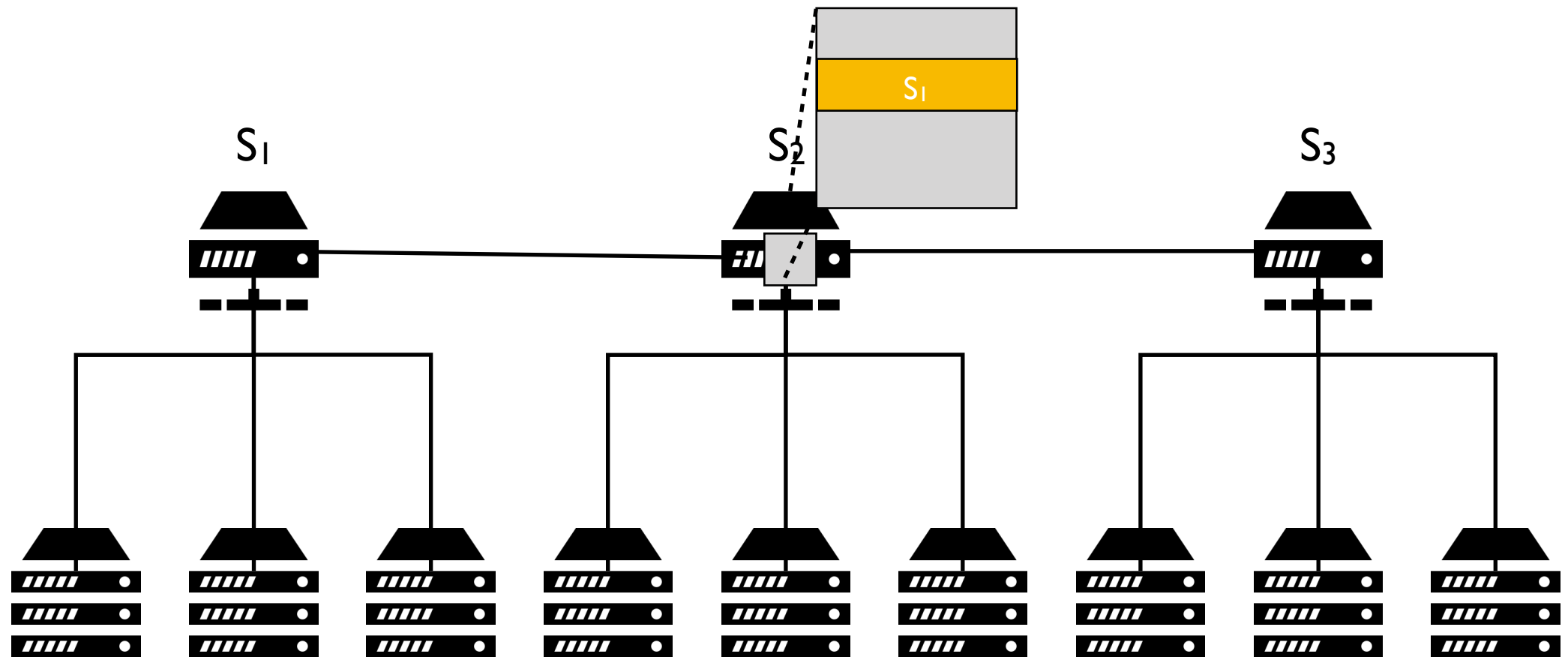
- Embed wide range of telemetry data within packet headers:
 - ▶ **Packet trajectory**
 - ▶ Hop latency
 - ▶ Queue lengths
 - ▶ Link utilization, and many more...
- *Does the packet pass through switch S_1 ?* **[PathDump, OSDI'16]**

Example: Checking Path Conformance



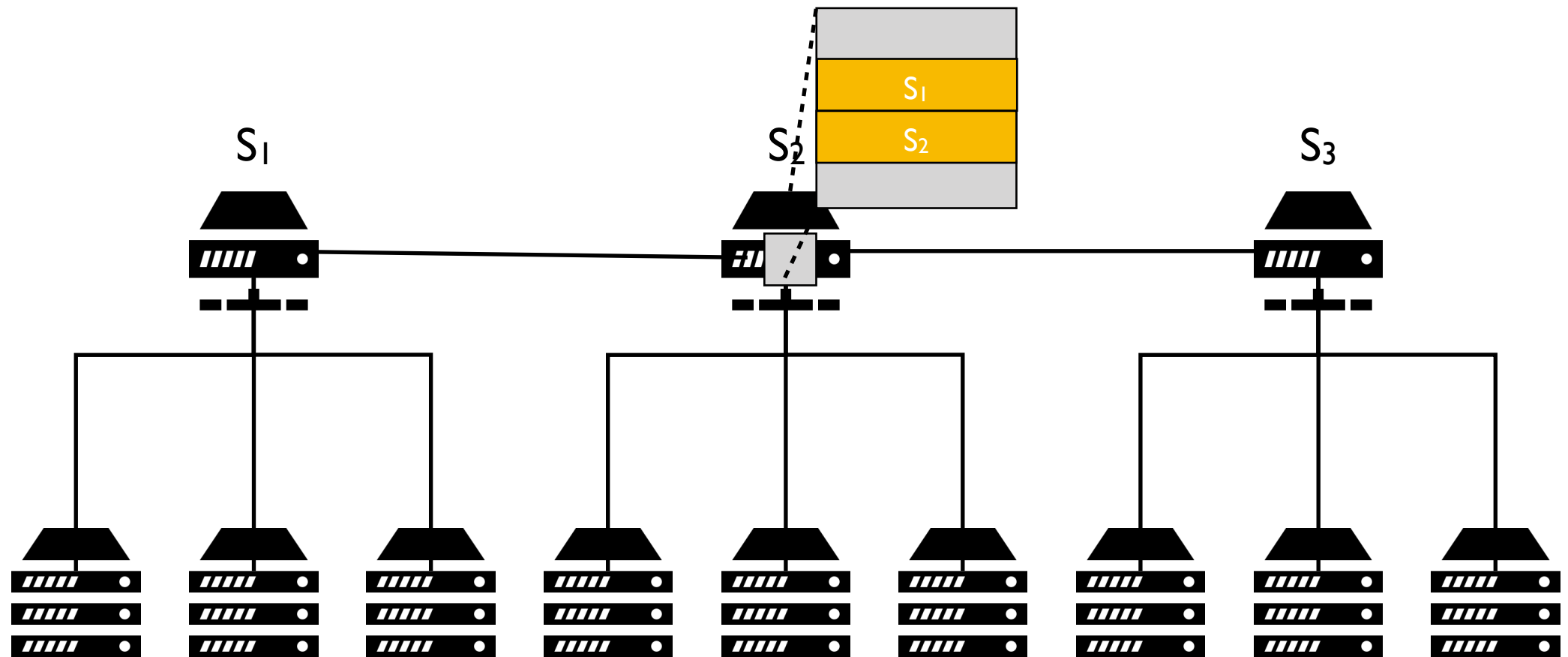
- Embed wide range of telemetry data within packet headers:
 - ▶ **Packet trajectory**
 - ▶ Hop latency
 - ▶ Queue lengths
 - ▶ Link utilization, and many more...
- *Does the packet pass through switch S_1 ?* **[PathDump, OSDI'16]**

Example: Checking Path Conformance



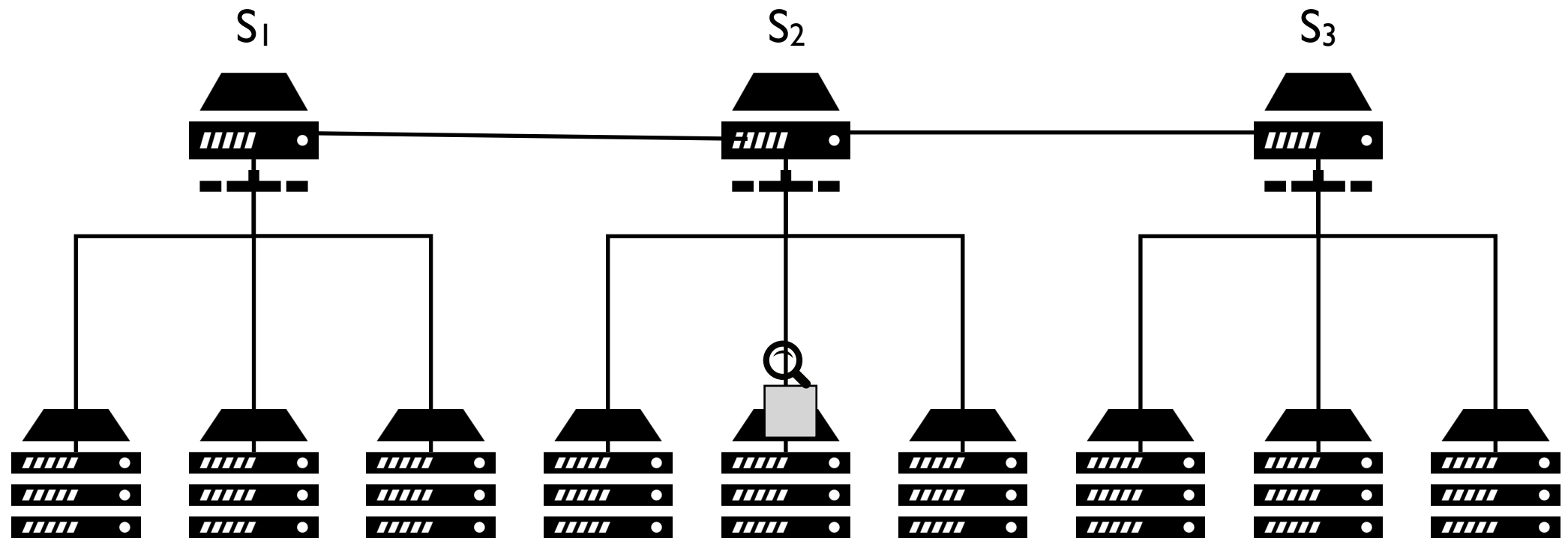
- Embed wide range of telemetry data within packet headers:
 - ▶ **Packet trajectory**
 - ▶ Hop latency
 - ▶ Queue lengths
 - ▶ Link utilization, and many more...
- *Does the packet pass through switch S_1 ?* **[PathDump, OSDI'16]**

Example: Checking Path Conformance



- Embed wide range of telemetry data within packet headers:
 - ▶ **Packet trajectory**
 - ▶ Hop latency
 - ▶ Queue lengths
 - ▶ Link utilization, and many more...
- *Does the packet pass through switch S_1 ?* **[PathDump, OSDI'16]**

Example: Checking Path Conformance



- Embed wide range of telemetry data within packet headers:
 - ▶ **Packet trajectory**
 - ▶ Hop latency
 - ▶ Queue lengths
 - ▶ Link utilization, and many more...
- *Does the packet pass through switch S_1 ?* **[PathDump, OSDI'16]**

Goals for end-host stack design

Goals for end-host stack design

End-host stacks need to support:

Real-time monitoring of rich telemetry data

Goals for end-host stack design

End-host stacks need to support:

Real-time monitoring of rich telemetry data

Low-overhead distributed diagnosis of network events

Goals for end-host stack design

End-host stacks need to support:

Real-time monitoring of rich telemetry data

Low-overhead distributed diagnosis of network events

Highly-concurrent reads & writes of headers using minimal CPU

Challenge: ...Networks capture a lot of data

Challenge: ...Networks capture a lot of data

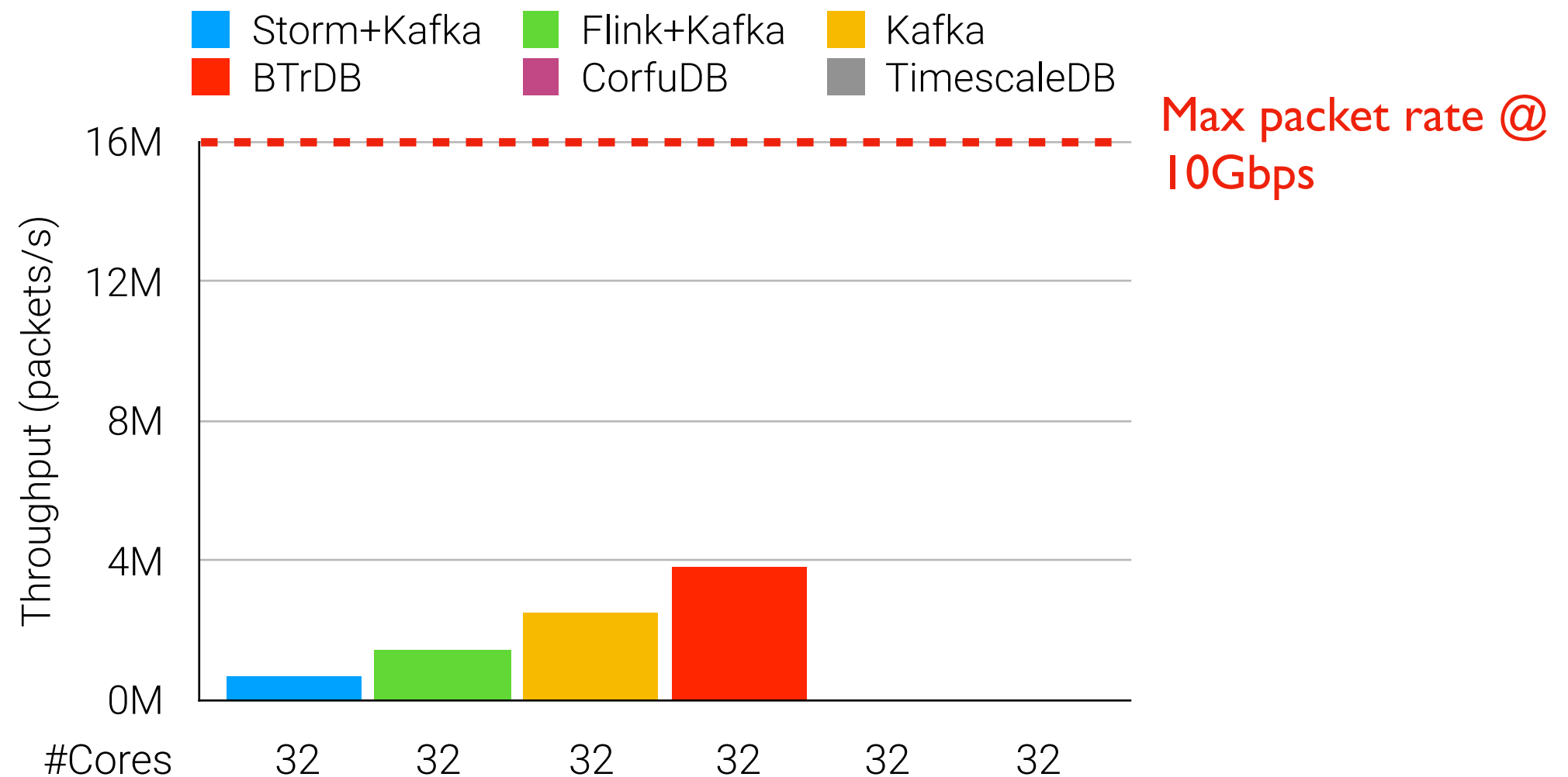
Line-rate for 10Gbps links $\Rightarrow$ 0.9-16 million packets/second

~ 50 nanoseconds budget per packet header!

Challenge: ...Networks capture a lot of data

Line-rate for 10Gbps links $\Rightarrow$ 0.9-16 million packets/second

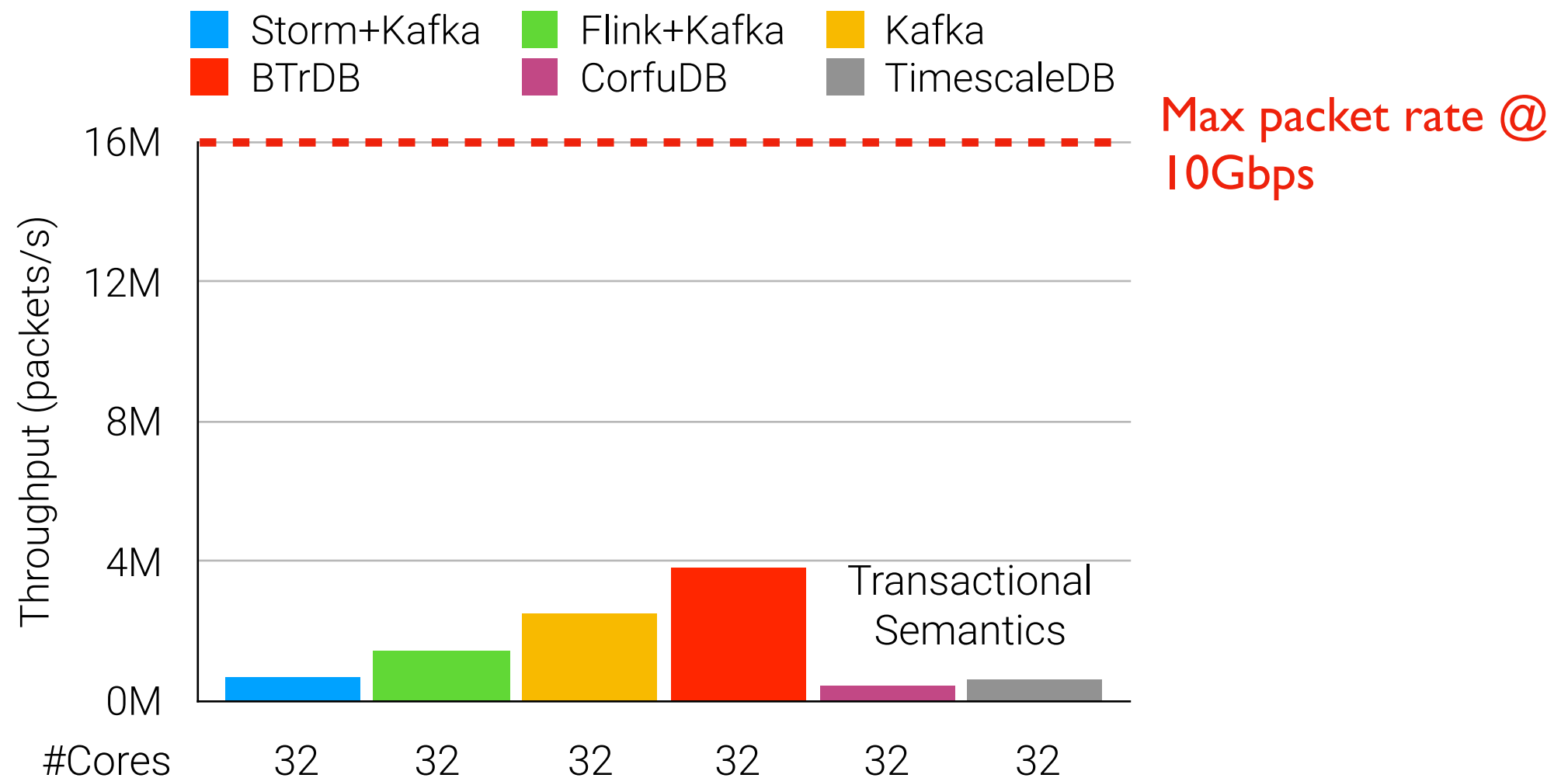
~ 50 nanoseconds budget per packet header!



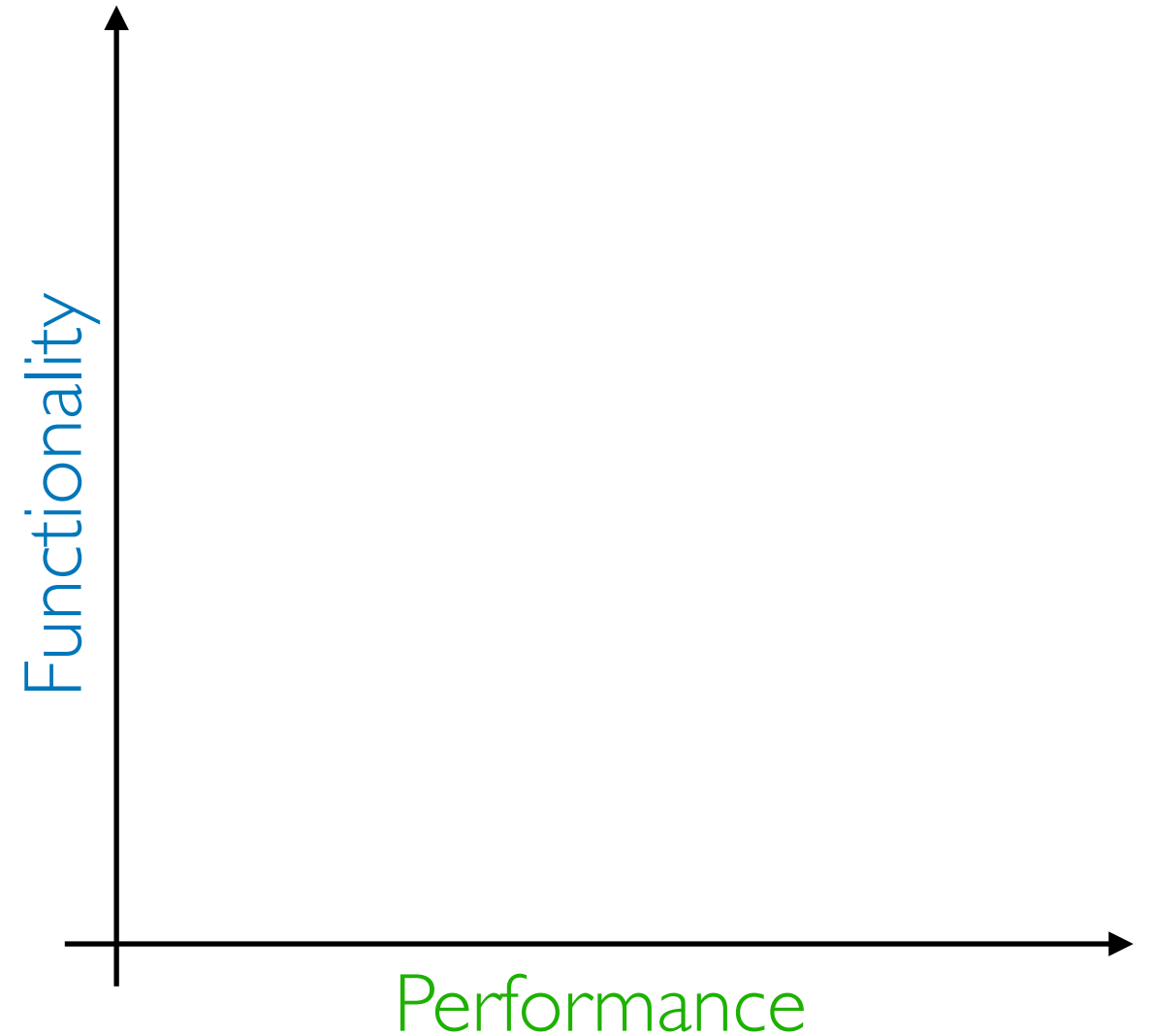
Challenge: ...Networks capture a lot of data

Line-rate for 10Gbps links $\Rightarrow$ 0.9-16 million packets/second

~ 50 nanoseconds budget per packet header!



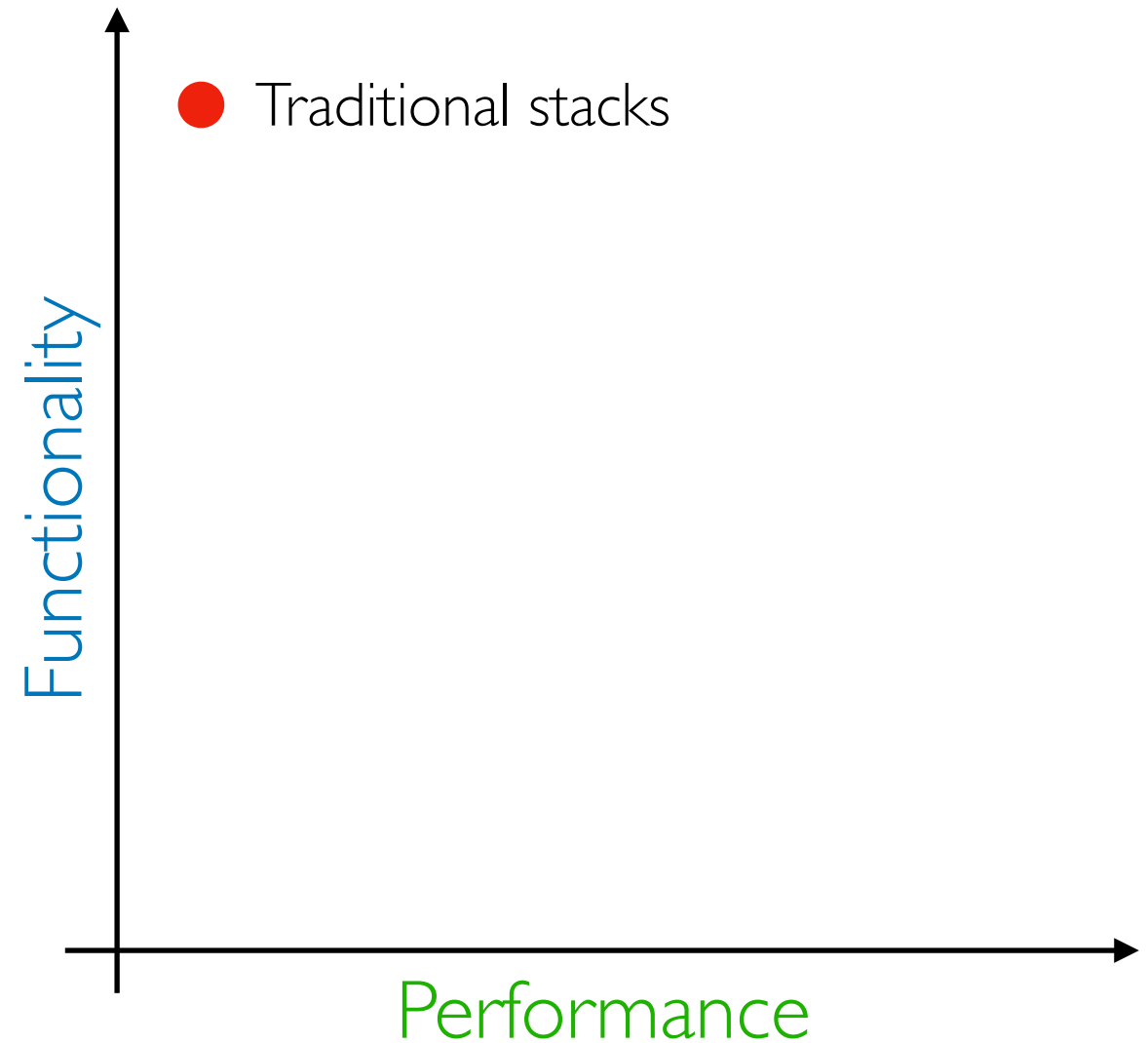
Existing Approaches



Existing Approaches

Traditional End-host Stacks

Tribeca [VLDB'96], Gigascope [SIGMOD'03],
Time Machine [SIGCOMM'08]



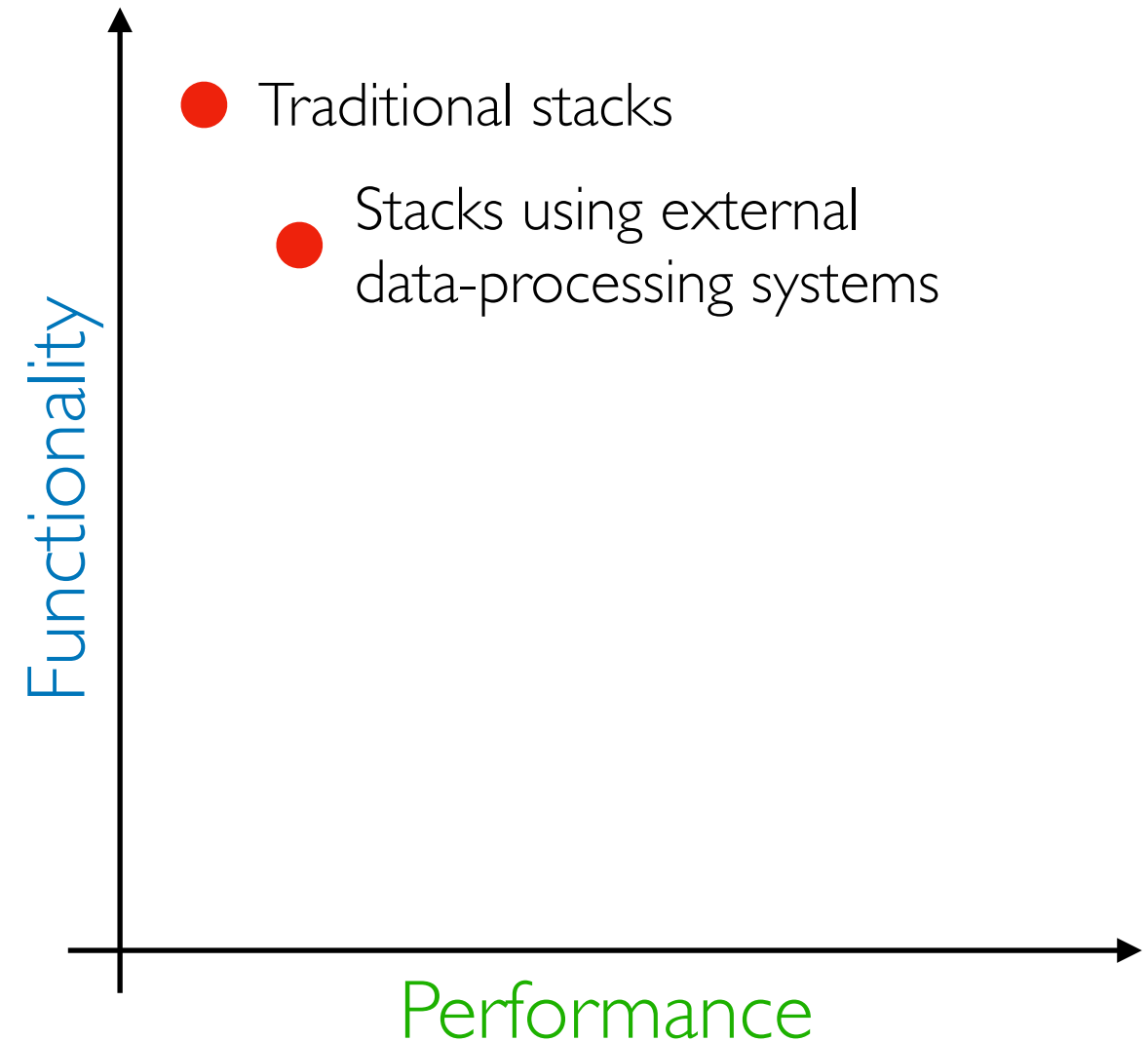
Existing Approaches

Traditional End-host Stacks

Tribeca [VLDB'96], Gigascope [SIGMOD'03],
Time Machine [SIGCOMM'08]

End-host Stacks using Stream-processing systems/Key-Value Stores

OpenSOC, Tigon, PathDump [OSDI'16],
SwitchPointer [NSDI'18]



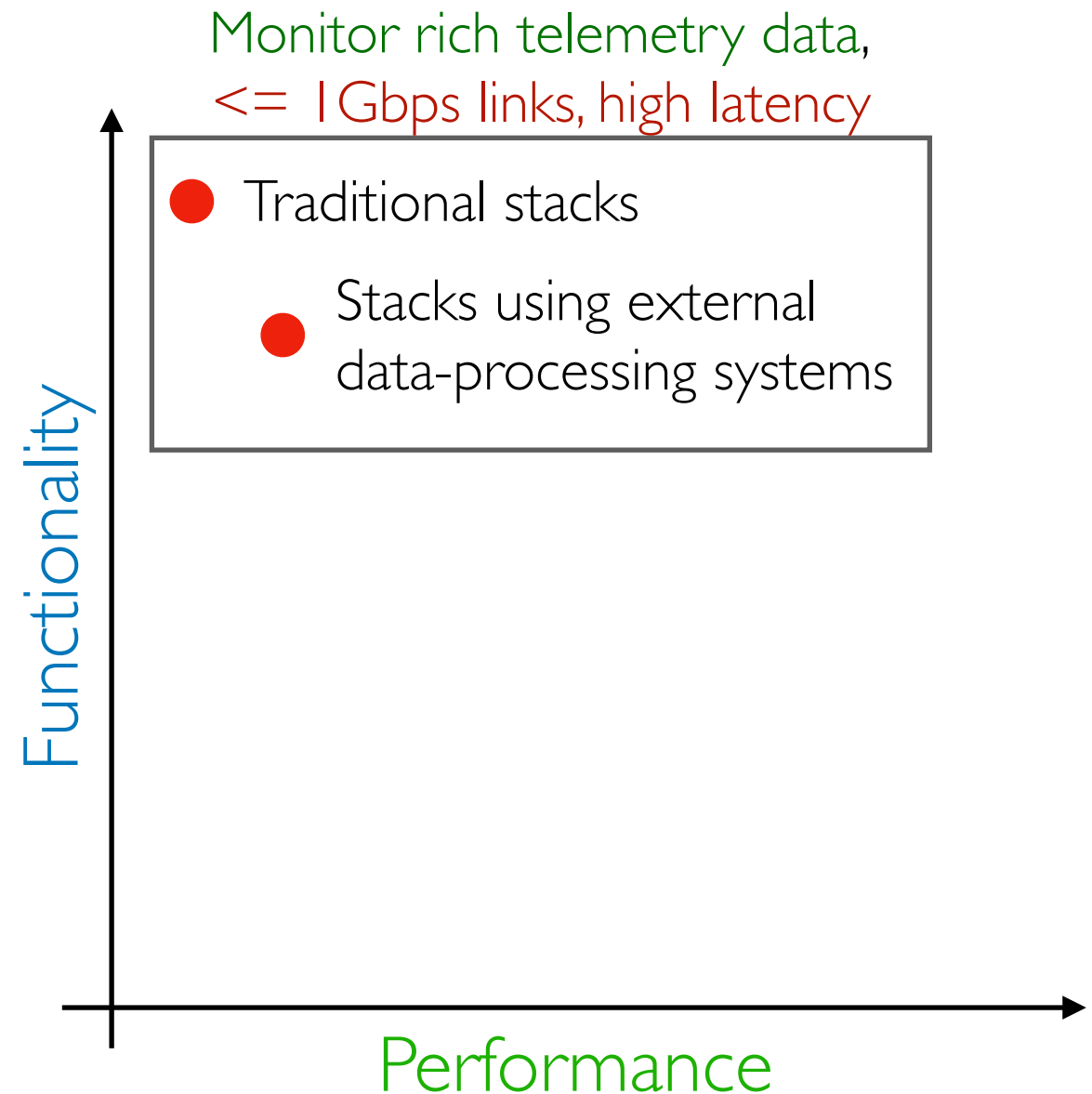
Existing Approaches

Traditional End-host Stacks

Tribeca [VLDB'96], Gigascope [SIGMOD'03],
Time Machine [SIGCOMM'08]

End-host Stacks using Stream-processing systems/Key-Value Stores

OpenSOC, Tigon, PathDump [OSDI'16],
SwitchPointer [NSDI'18]



Existing Approaches

Traditional End-host Stacks

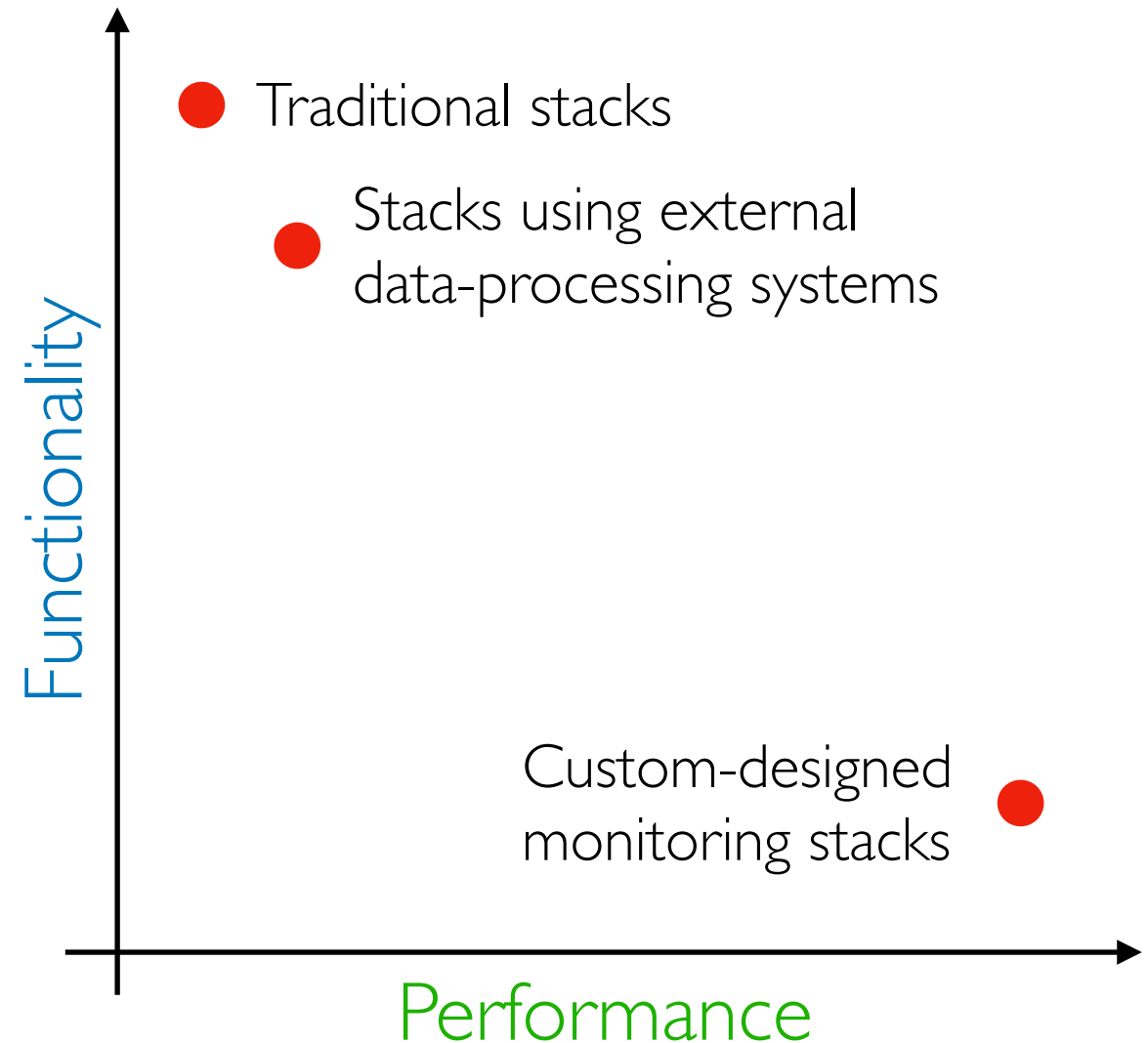
Tribeca [VLDB'96], Gigascope [SIGMOD'03],
Time Machine [SIGCOMM'08]

End-host Stacks using Stream-processing systems/Key-Value Stores

OpenSOC, Tigon, PathDump [OSDI'16],
SwitchPointer [NSDI'18]

Custom-designed monitoring stacks

FloSIS [USENIX ATC'15], Trumpet
[SIGCOMM'16]



Existing Approaches

Traditional End-host Stacks

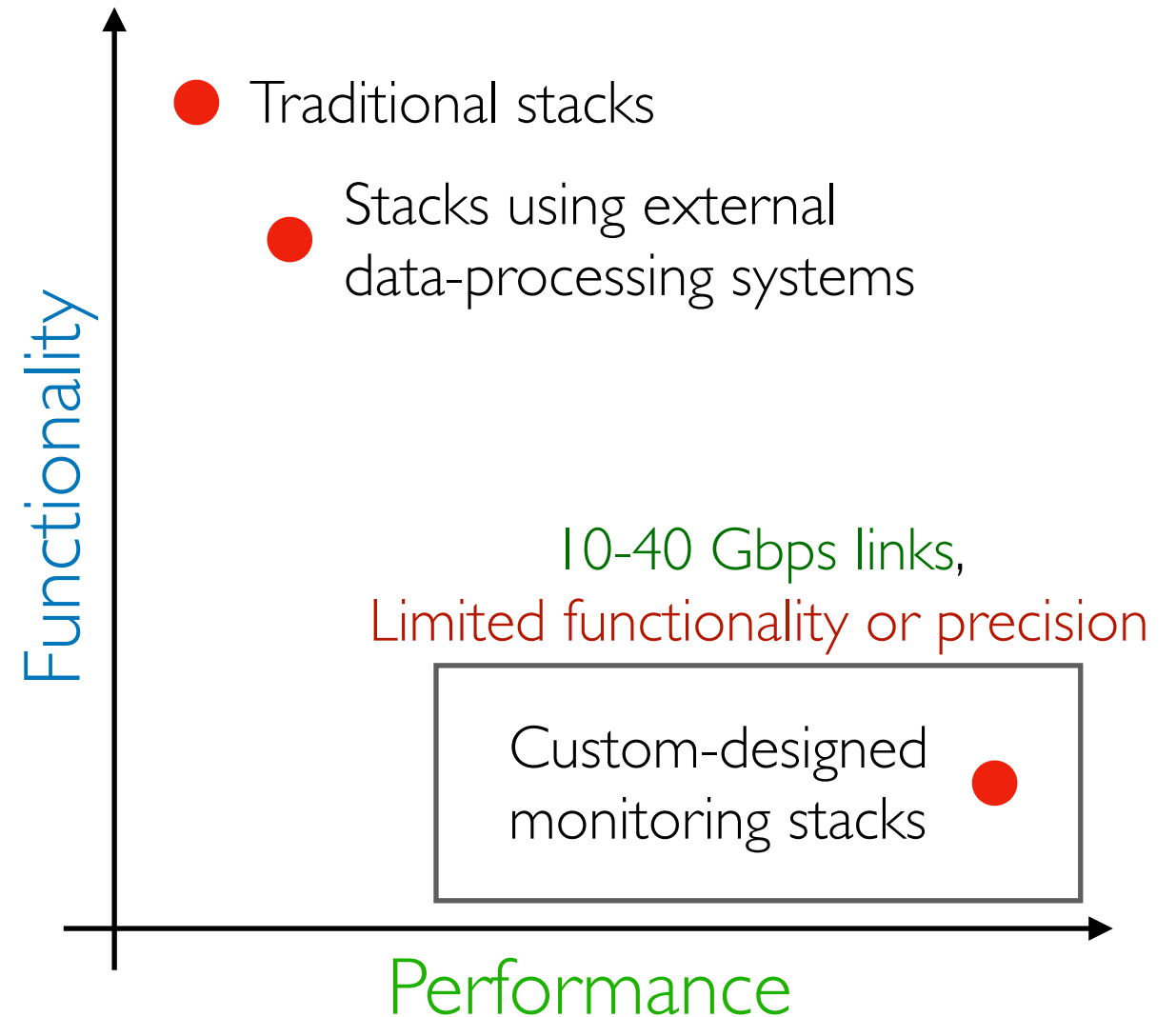
Tribeca [VLDB'96], Gigascope [SIGMOD'03],
Time Machine [SIGCOMM'08]

End-host Stacks using Stream-processing systems/Key-Value Stores

OpenSOC, Tigon, PathDump [OSDI'16],
SwitchPointer [NSDI'18]

Custom-designed monitoring stacks

FloSIS [USENIX ATC'15], Trumpet
[SIGCOMM'16]



Existing Approaches

Traditional End-host Stacks

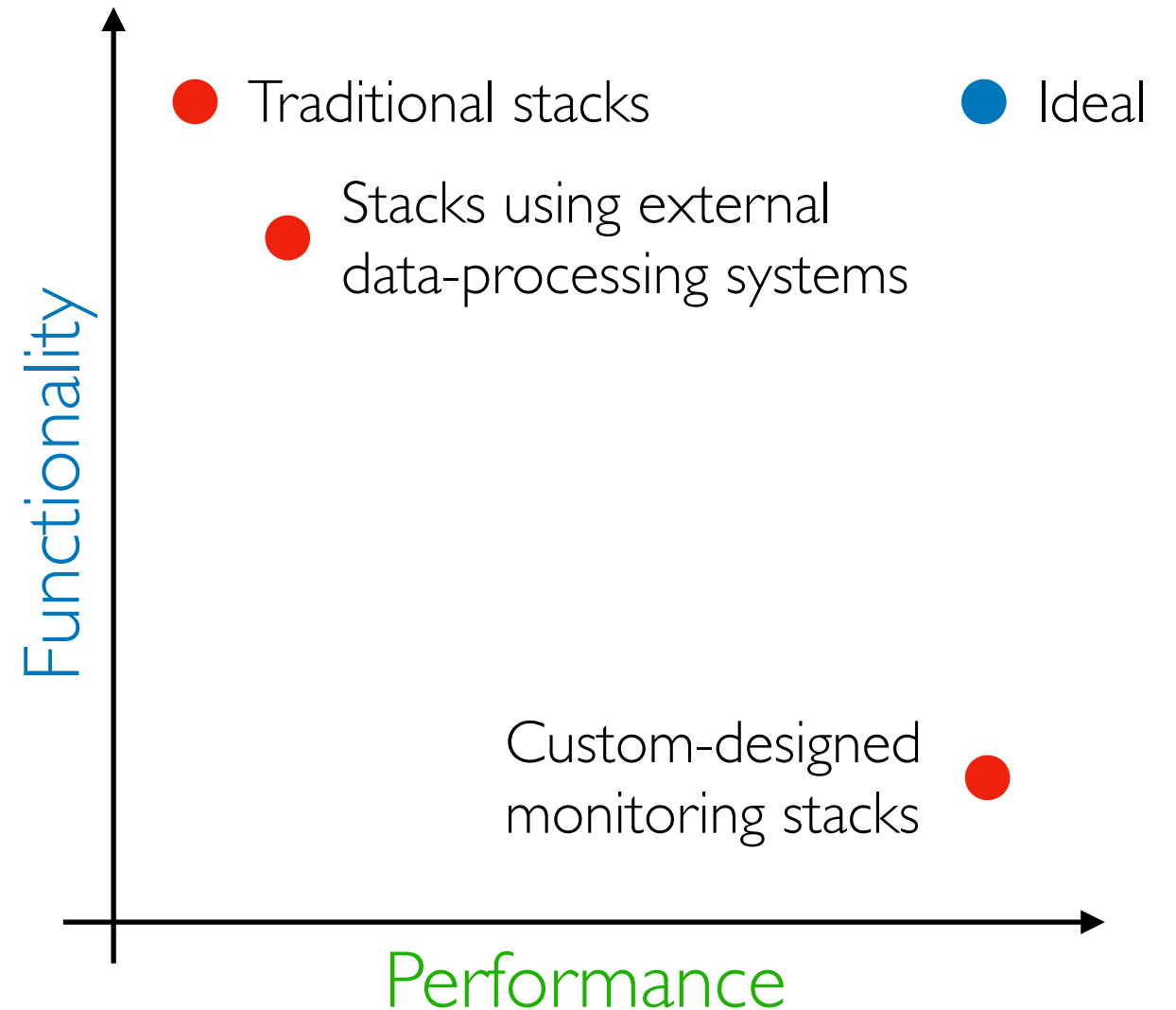
Tribeca [VLDB'96], Gigascope [SIGMOD'03],
Time Machine [SIGCOMM'08]

End-host Stacks using Stream-processing systems/Key-Value Stores

OpenSOC, Tigon, PathDump [OSDI'16],
SwitchPointer [NSDI'18]

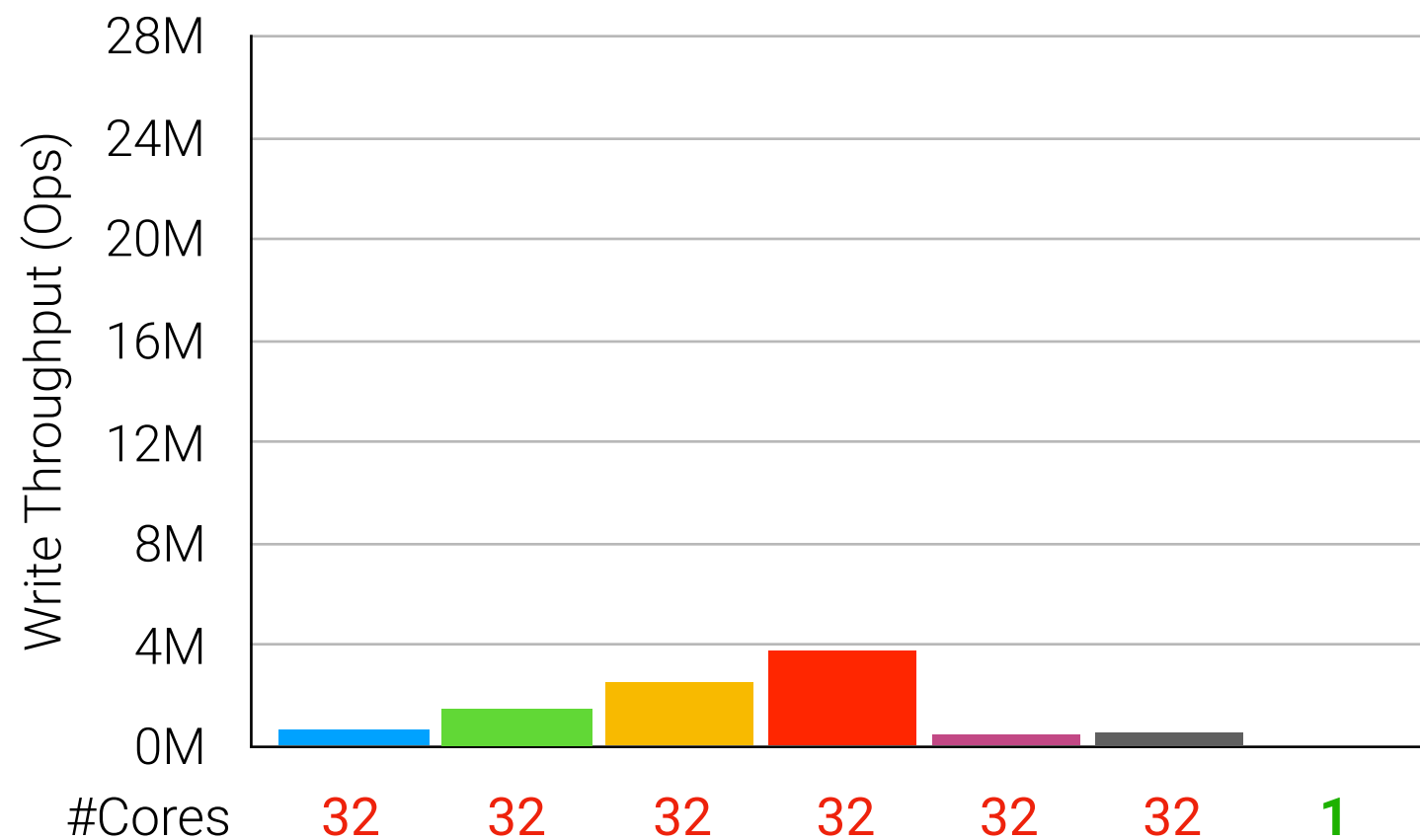
Custom-designed monitoring stacks

FloSIS [USENIX ATC'15], Trumpet
[SIGCOMM'16]

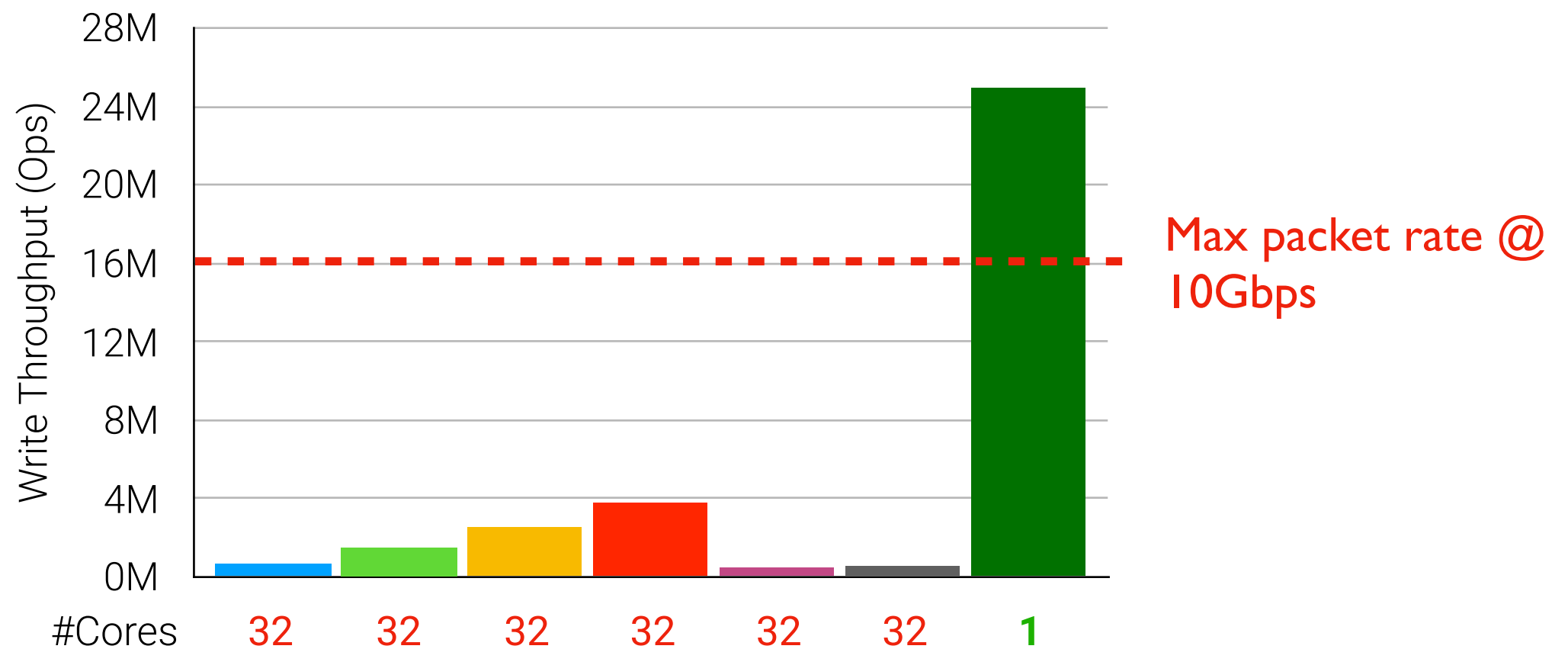


Can we achieve **both** simultaneously?

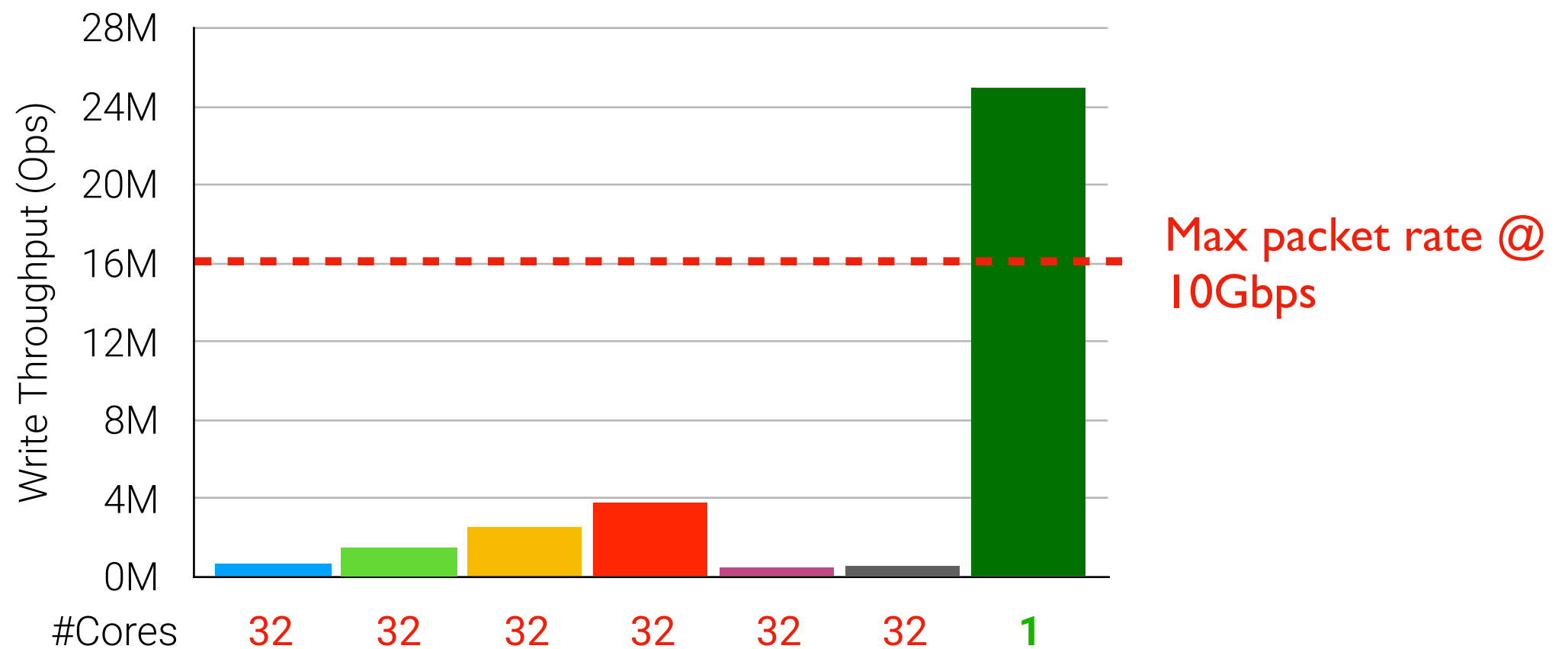
Confluo



Confluo



Confluo



Confluo achieves this using a new data structure

Atomic MultiLog

Atomic MultiLog

New data structure that exploits **structure** in telemetry data to meet all goals

Atomic MultiLog

New data structure that exploits **structure** in telemetry data to meet all goals

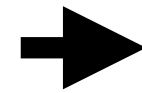
Attributes of interest are *fixed-sized*

32-bit IP addresses, timestamps, 16-bit port numbers, switchIDs, queue-lengths, etc.

Atomic MultiLog

New data structure that exploits **structure** in telemetry data to meet all goals

Attributes of interest are *fixed-sized*



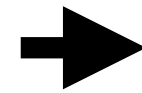
Low-overhead indexing with
specialized perfect k-ary trees

32-bit IP addresses, timestamps, 16-bit port
numbers, switchIDs, queue-lengths, etc.

Atomic MultiLog

New data structure that exploits **structure** in telemetry data to meet all goals

Attributes of interest are *fixed-sized*



Low-overhead indexing with
specialized perfect k-ary trees

32-bit IP addresses, timestamps, 16-bit port
numbers, switchIDs, queue-lengths, etc.

Data once written is *not updated*

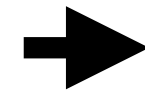
Aggregated only at coarse-grained timescales.

Atomic MultiLog

New data structure that exploits **structure** in telemetry data to meet all goals

Attributes of interest are *fixed-sized*

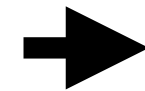
32-bit IP addresses, timestamps, 16-bit port numbers, switchIDs, queue-lengths, etc.



Low-overhead indexing with specialized perfect k-ary trees

Data once written is *not updated*

Aggregated only at coarse-grained timescales.



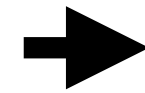
Append-only write-efficient data structures

Atomic MultiLog

New data structure that exploits **structure** in telemetry data to meet all goals

Attributes of interest are *fixed-sized*

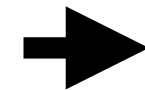
32-bit IP addresses, timestamps, 16-bit port numbers, switchIDs, queue-lengths, etc.



Low-overhead indexing with specialized perfect k-ary trees

Data once written is *not updated*

Aggregated only at coarse-grained timescales.



Append-only write-efficient data structures

Do not require serializable transactions, *linearizability is sufficient*

Linearizability: *single-operation, single-object*

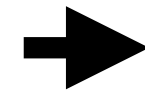
Serializability: *multi-operation, multi-object*

Atomic MultiLog

New data structure that exploits **structure** in telemetry data to meet all goals

Attributes of interest are *fixed-sized*

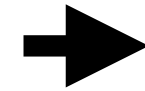
32-bit IP addresses, timestamps, 16-bit port numbers, switchIDs, queue-lengths, etc.



Low-overhead indexing with specialized perfect k-ary trees

Data once written is *not updated*

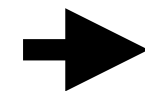
Aggregated only at coarse-grained timescales.



Append-only write-efficient data structures

Do not require serializable transactions, *linearizability is sufficient*

Linearizability: *single-operation, single-object*
Serializability: *multi-operation, multi-object*



Trim down concurrency mechanisms to updating 2 integers

Atomic MultiLog: Write Efficient Storage

Atomic MultiLog: Write Efficient Storage

Traditional Data Stores: Use complex data structures to support general workloads, compromising on write efficiency

Atomic MultiLog: Write Efficient Storage

Data once written is *not updated*

Atomic MultiLog: Write Efficient Storage

Data once written is *not updated*

HEADER LOG

<HEADER1>

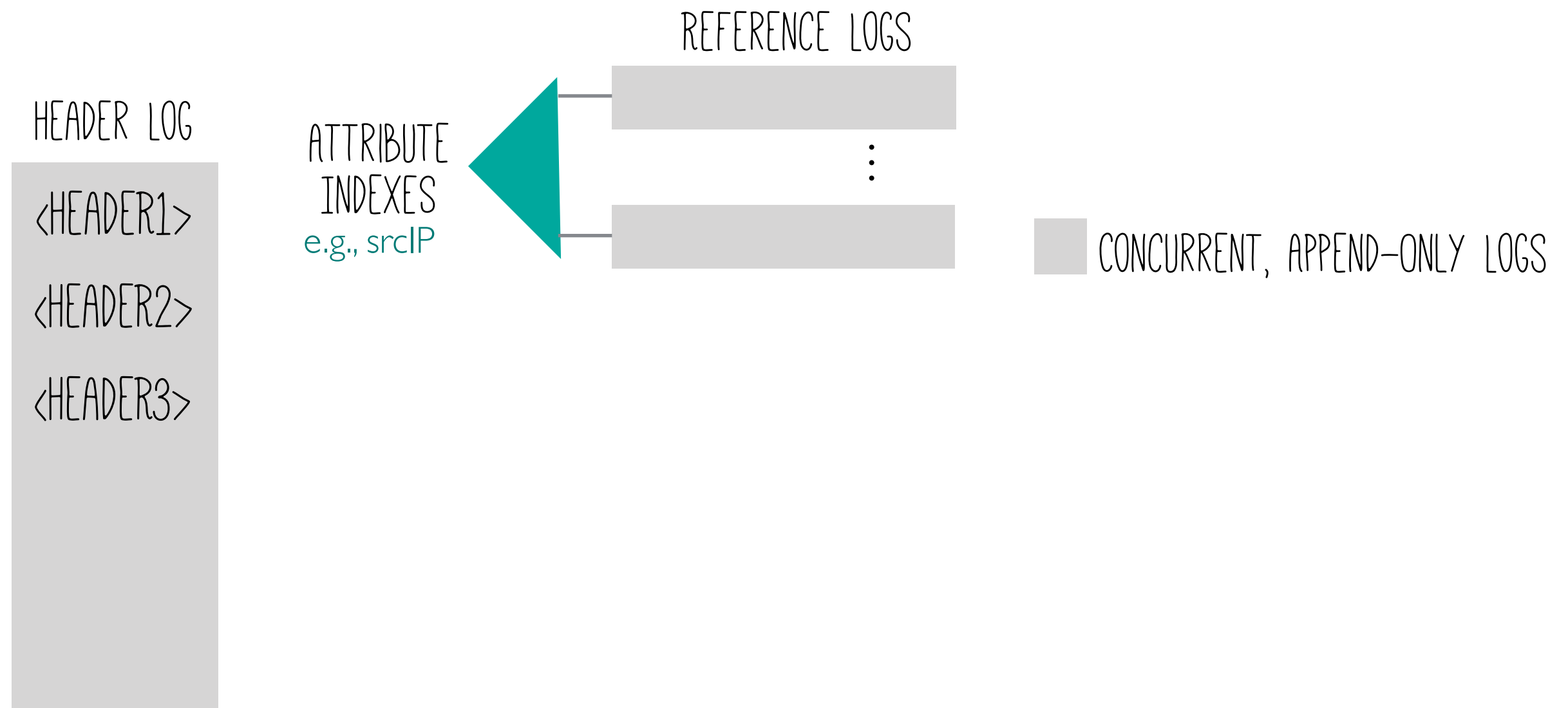
<HEADER2>

<HEADER3>

■ CONCURRENT, APPEND-ONLY LOGS

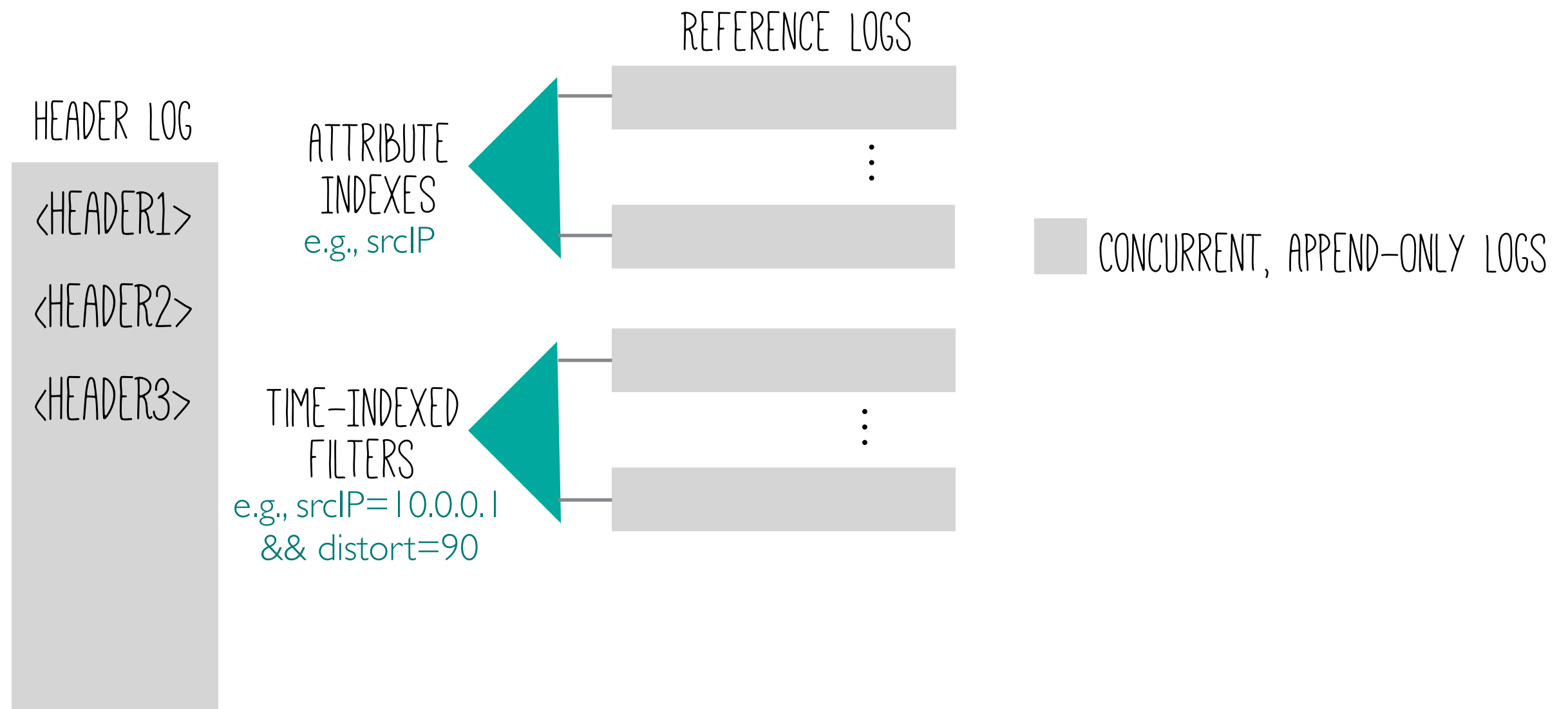
Atomic MultiLog: Write Efficient Storage

Data once written is *not updated*



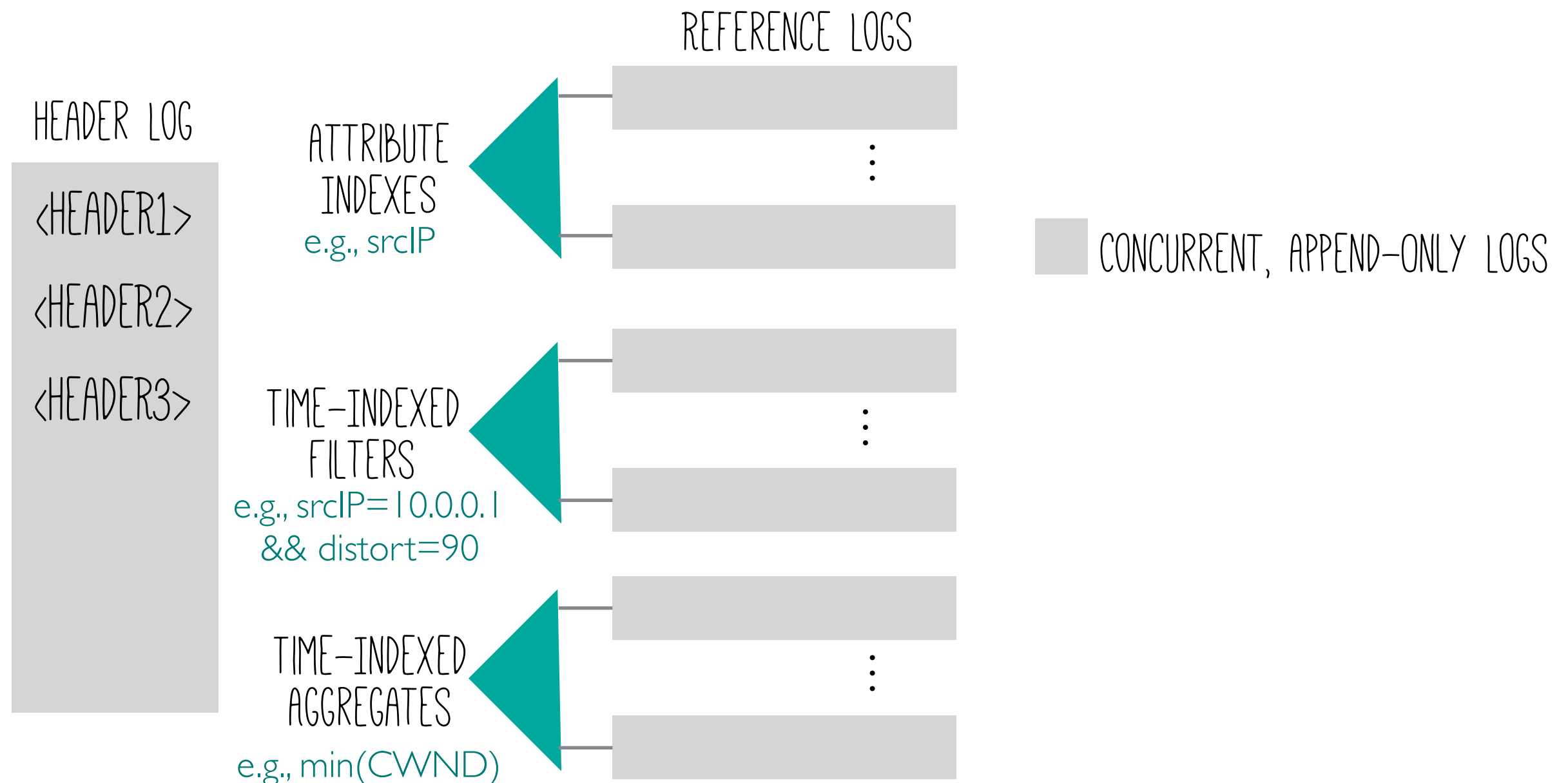
Atomic MultiLog: Write Efficient Storage

Data once written is *not updated*



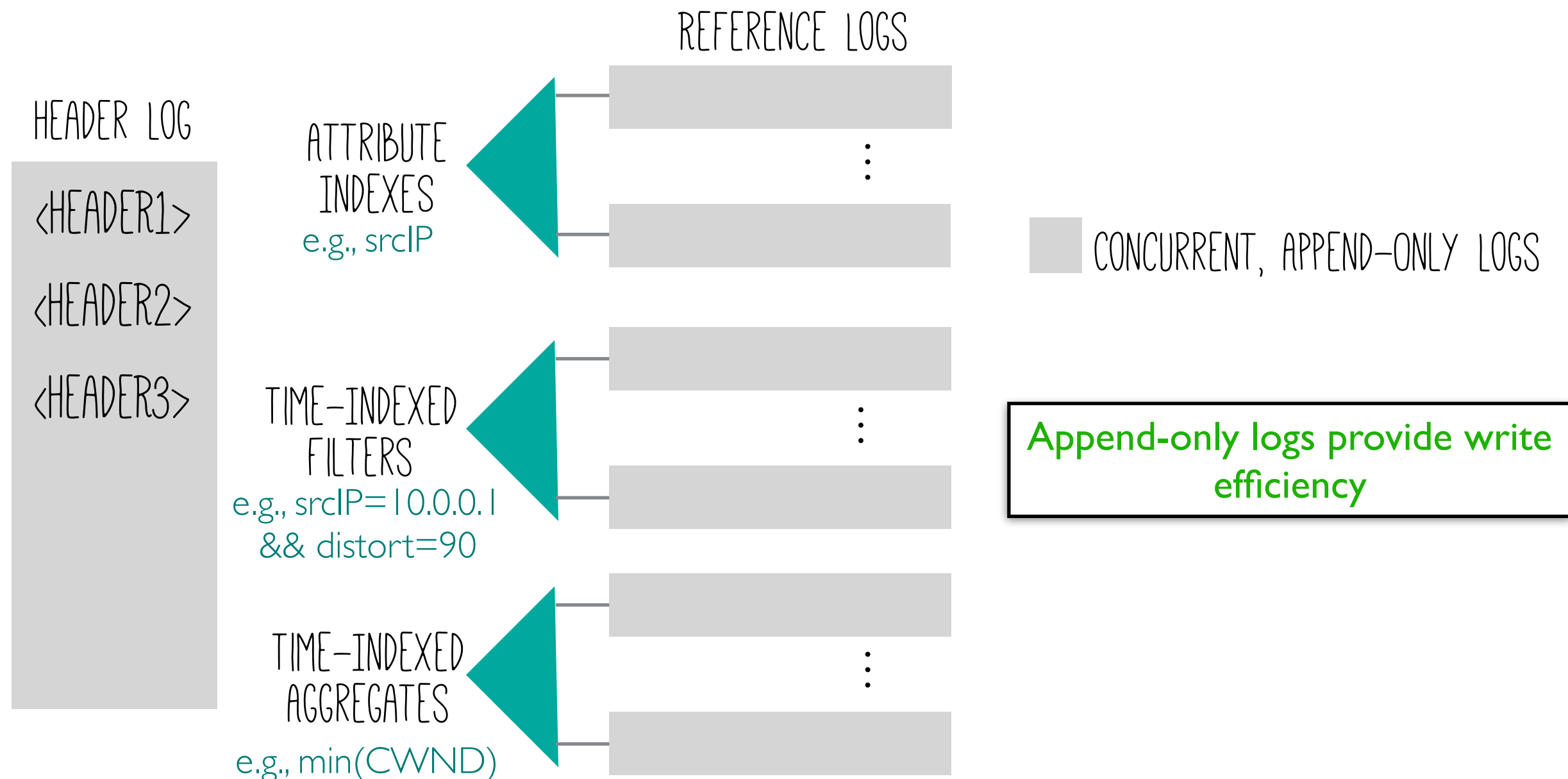
Atomic MultiLog: Write Efficient Storage

Data once written is *not updated*



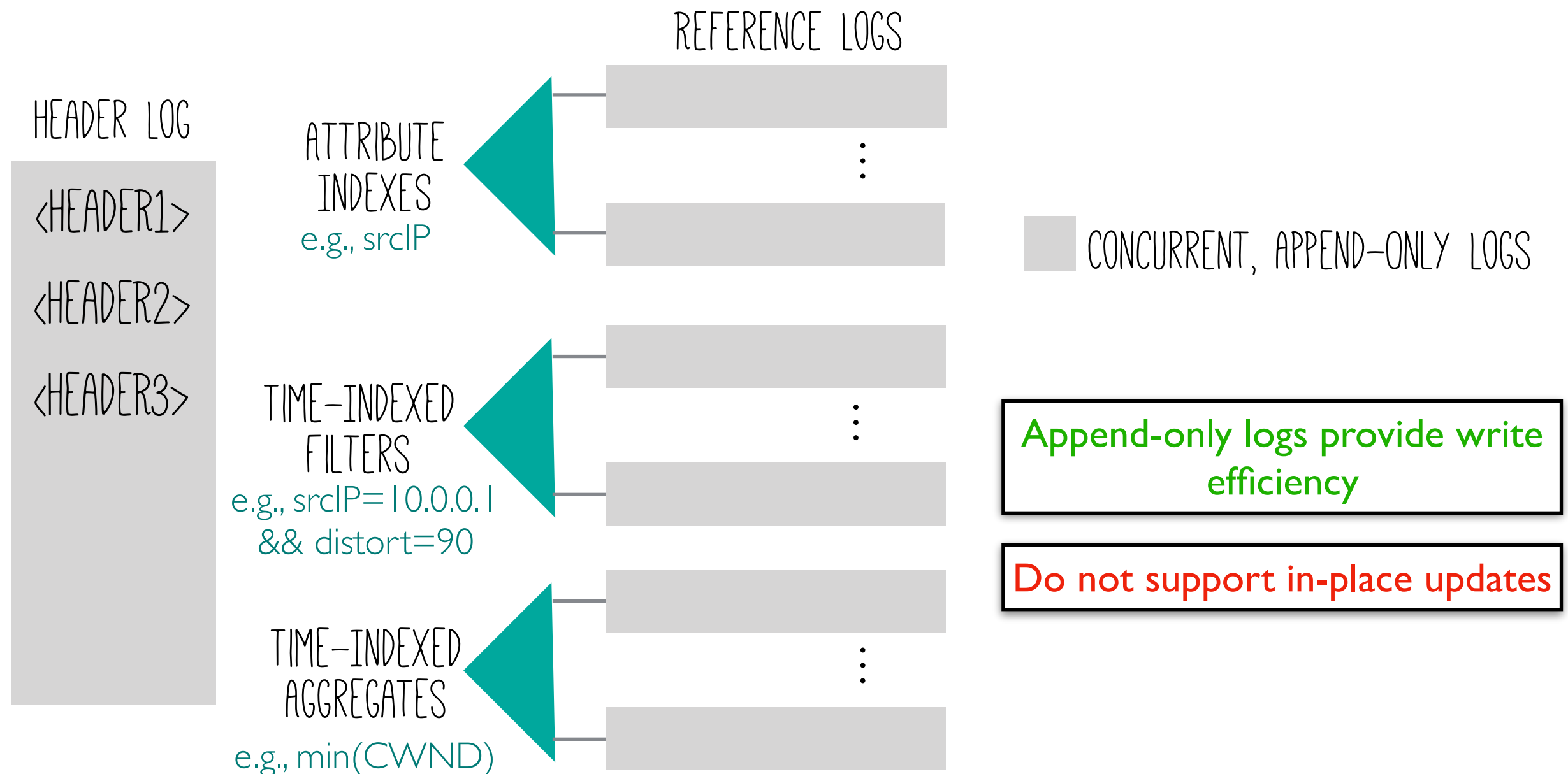
Atomic MultiLog: Write Efficient Storage

Data once written is *not updated*



Atomic MultiLog: Write Efficient Storage

Data once written is *not updated*

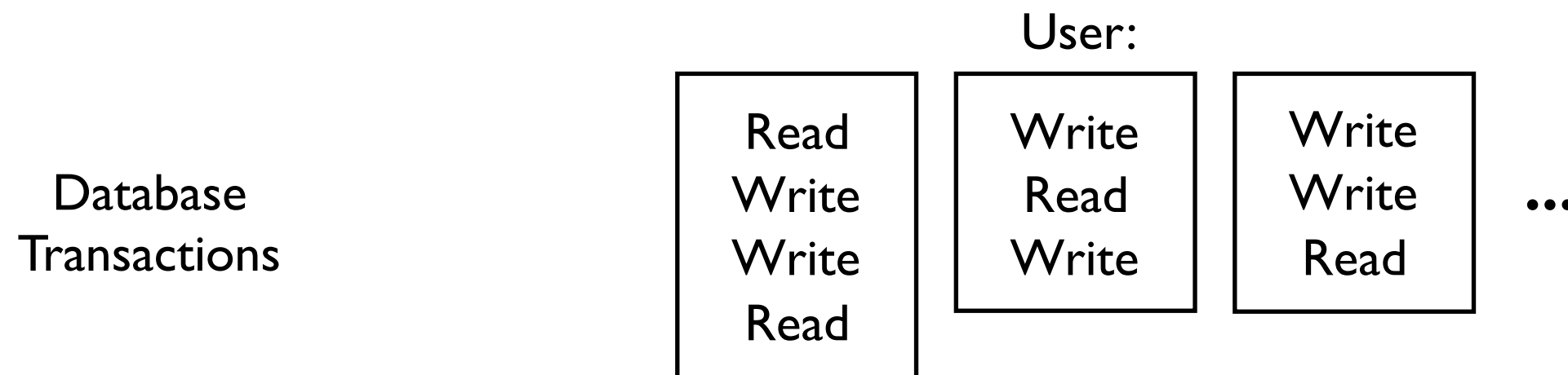


Atomic MultiLog Consistency

Atomic MultiLog Consistency

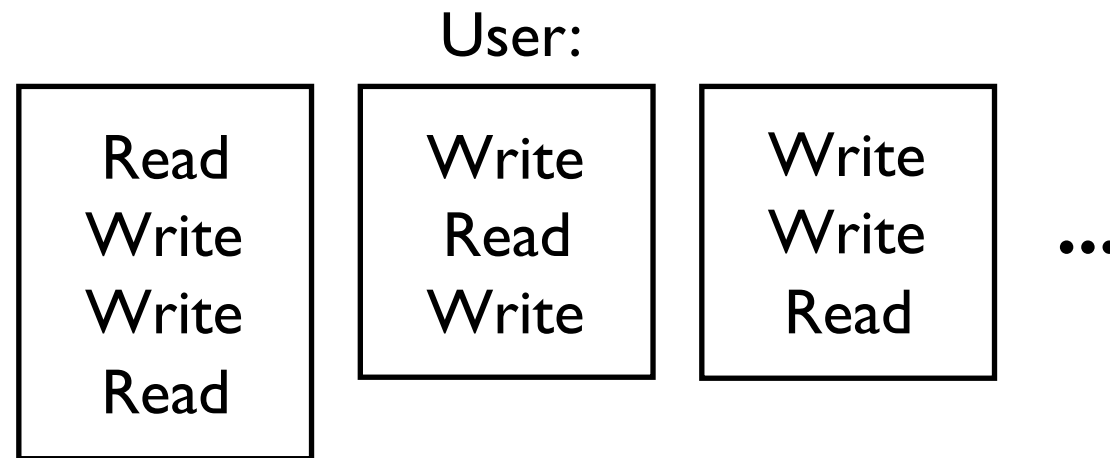
Database
Transactions

Atomic MultiLog Consistency



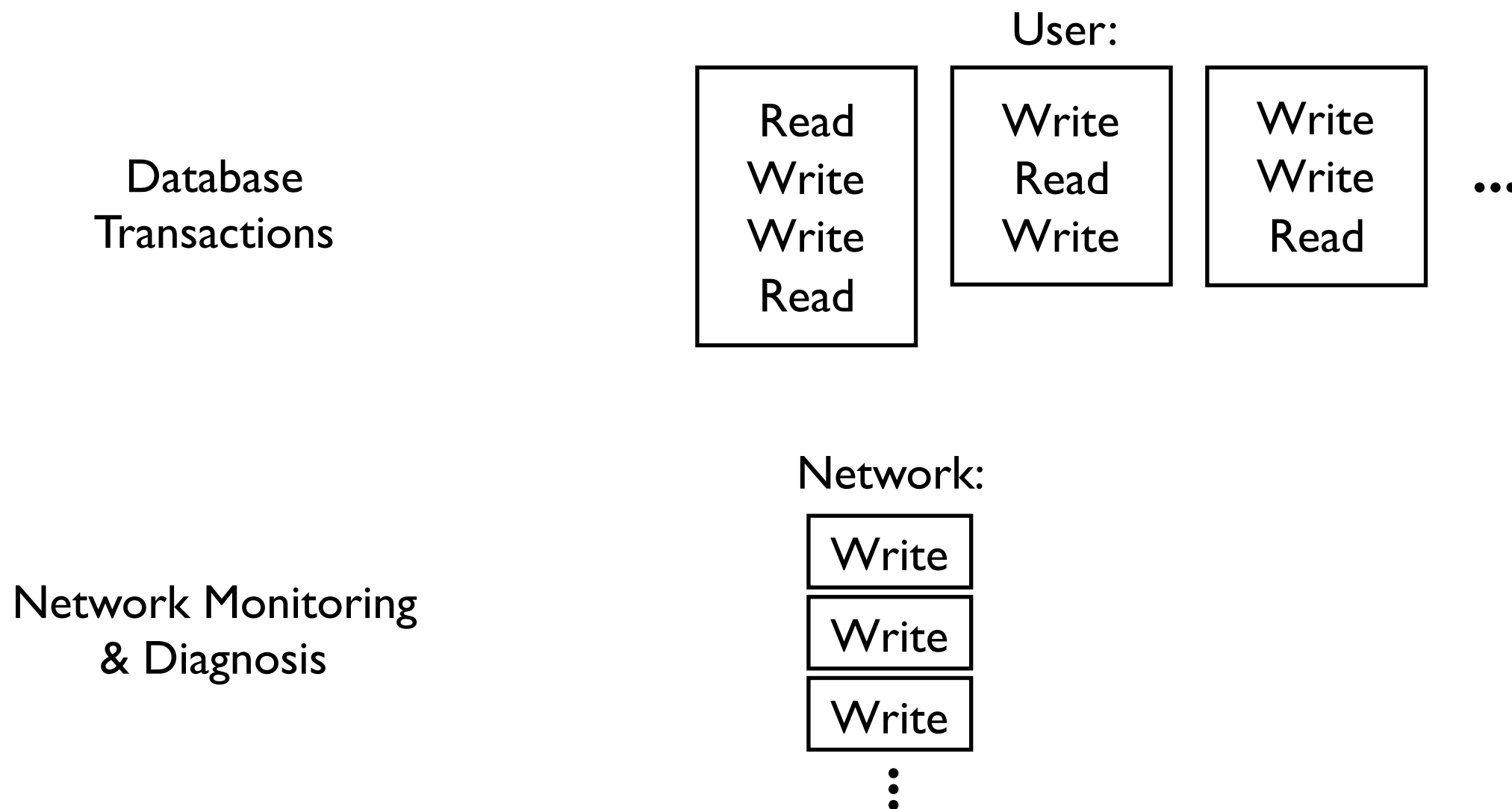
Atomic MultiLog Consistency

Database
Transactions

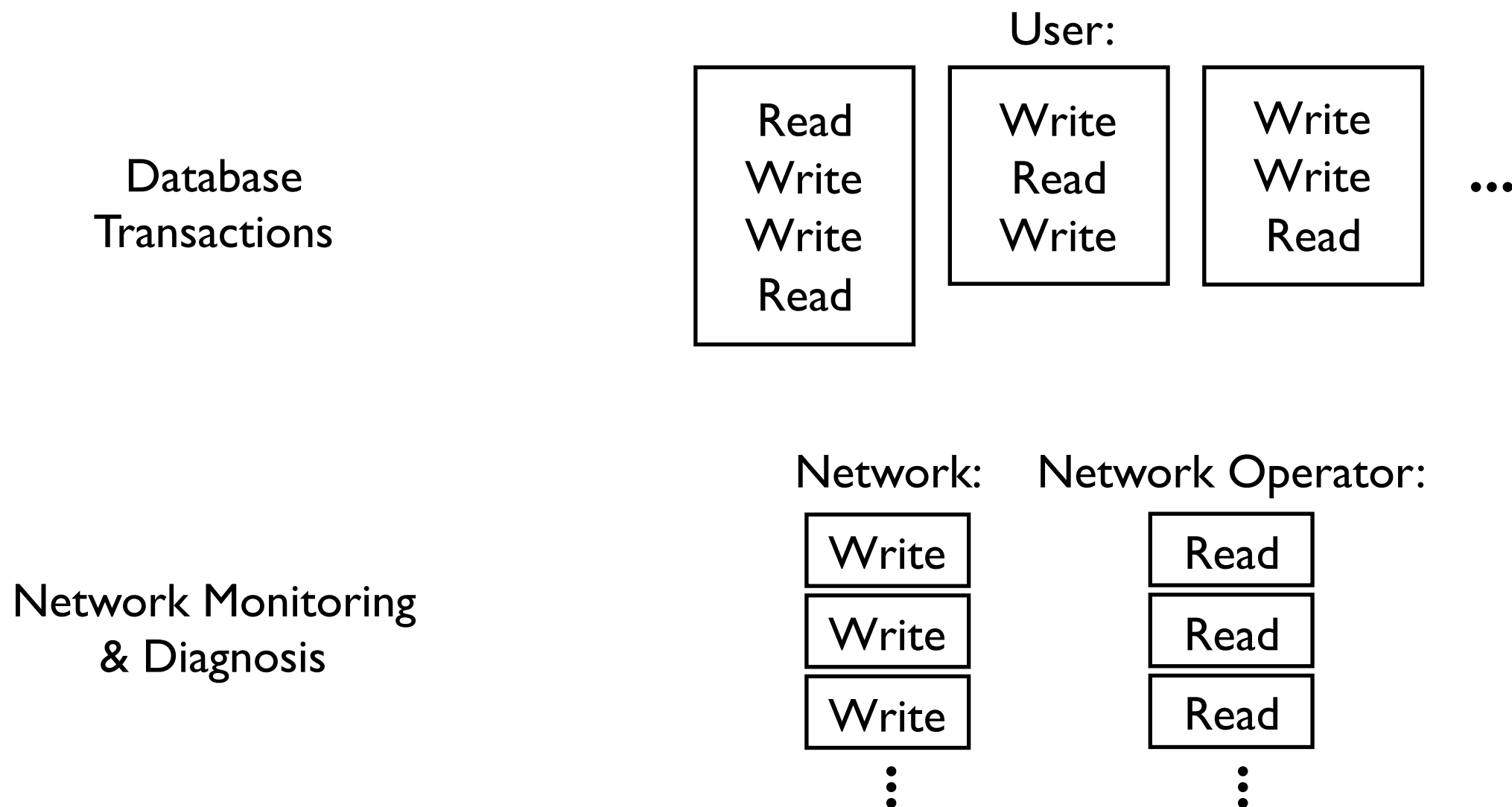


Network Monitoring
& Diagnosis

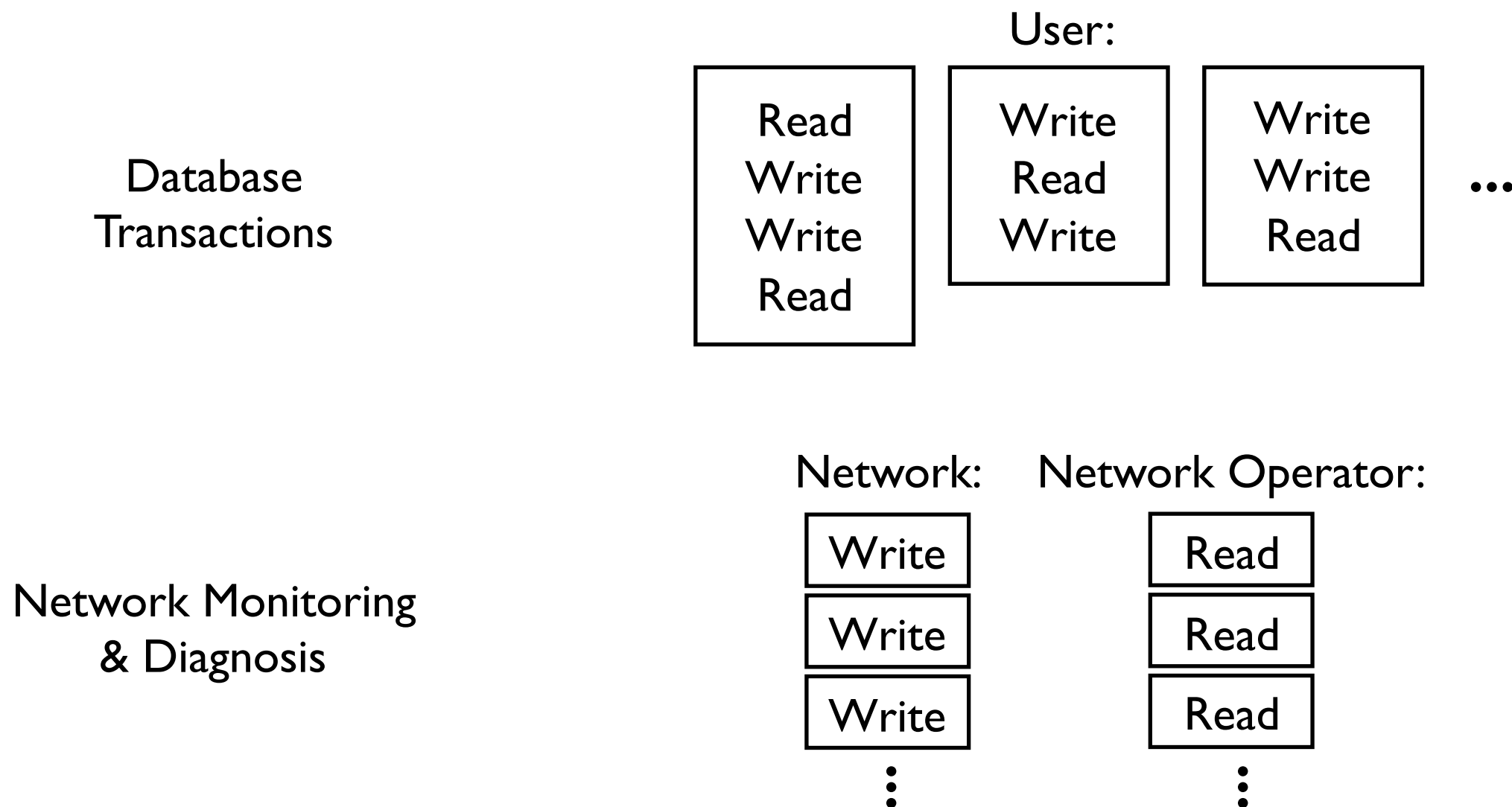
Atomic MultiLog Consistency



Atomic MultiLog Consistency



Atomic MultiLog Consistency



Do not require serializable transactions, *linearizability is sufficient*

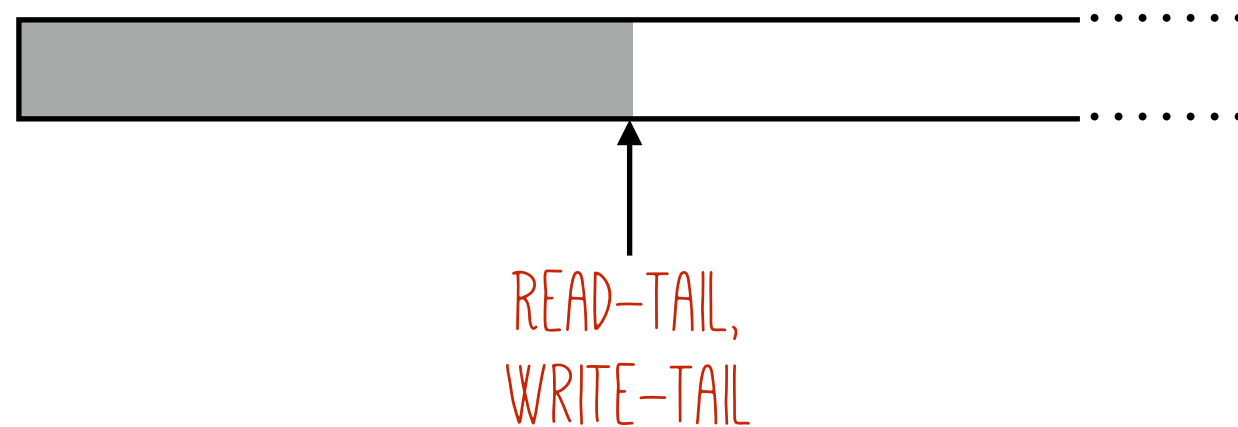
Efficient Linearizability for Logs

Support for concurrent appends & reads



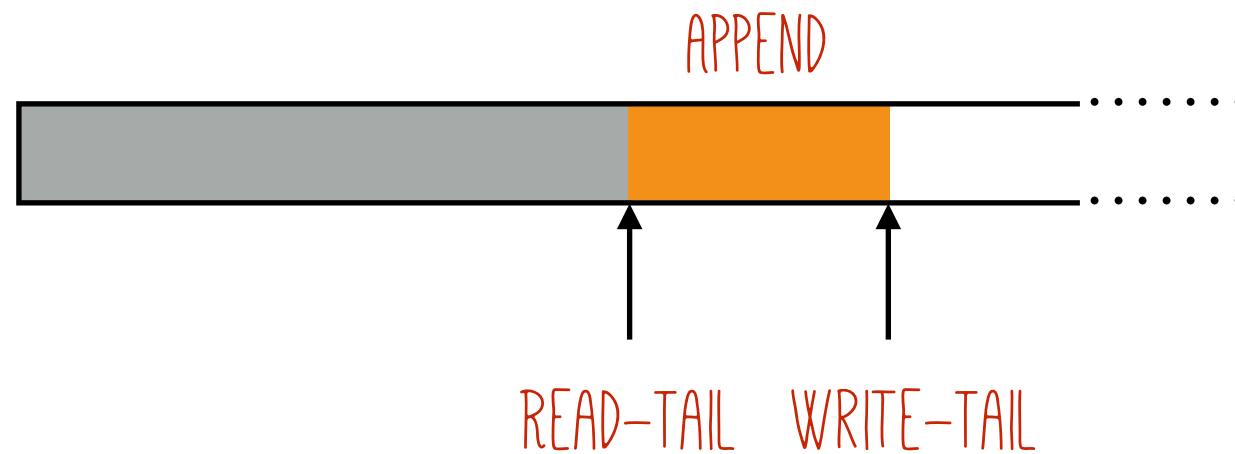
Efficient Linearizability for Logs

Support for concurrent appends & reads



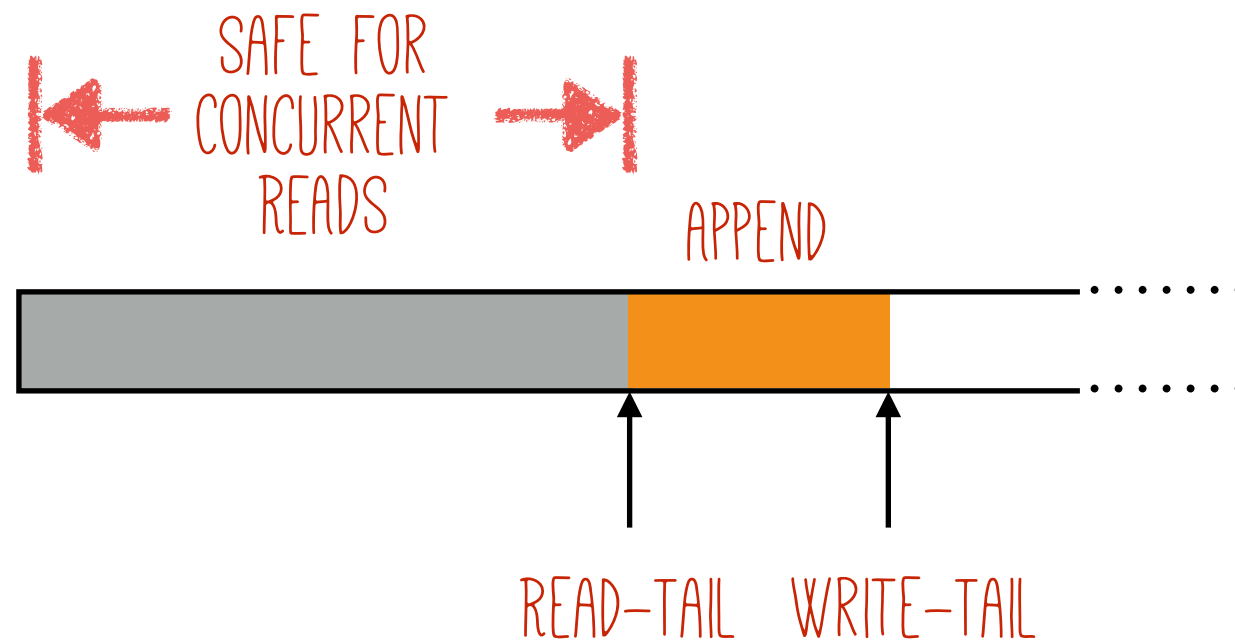
Efficient Linearizability for Logs

Support for concurrent appends & reads



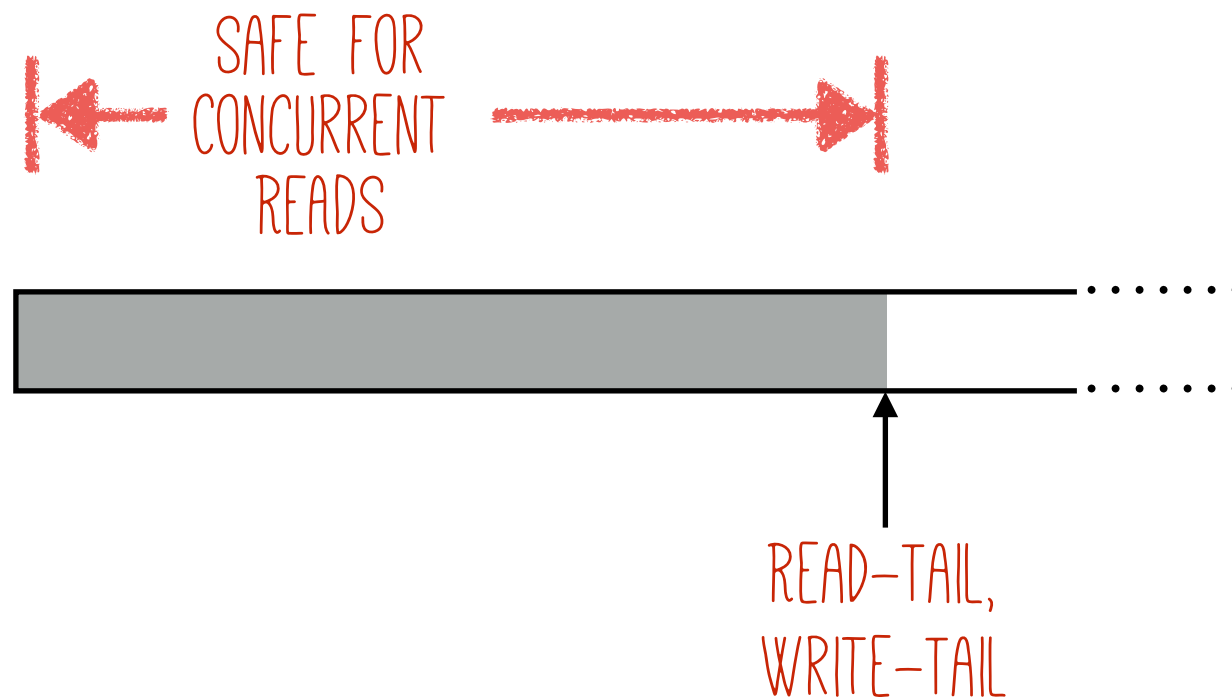
Efficient Linearizability for Logs

Support for concurrent appends & reads



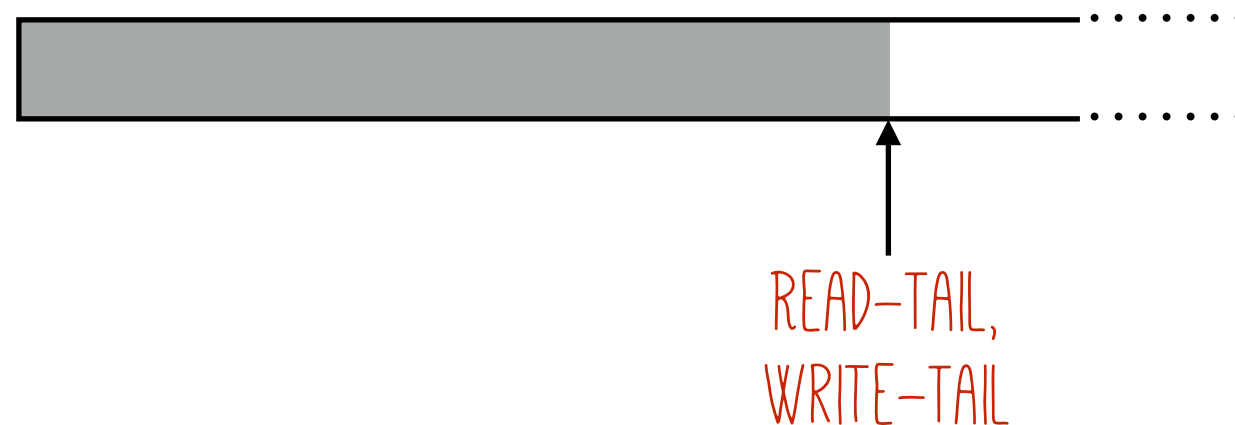
Efficient Linearizability for Logs

Support for concurrent appends & reads



Efficient Linearizability for Logs

Support for concurrent appends & reads



Linearizable reads & appends

Efficient Linearizability for Logs

Support for concurrent appends & reads

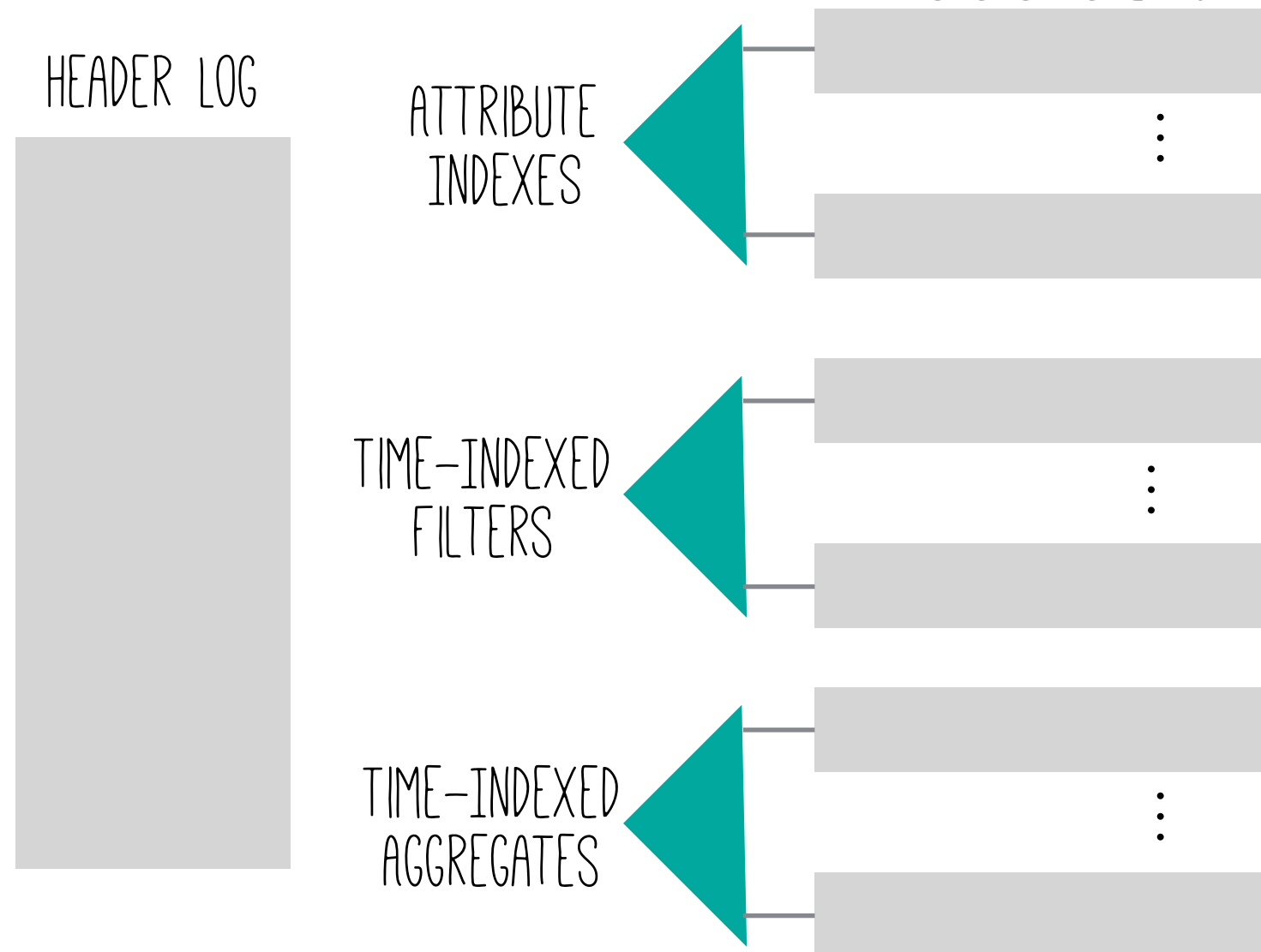


Linearizable reads & appends

Lock-free techniques for efficiency

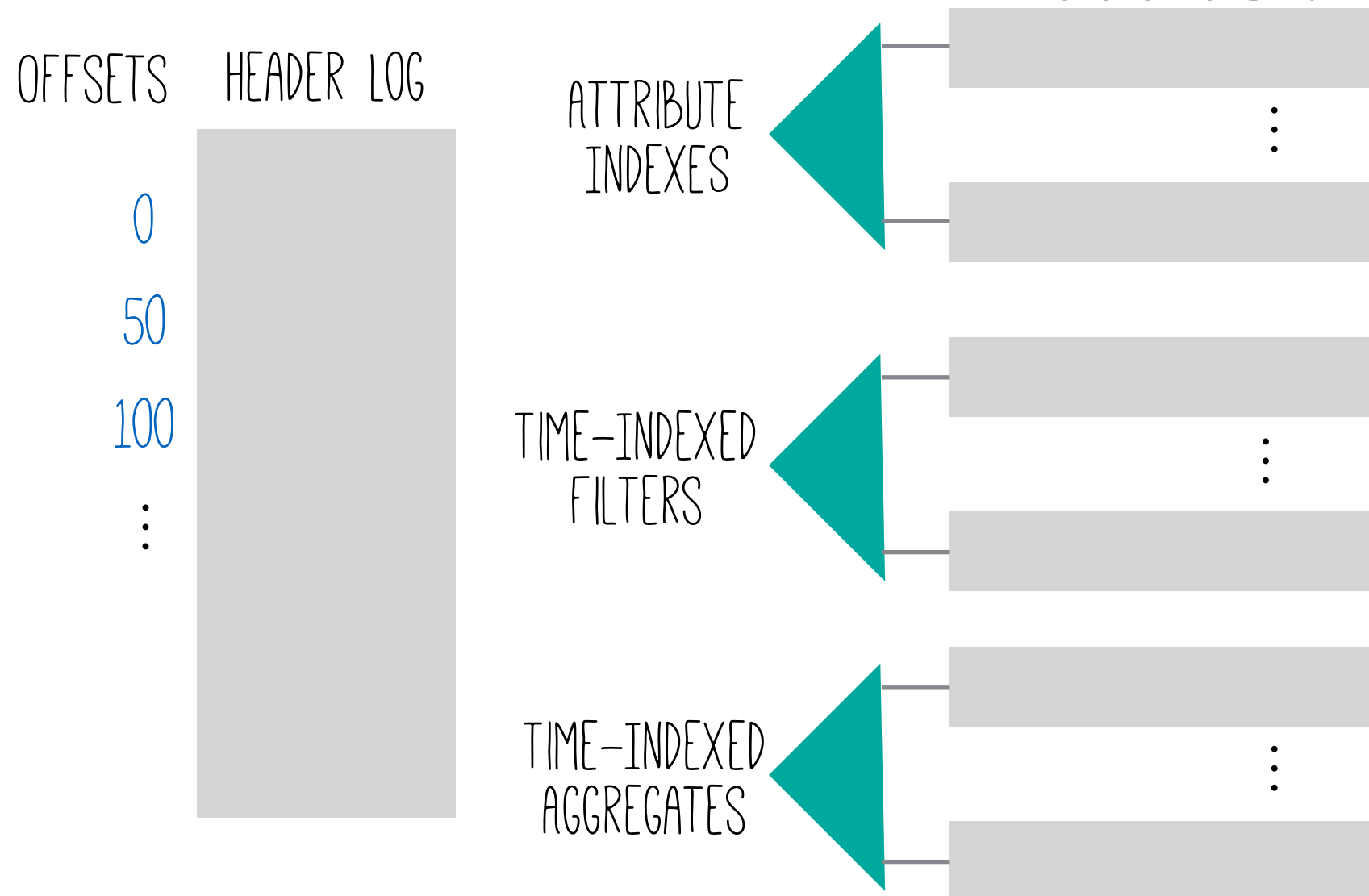
Atomic MultiLog Linearizability

ATOMIC MULTILOG



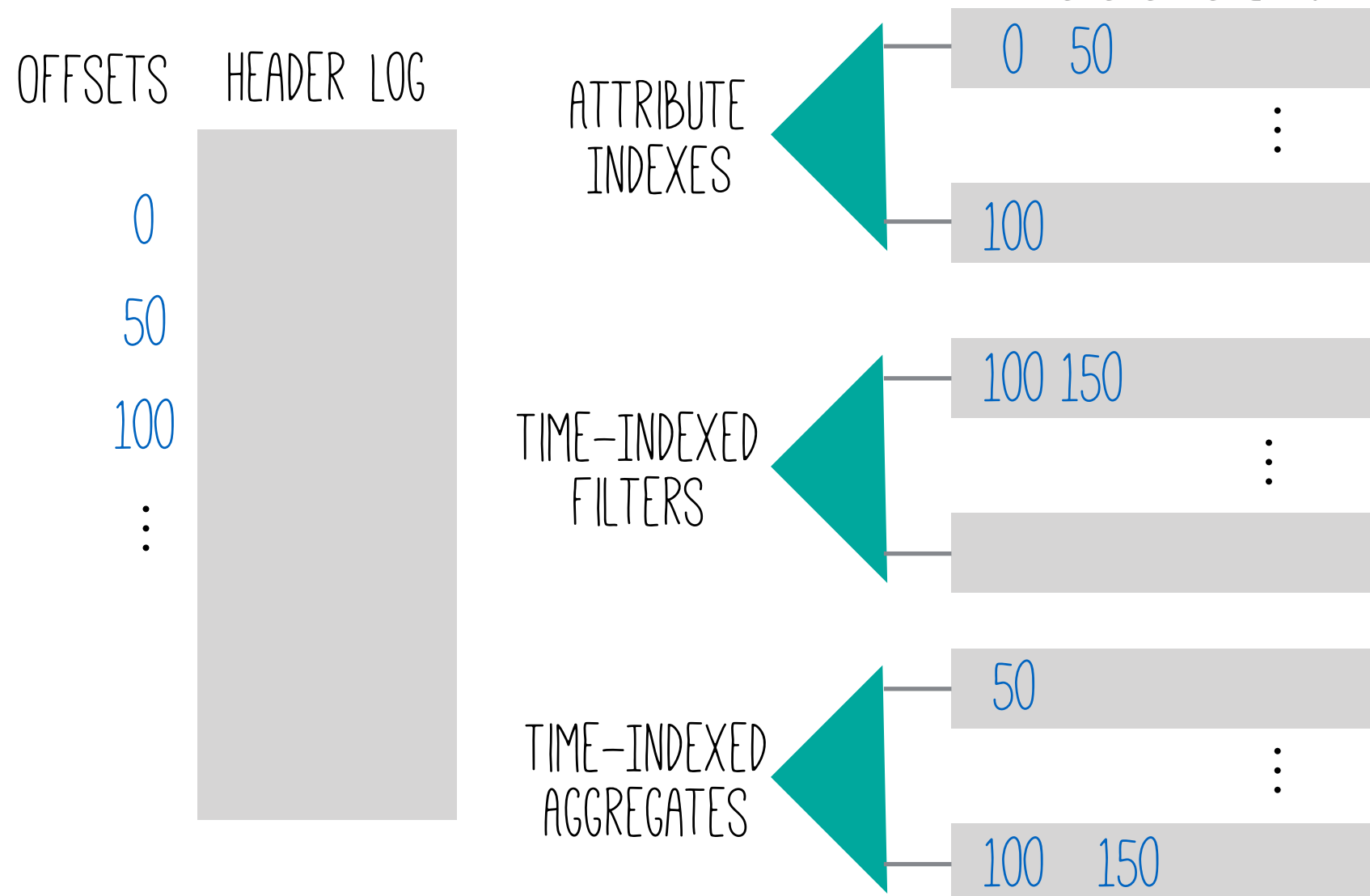
Atomic MultiLog Linearizability

ATOMIC MULTILOG



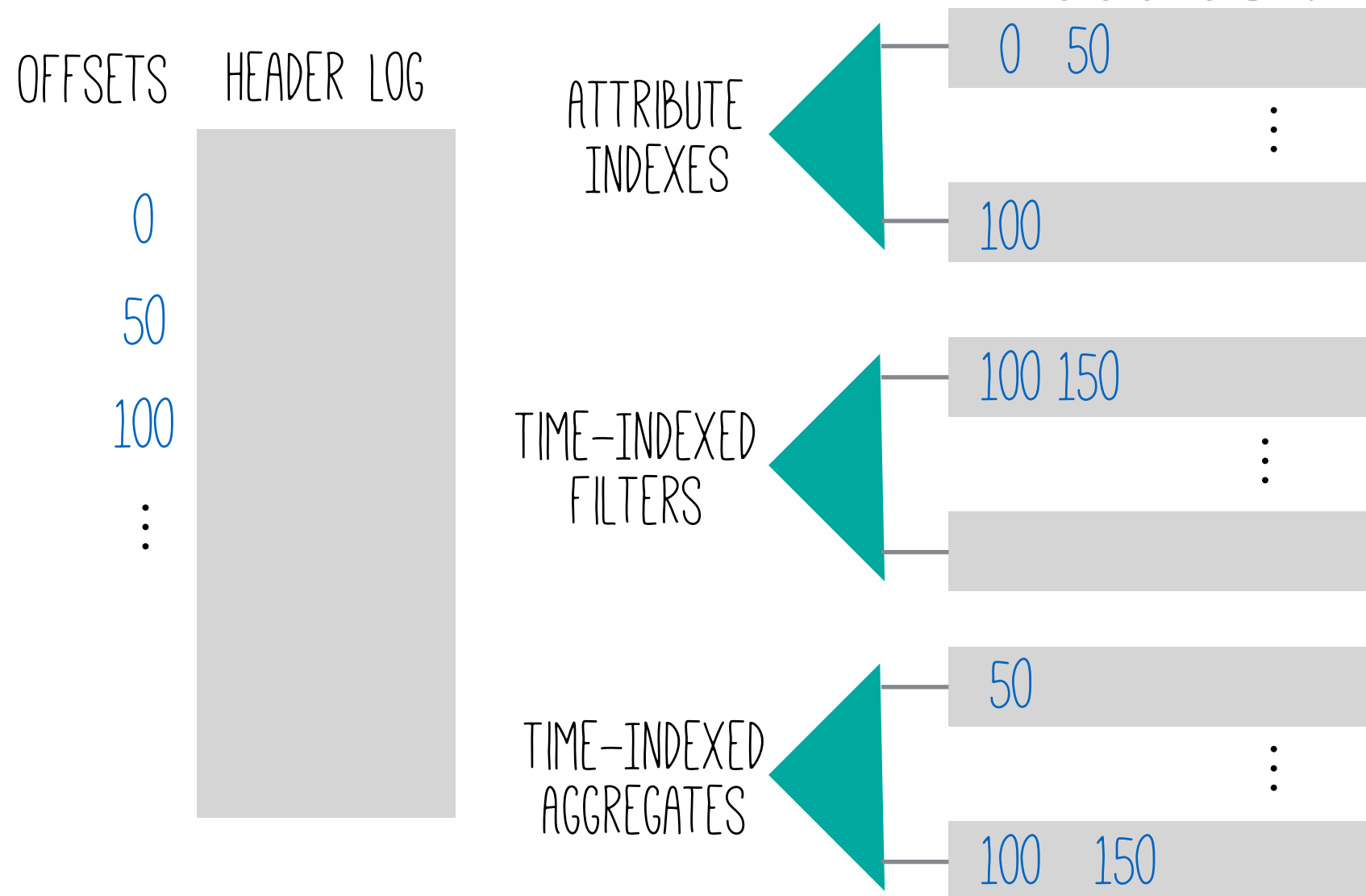
Atomic MultiLog Linearizability

ATOMIC MULTILOG



Atomic MultiLog Linearizability

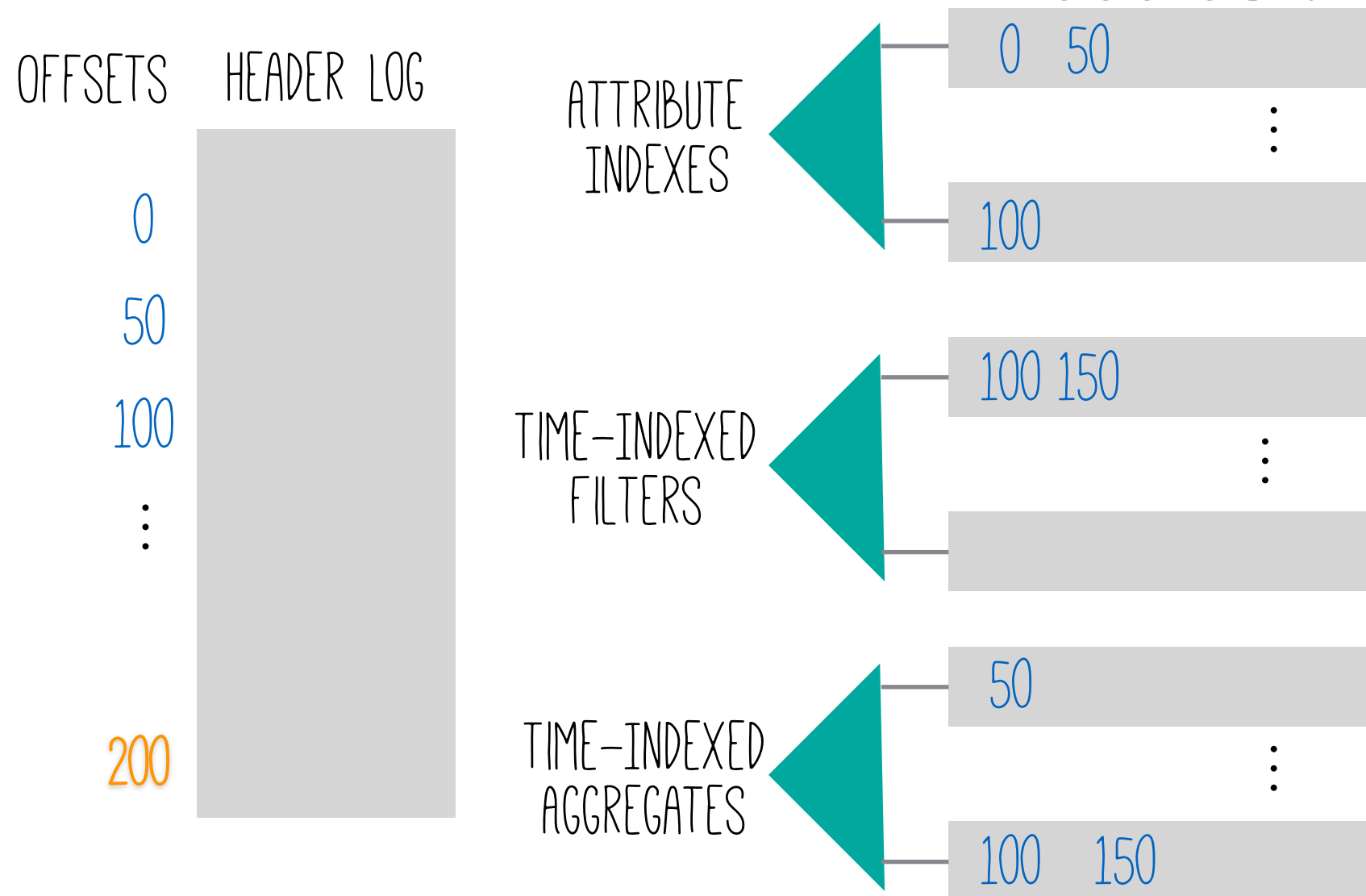
ATOMIC MULTILOG



GLOBAL READ TAIL 200
GLOBAL WRITE TAIL 200 } OFFSETS

Atomic MultiLog Linearizability

ATOMIC MULTILOG

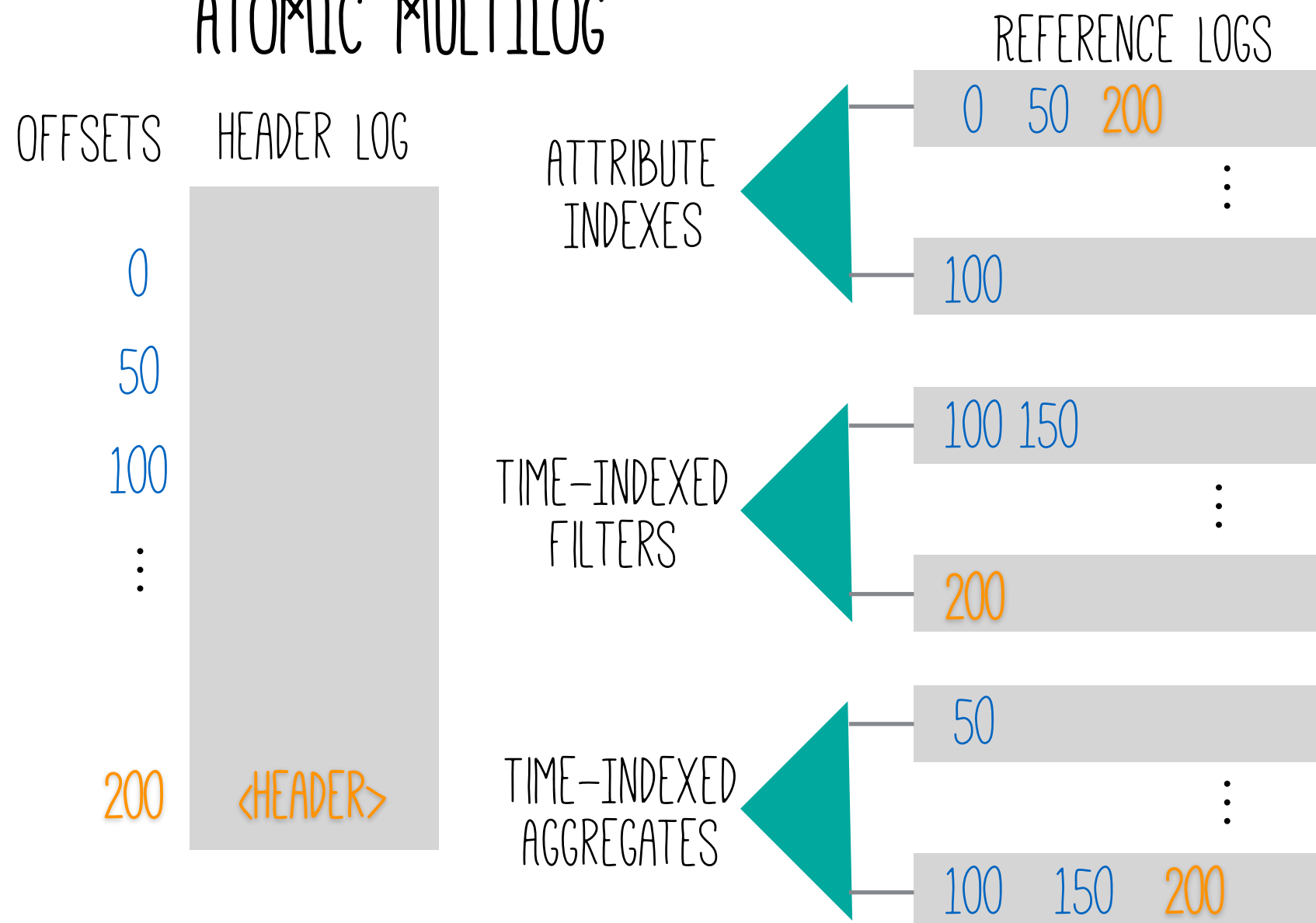


GLOBAL READ TAIL 200
GLOBAL WRITE TAIL 250

} OFFSETS

Atomic MultiLog Linearizability

ATOMIC MULTILOG

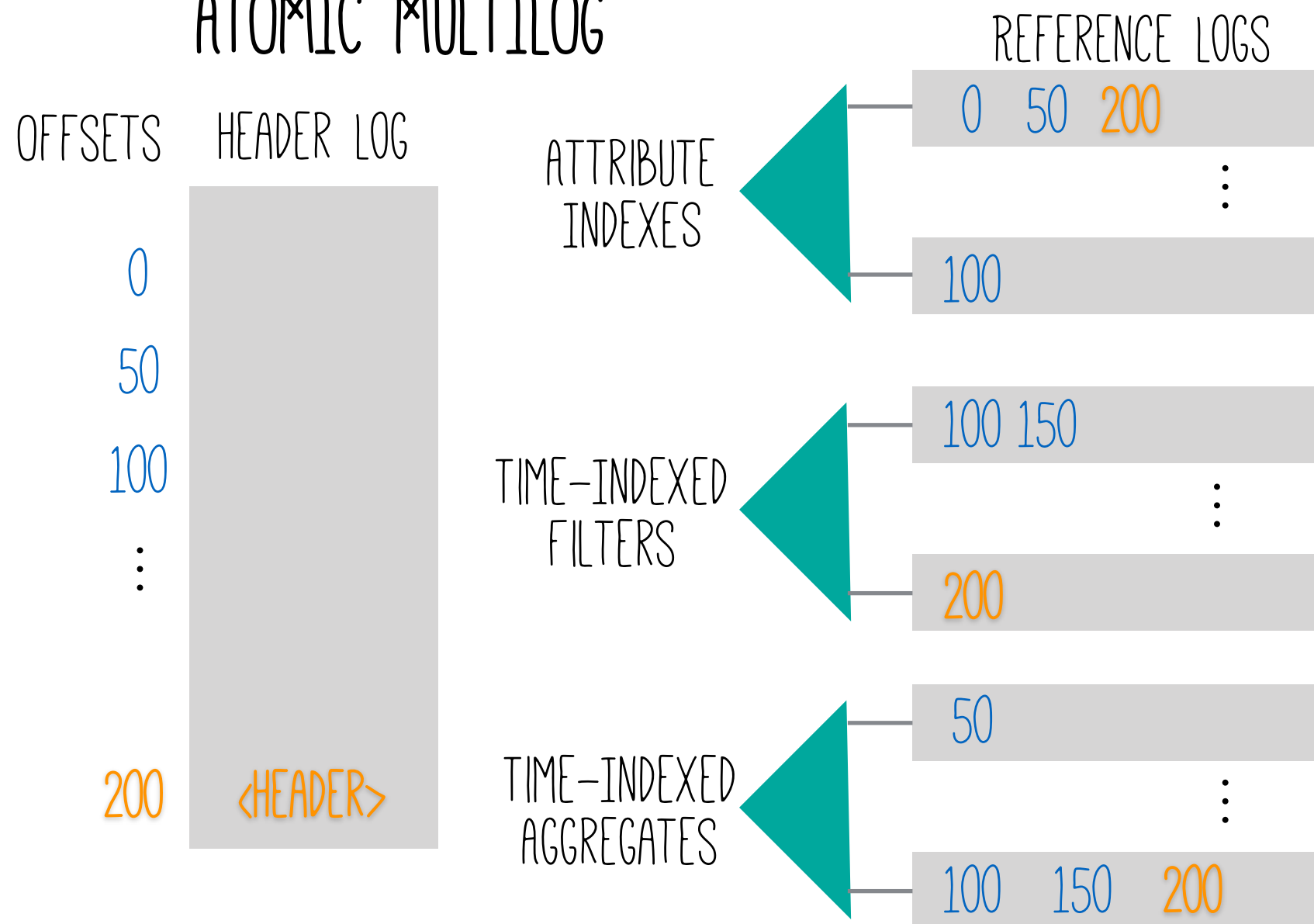


GLOBAL READ TAIL 200
GLOBAL WRITE TAIL 250

} OFFSETS

Atomic MultiLog Linearizability

ATOMIC MULTILOG

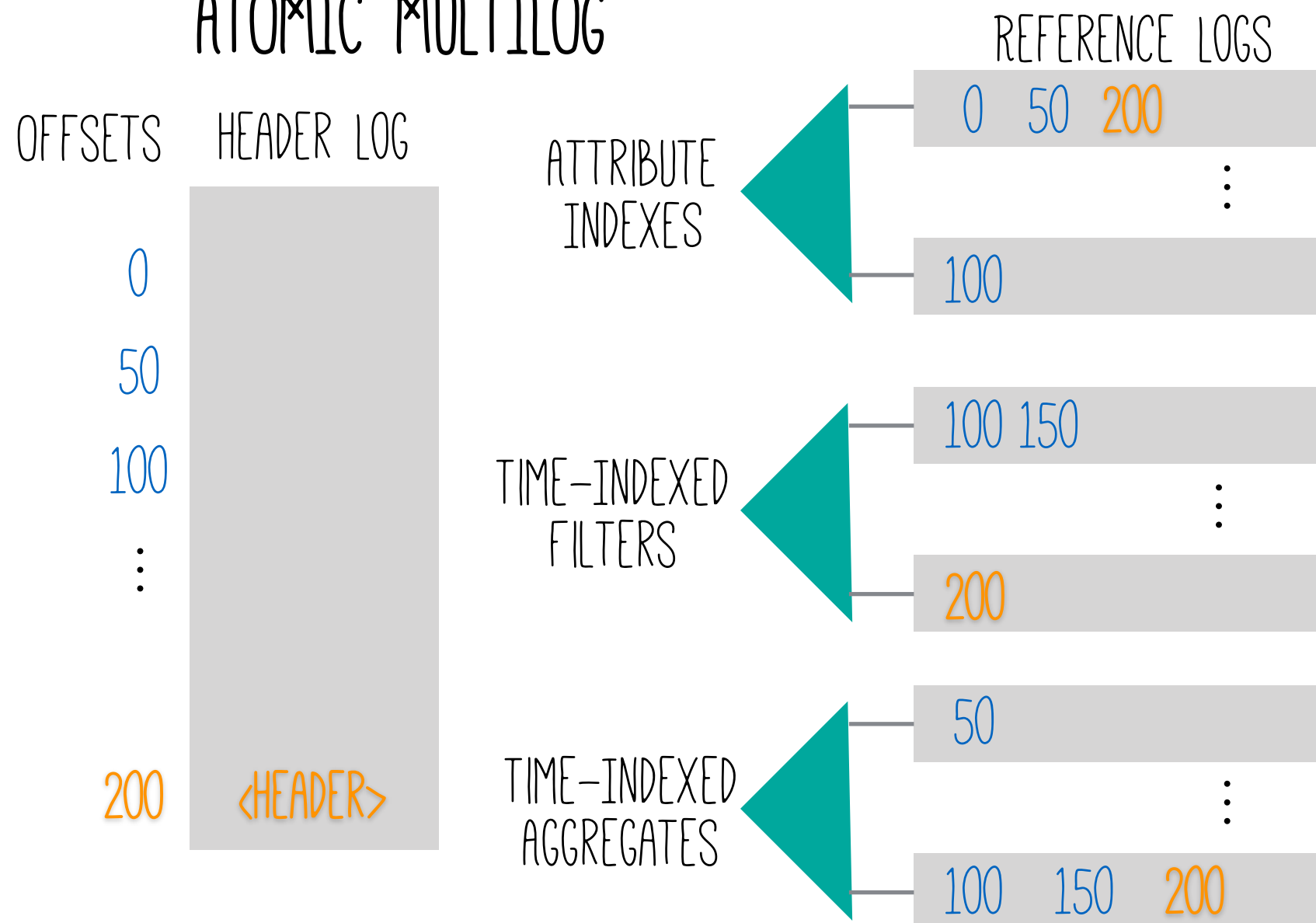


GLOBAL READ TAIL 250
GLOBAL WRITE TAIL 250

} OFFSETS

Atomic MultiLog Linearizability

ATOMIC MULTILOG



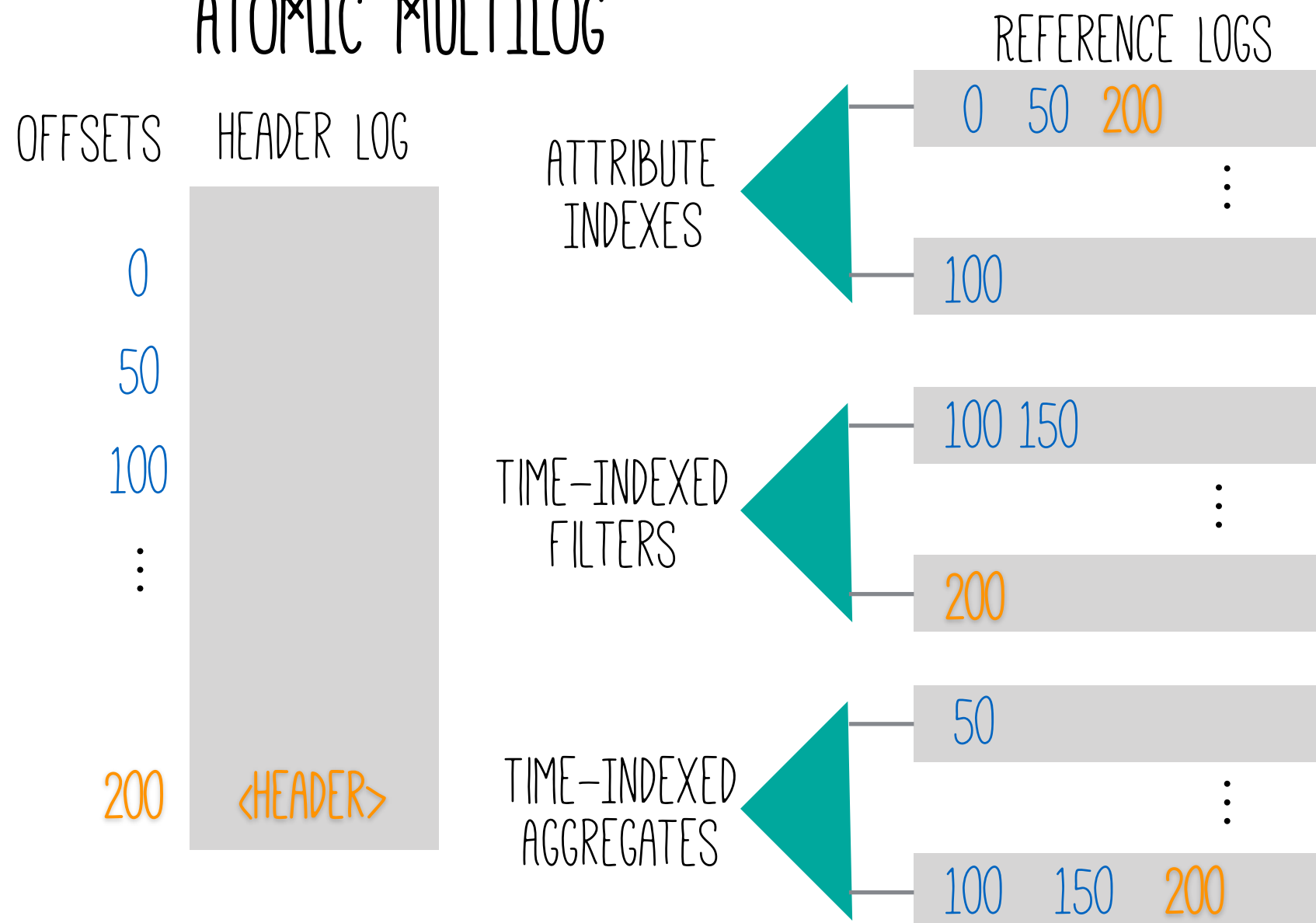
Relax linearizability for individual logs; ensure linearizability only for end-to-end operations

GLOBAL READ TAIL 250
GLOBAL WRITE TAIL 250

} OFFSETS

Atomic MultiLog Linearizability

ATOMIC MULTILOG



Relax linearizability for individual logs; ensure linearizability only for end-to-end operations

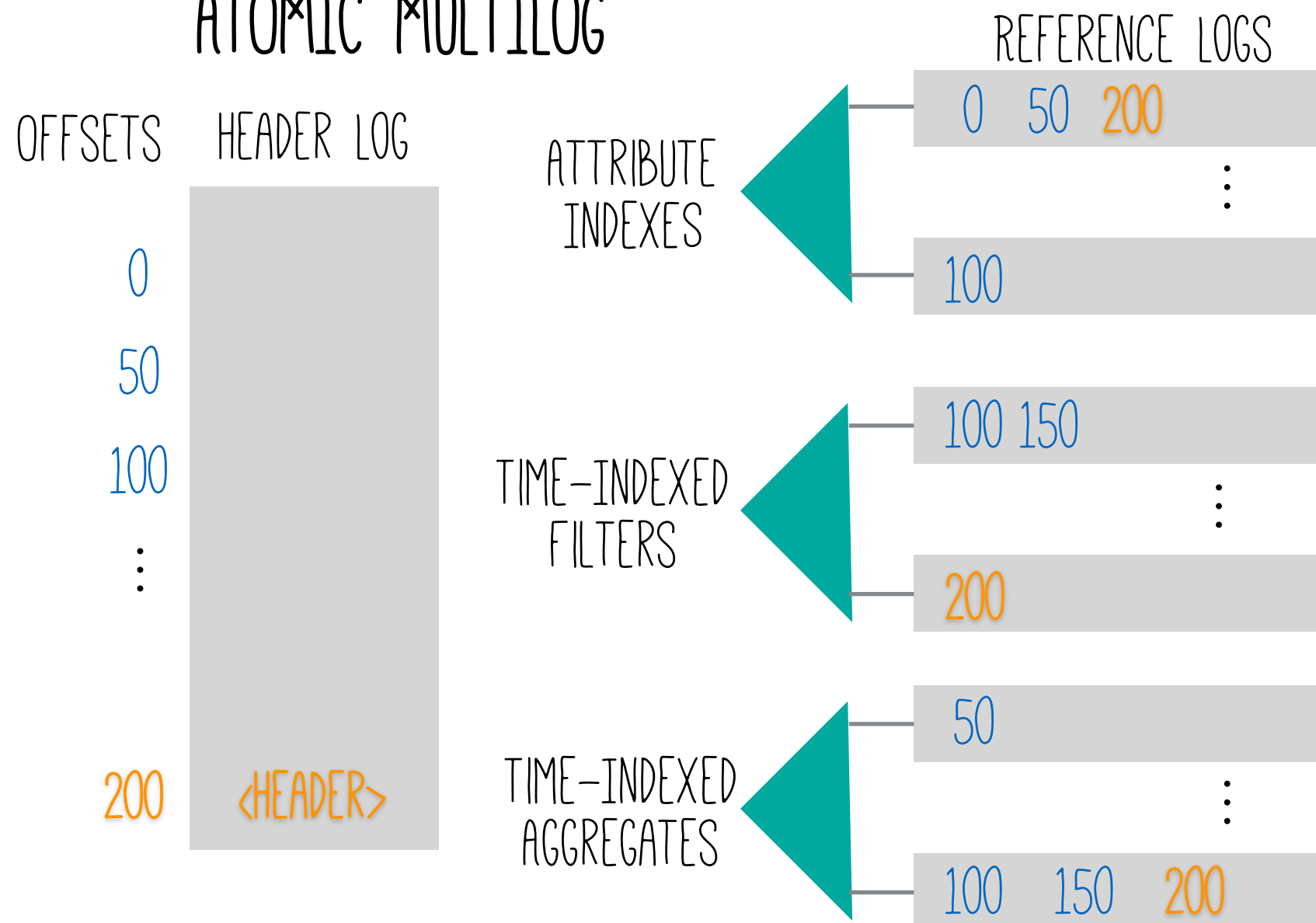
GLOBAL READ TAIL 250
~~GLOBAL READ TAIL 250~~
GLOBAL WRITE TAIL 250
~~GLOBAL WRITE TAIL 250~~

} OFFSETS

Significant performance gains with linearizability at high degrees of concurrency

Atomic MultiLog Linearizability

ATOMIC MULTILOG



Relax linearizability for individual logs; ensure linearizability only for end-to-end operations

GLOBAL READ TAIL 250
~~GLOBAL READ TAIL 250~~
GLOBAL WRITE TAIL 250
~~GLOBAL WRITE TAIL 250~~

} OFFSETS

Significant performance gains with linearizability at high degrees of concurrency

No support for transactions

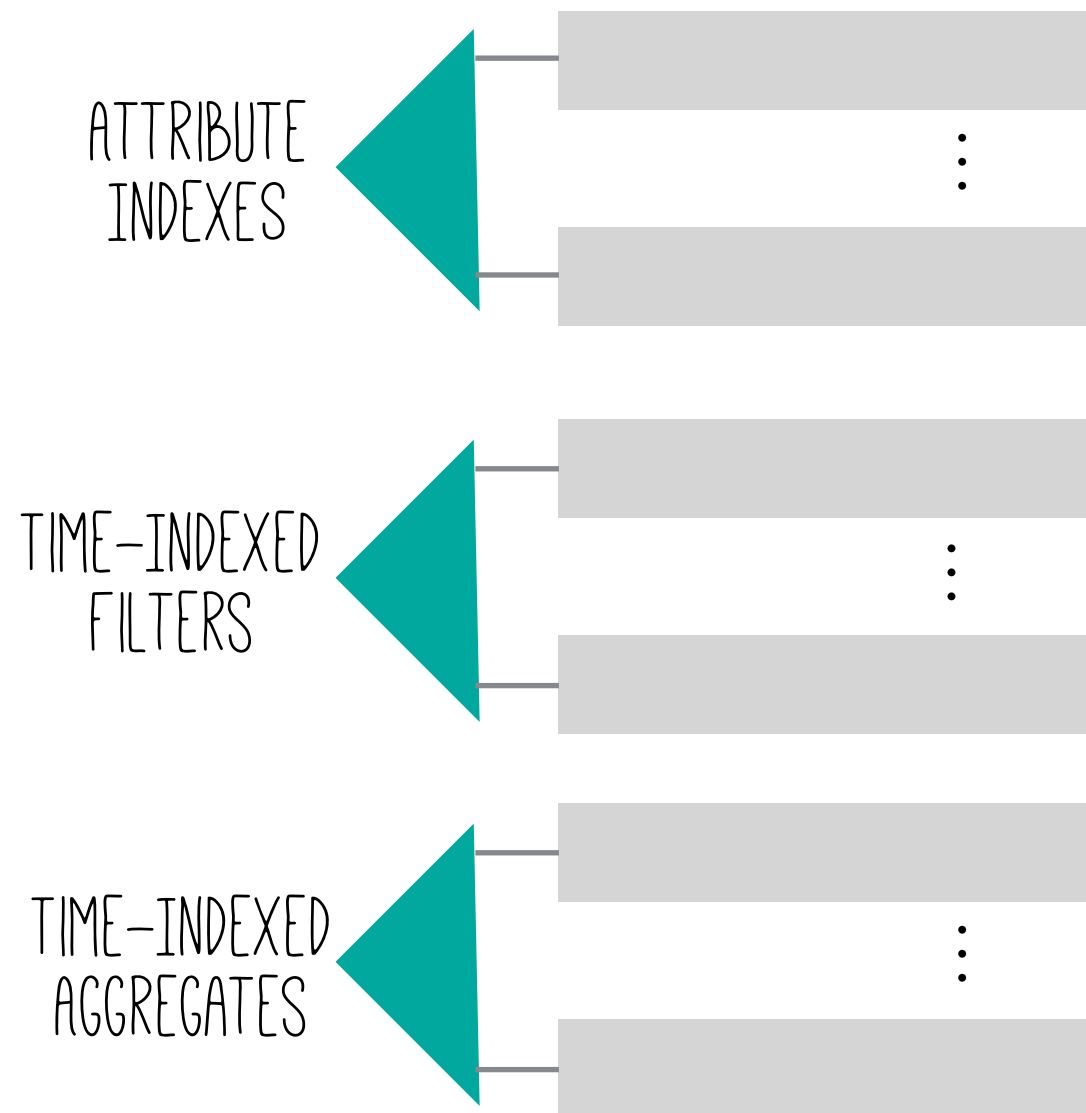
Atomic MultiLog Indexing

Atomic MultiLog Indexing

Traditional Indexes: Expensive to ensure atomicity, high overhead write paths, etc.

Atomic MultiLog Indexing

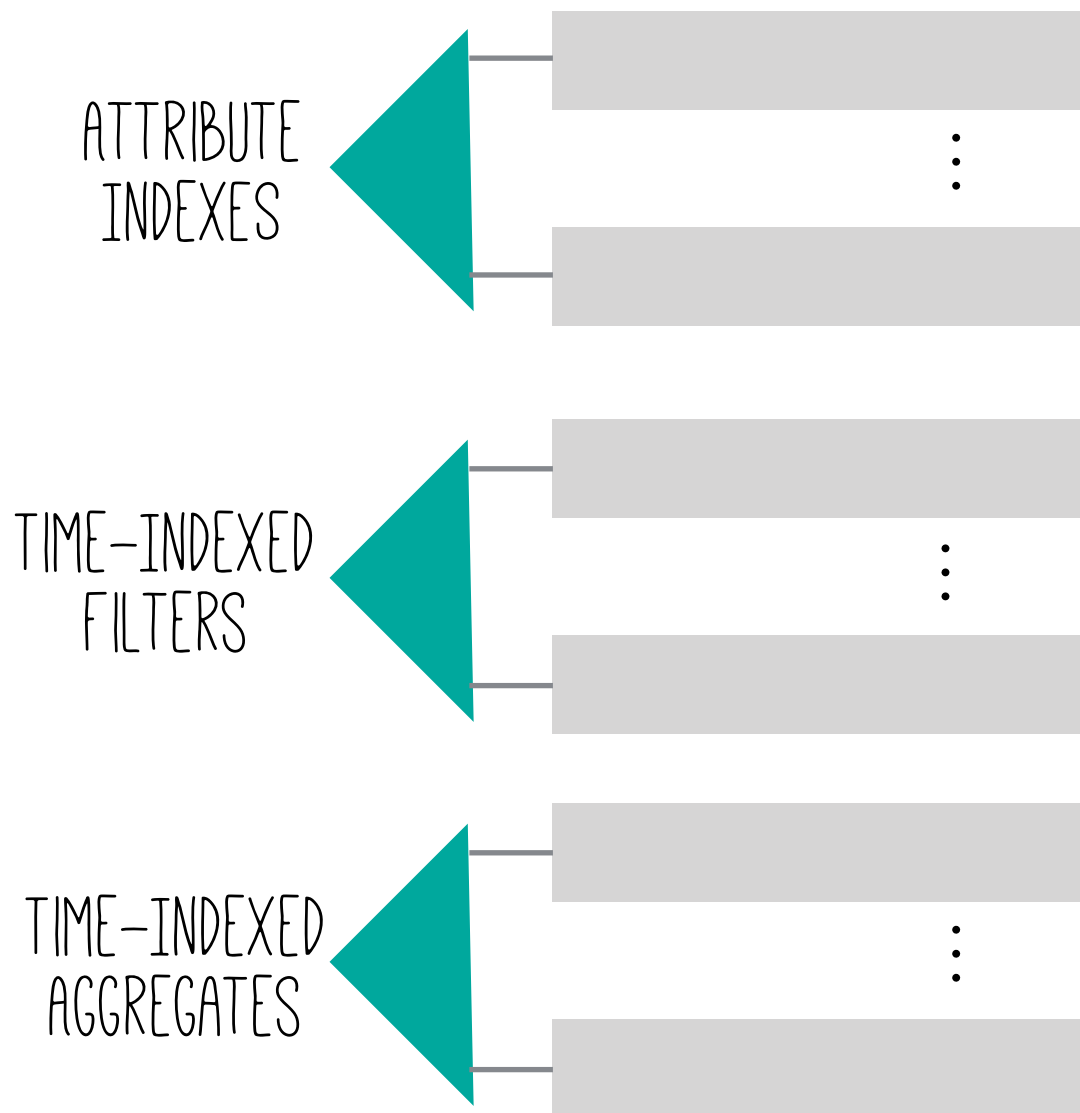
Attributes of interest are *fixed-sized*



Atomic MultiLog Indexing

Attributes of interest are *fixed-sized*

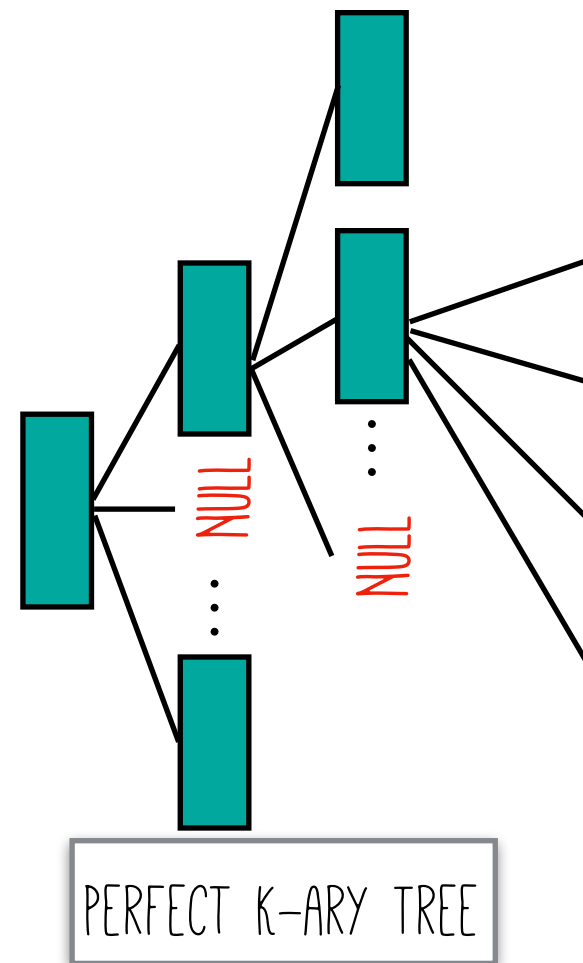
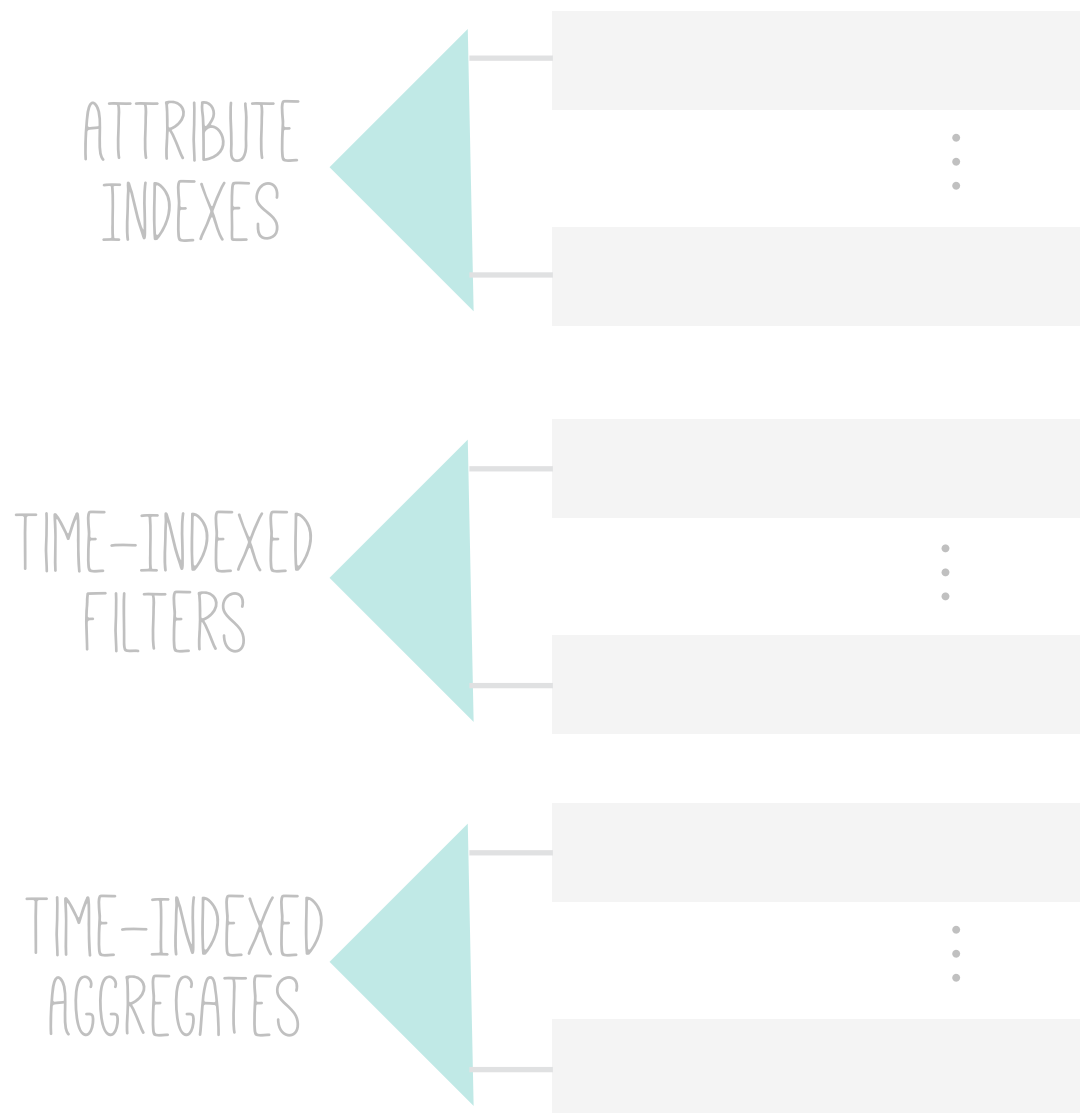
Header fields have fixed domain sizes,
e.g., 16-bit port in $[0, 2^{16}]$



Atomic MultiLog Indexing

Attributes of interest are *fixed-sized*

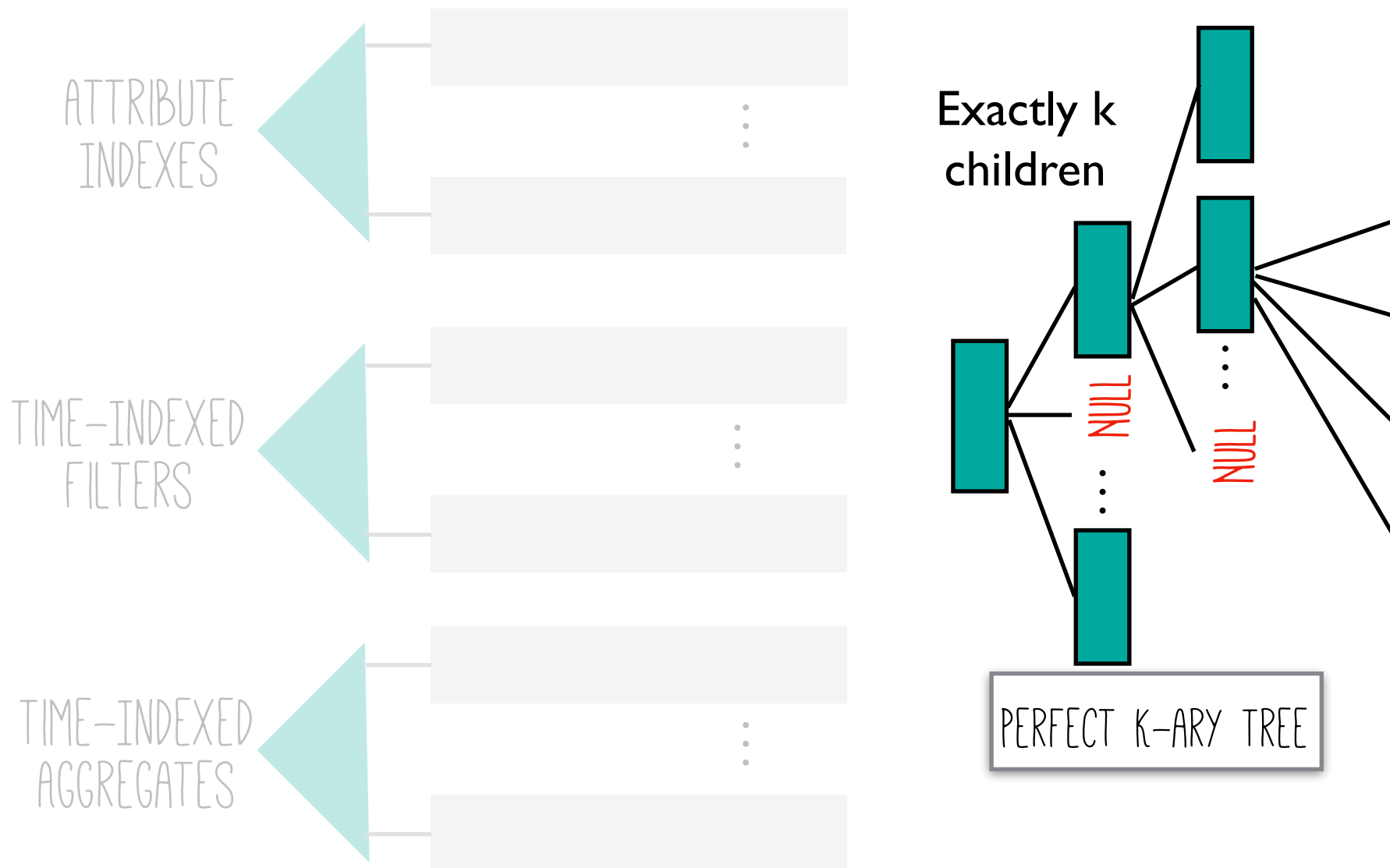
Header fields have fixed domain sizes,
e.g., 16-bit port in $[0, 2^{16}]$



Atomic MultiLog Indexing

Attributes of interest are *fixed-sized*

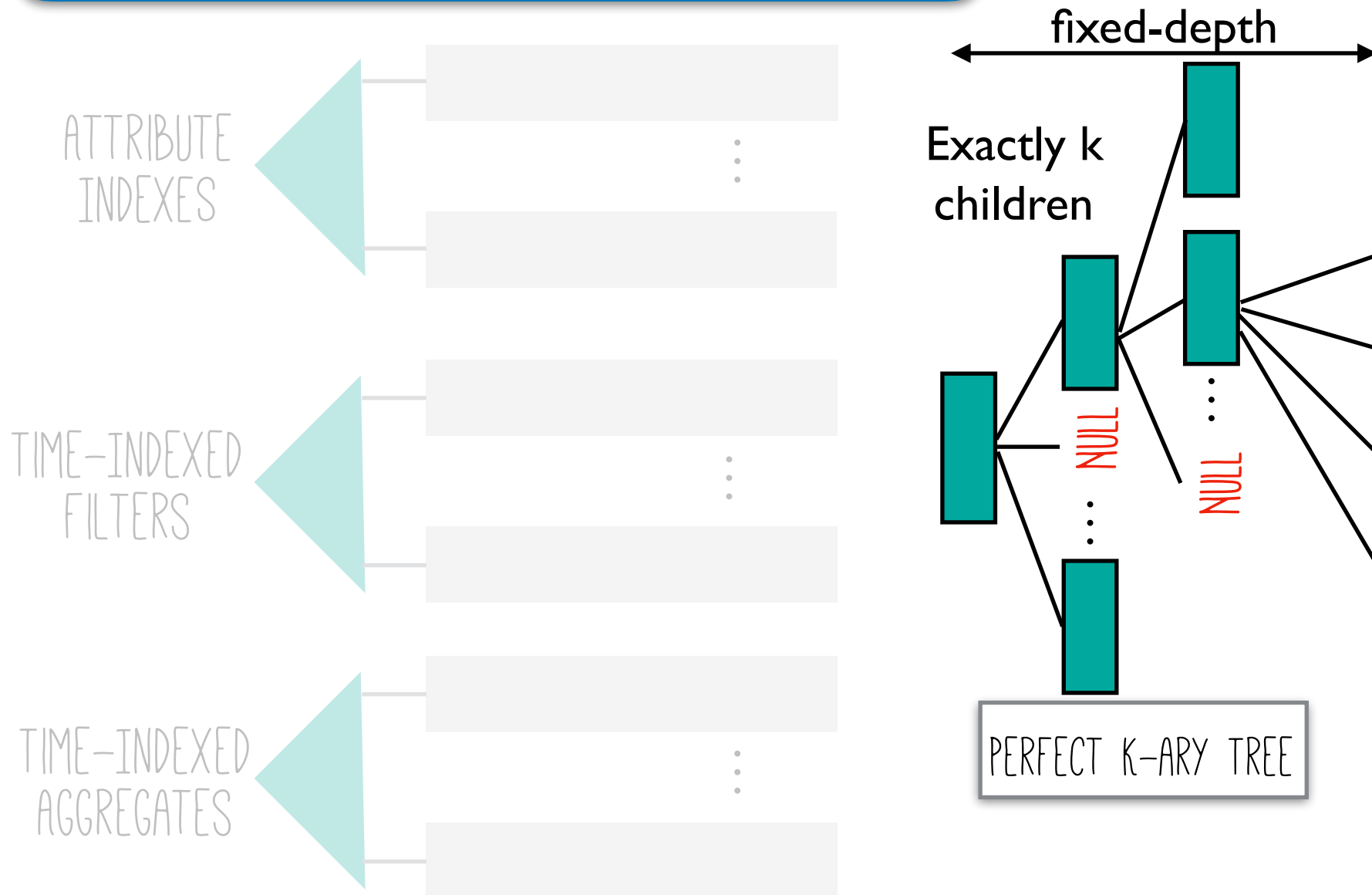
Header fields have fixed domain sizes,
e.g., 16-bit port in $[0, 2^{16}]$



Atomic MultiLog Indexing

Attributes of interest are *fixed-sized*

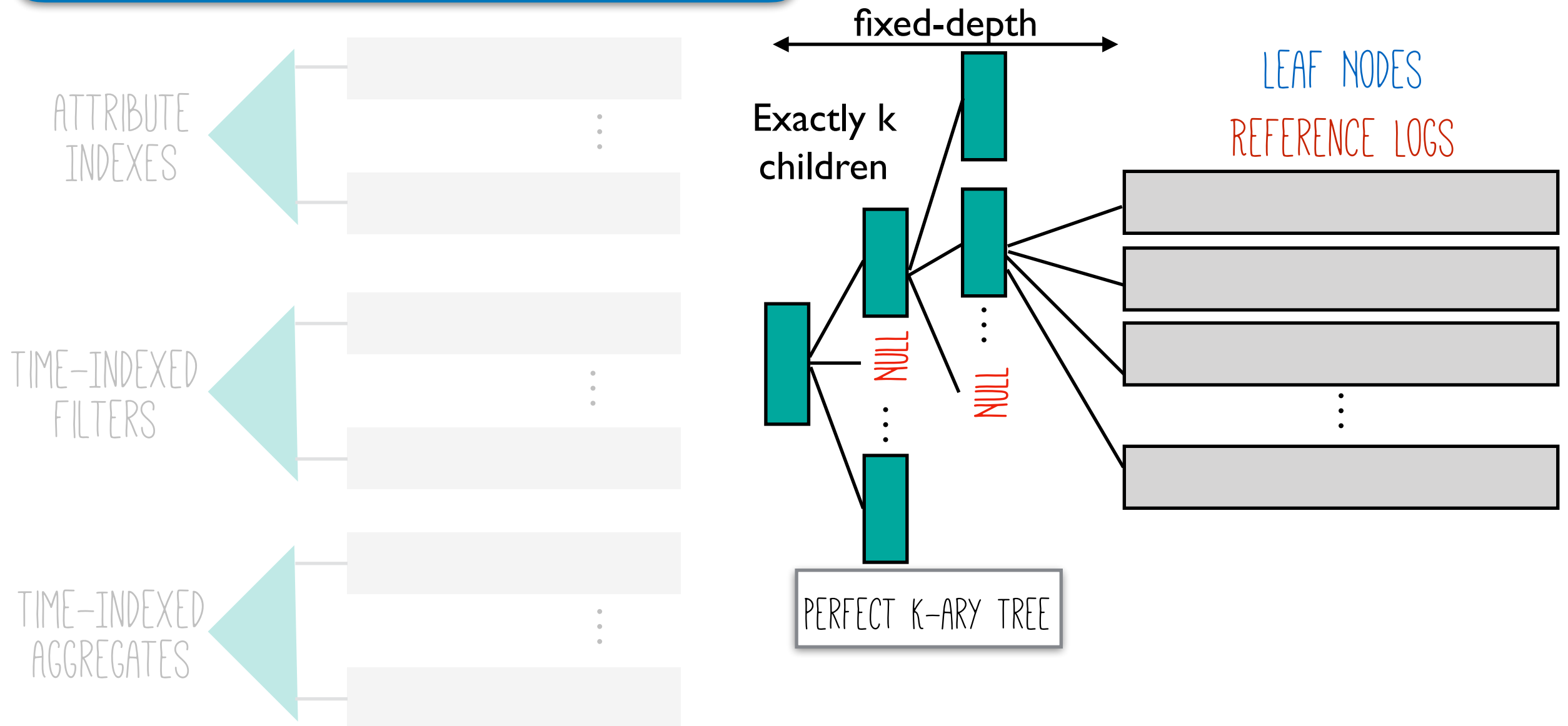
Header fields have fixed domain sizes,
e.g., 16-bit port in $[0, 2^{16}]$



Atomic MultiLog Indexing

Attributes of interest are *fixed-sized*

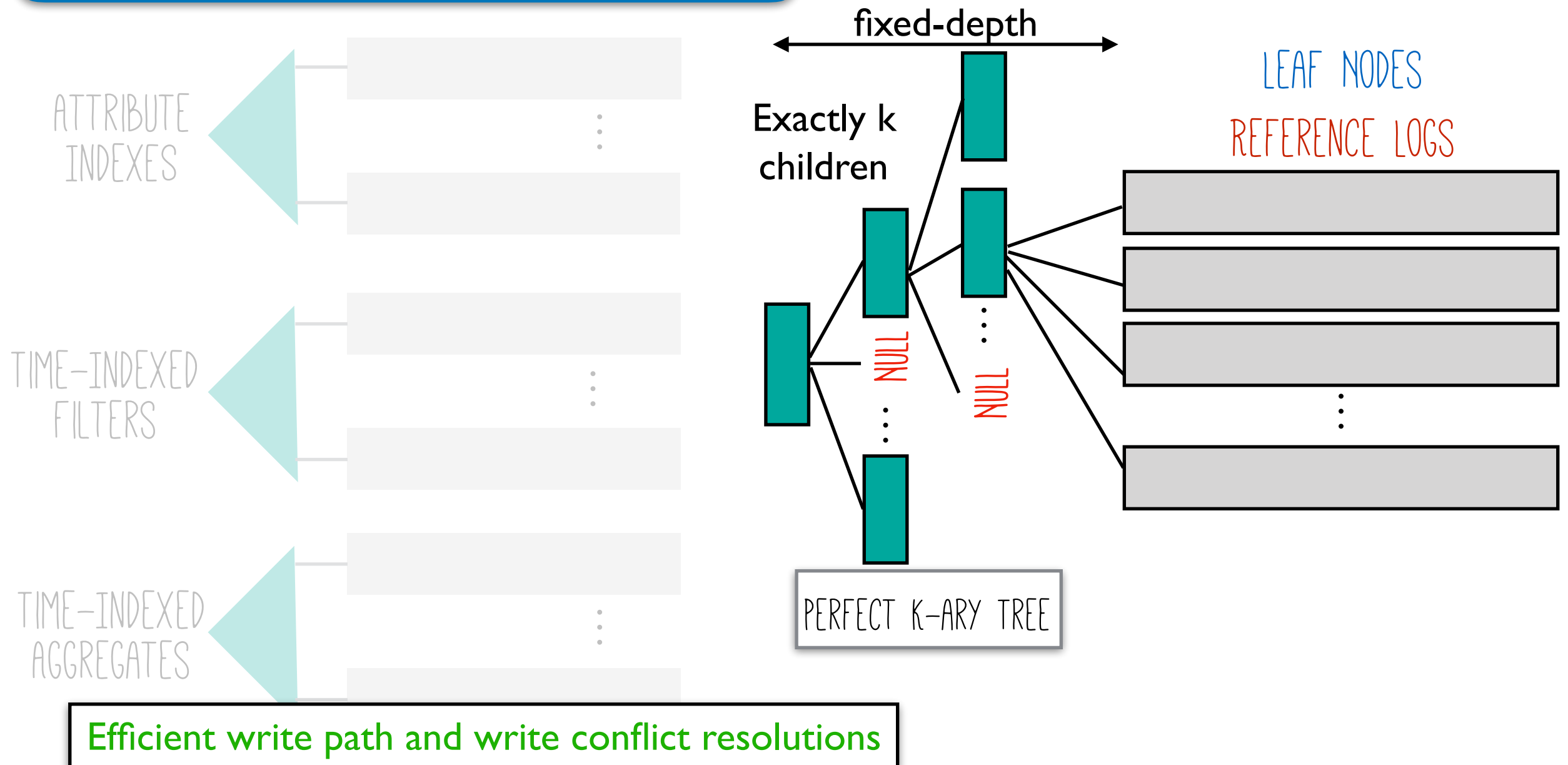
Header fields have fixed domain sizes,
e.g., 16-bit port in $[0, 2^{16}]$



Atomic MultiLog Indexing

Attributes of interest are *fixed-sized*

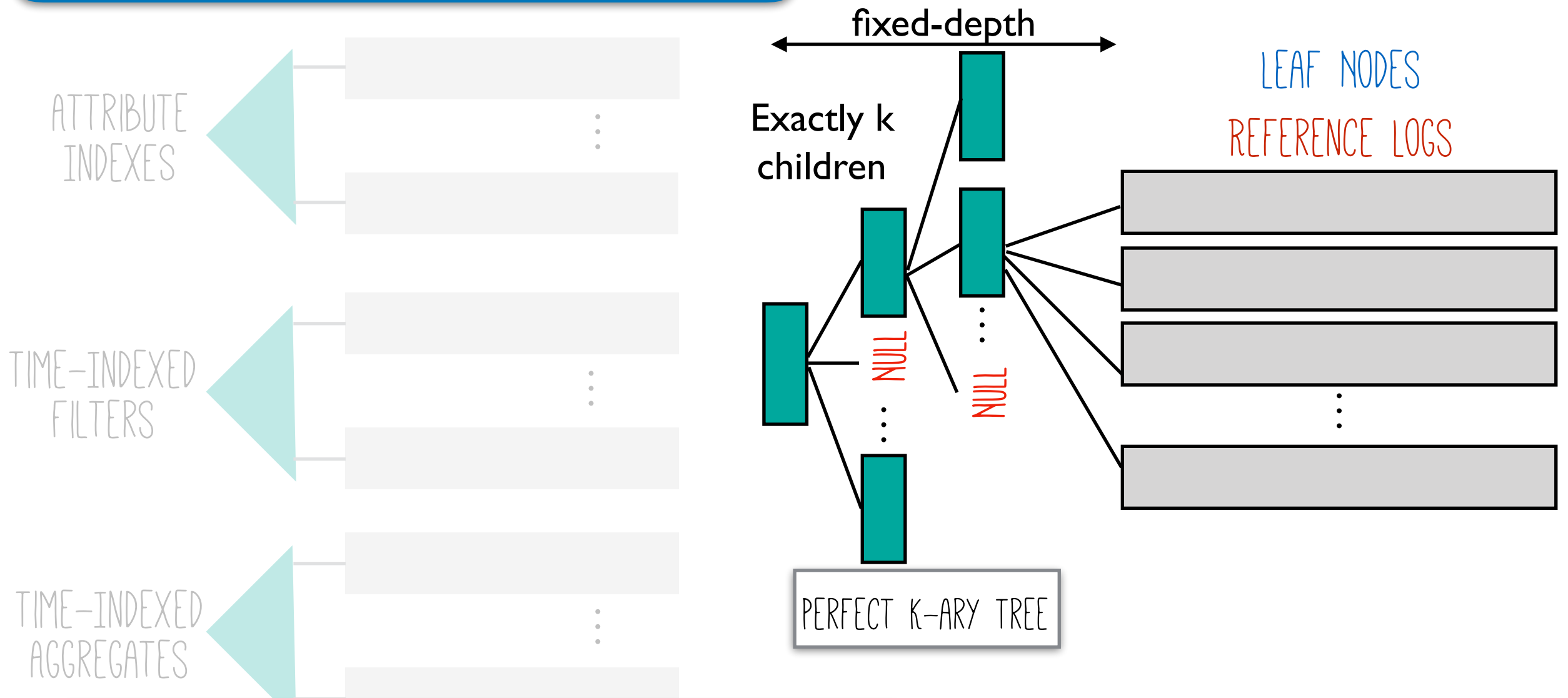
Header fields have fixed domain sizes,
e.g., 16-bit port in $[0, 2^{16}]$



Atomic MultiLog Indexing

Attributes of interest are *fixed-sized*

Header fields have fixed domain sizes,
e.g., 16-bit port in $[0, 2^{16}]$



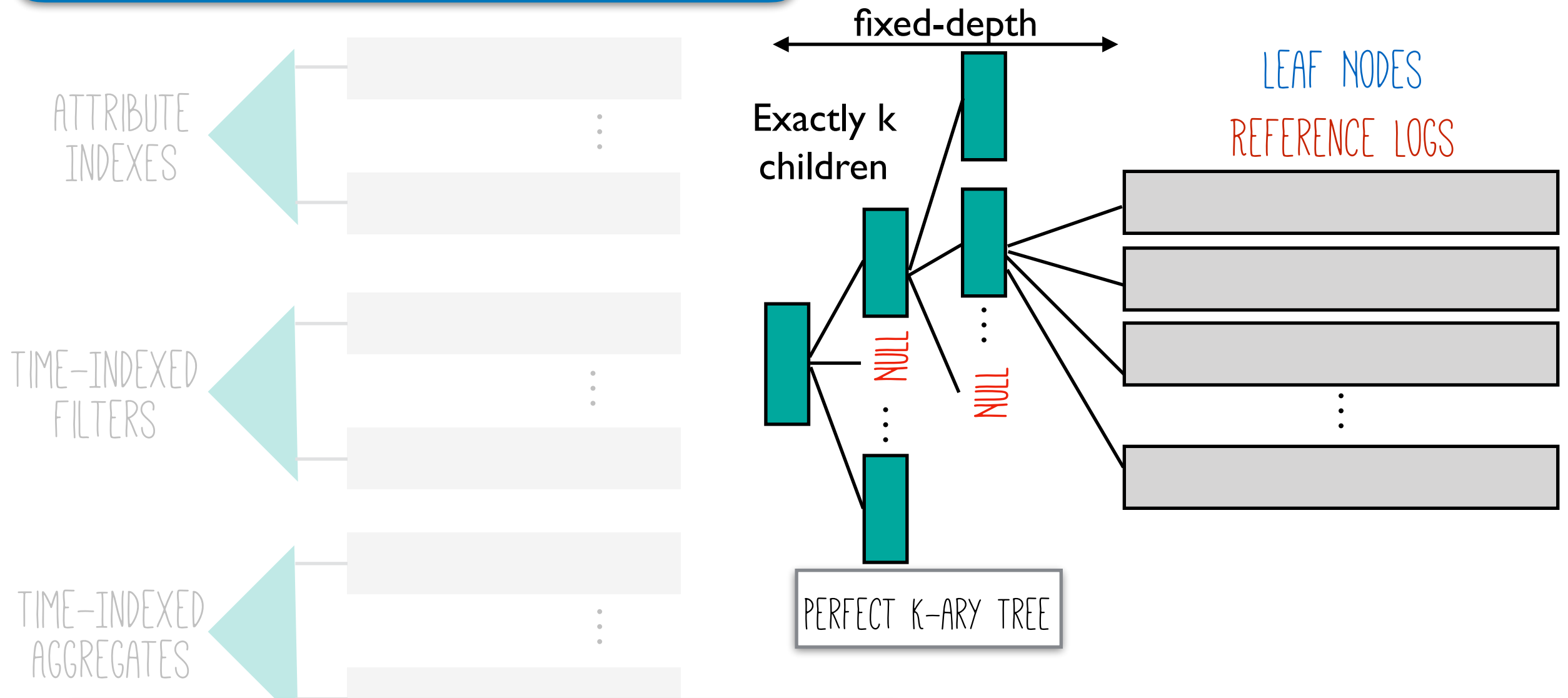
Efficient write path and write conflict resolutions

2.2x faster (1 core), 7.8x faster (48 cores)

Atomic MultiLog Indexing

Attributes of interest are *fixed-sized*

Header fields have fixed domain sizes,
e.g., 16-bit port in $[0, 2^{16}]$



Efficient write path and write conflict resolutions

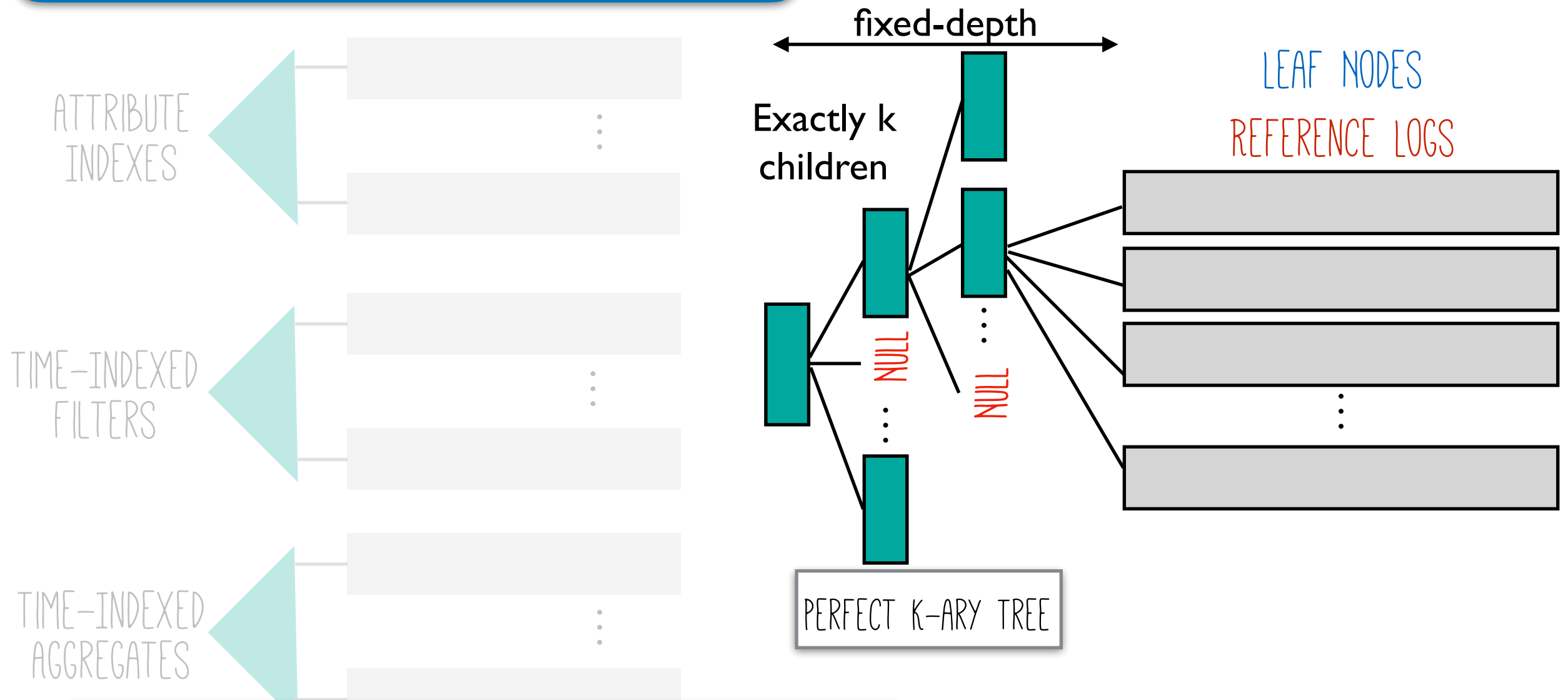
2.2x faster (1 core), 7.8x faster (48 cores)

Ordered access via inexpensive range queries

Atomic MultiLog Indexing

Attributes of interest are *fixed-sized*

Header fields have fixed domain sizes,
e.g., 16-bit port in $[0, 2^{16}]$



Efficient write path and write conflict resolutions

Ordered access via inexpensive range queries

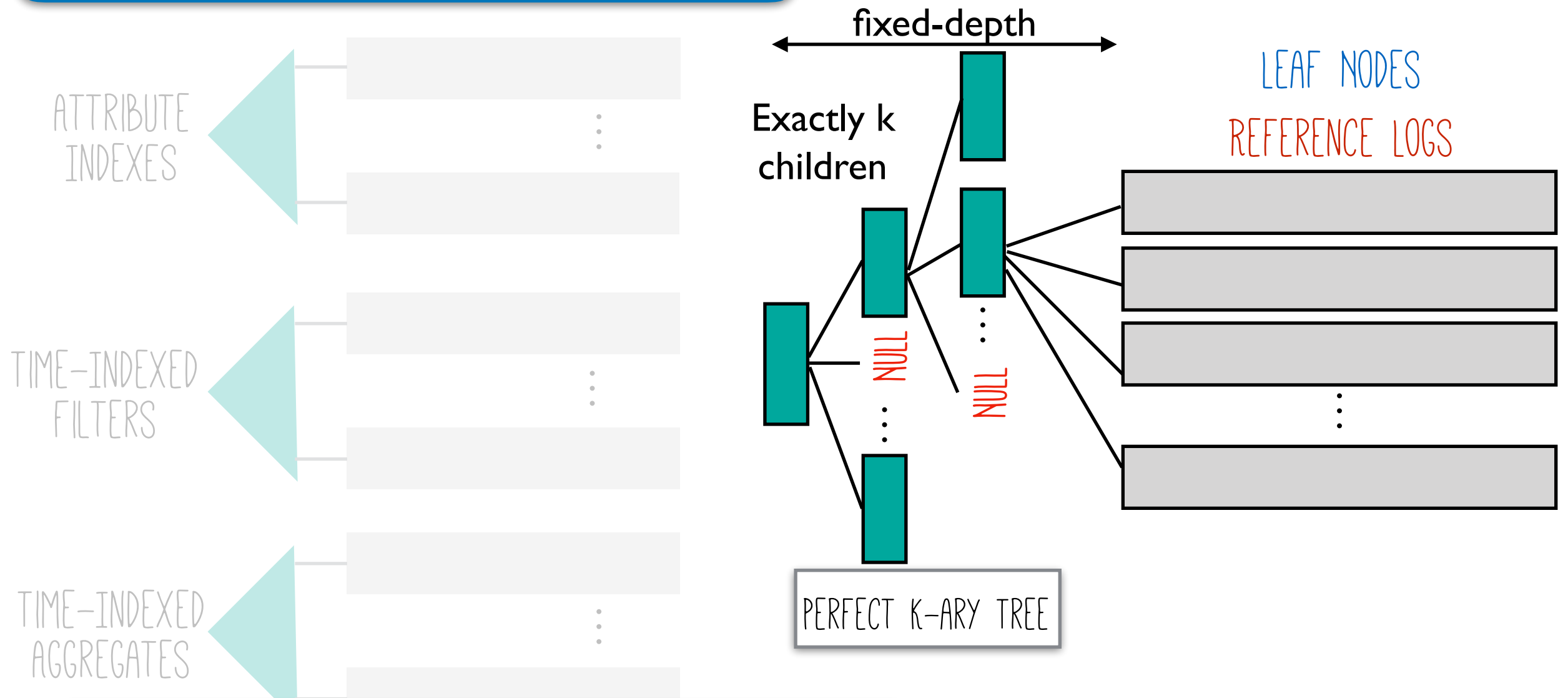
2.2x faster (1 core), 7.8x faster (48 cores)

~5.1x faster

Atomic MultiLog Indexing

Attributes of interest are *fixed-sized*

Header fields have fixed domain sizes,
e.g., 16-bit port in $[0, 2^{16}]$



Efficient write path and write conflict resolutions

2.2x faster (1 core), 7.8x faster (48 cores)

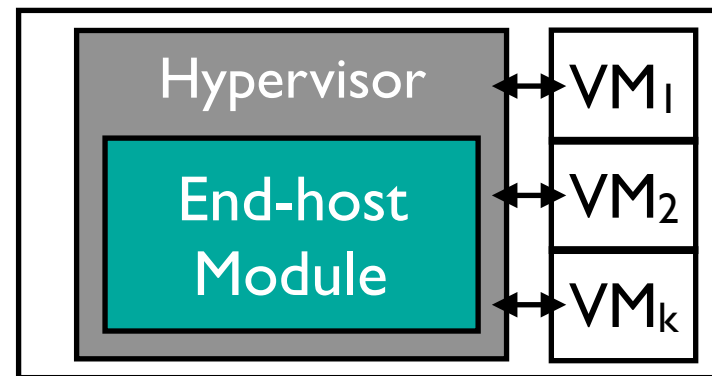
Ordered access via inexpensive range queries

~5.1x faster

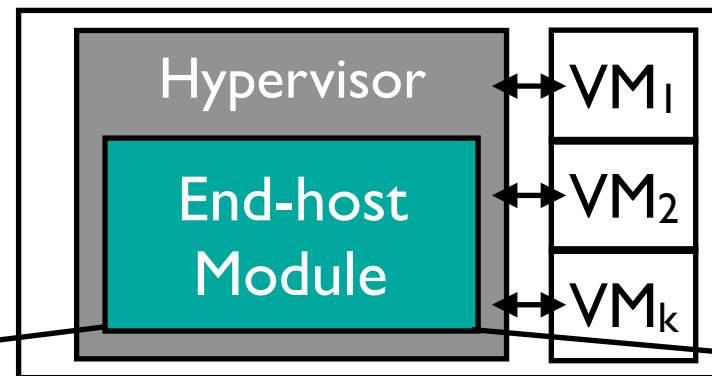
Only supports fixed-sized attributes

Confluo End-host Architecture

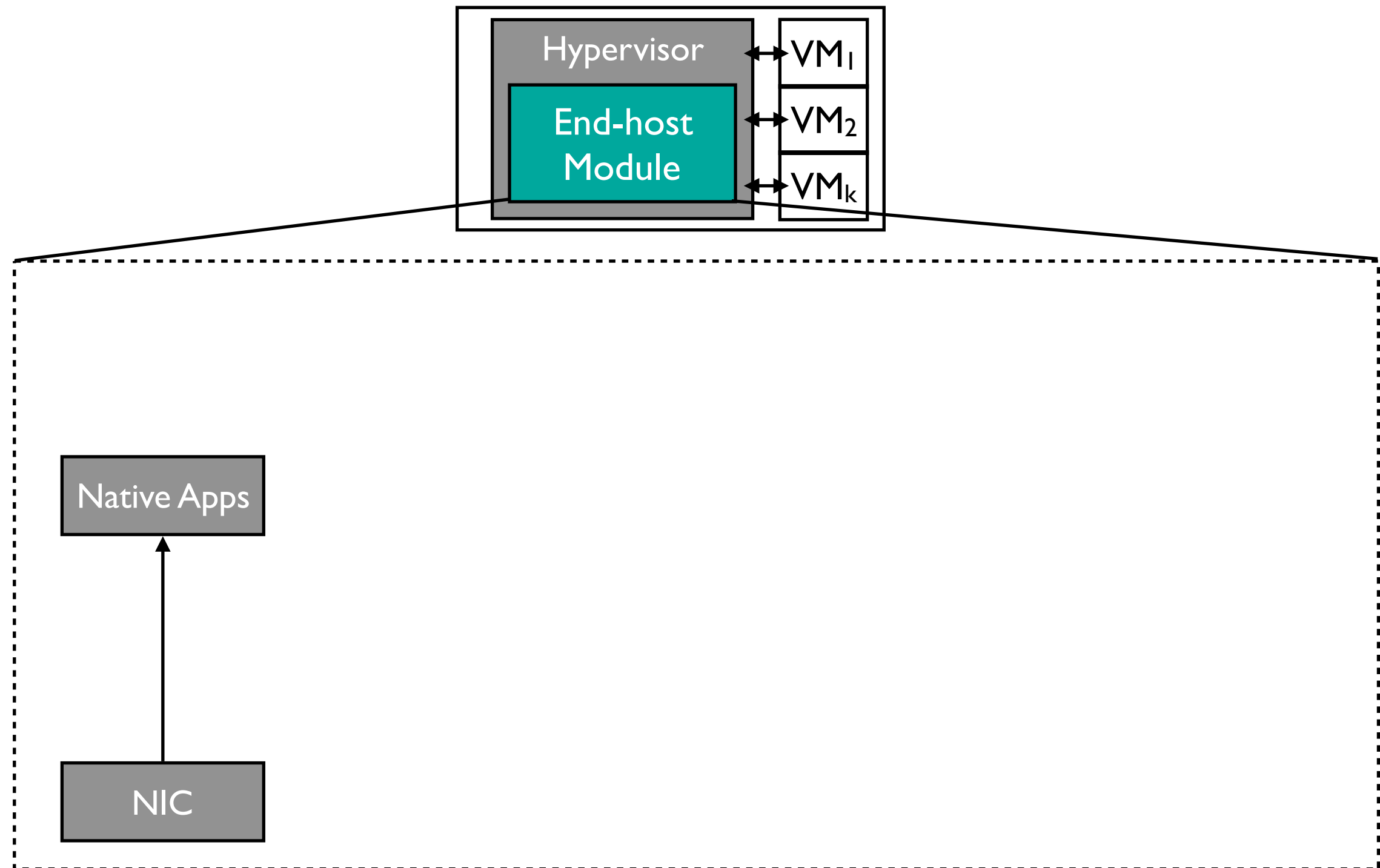
Confluo End-host Architecture



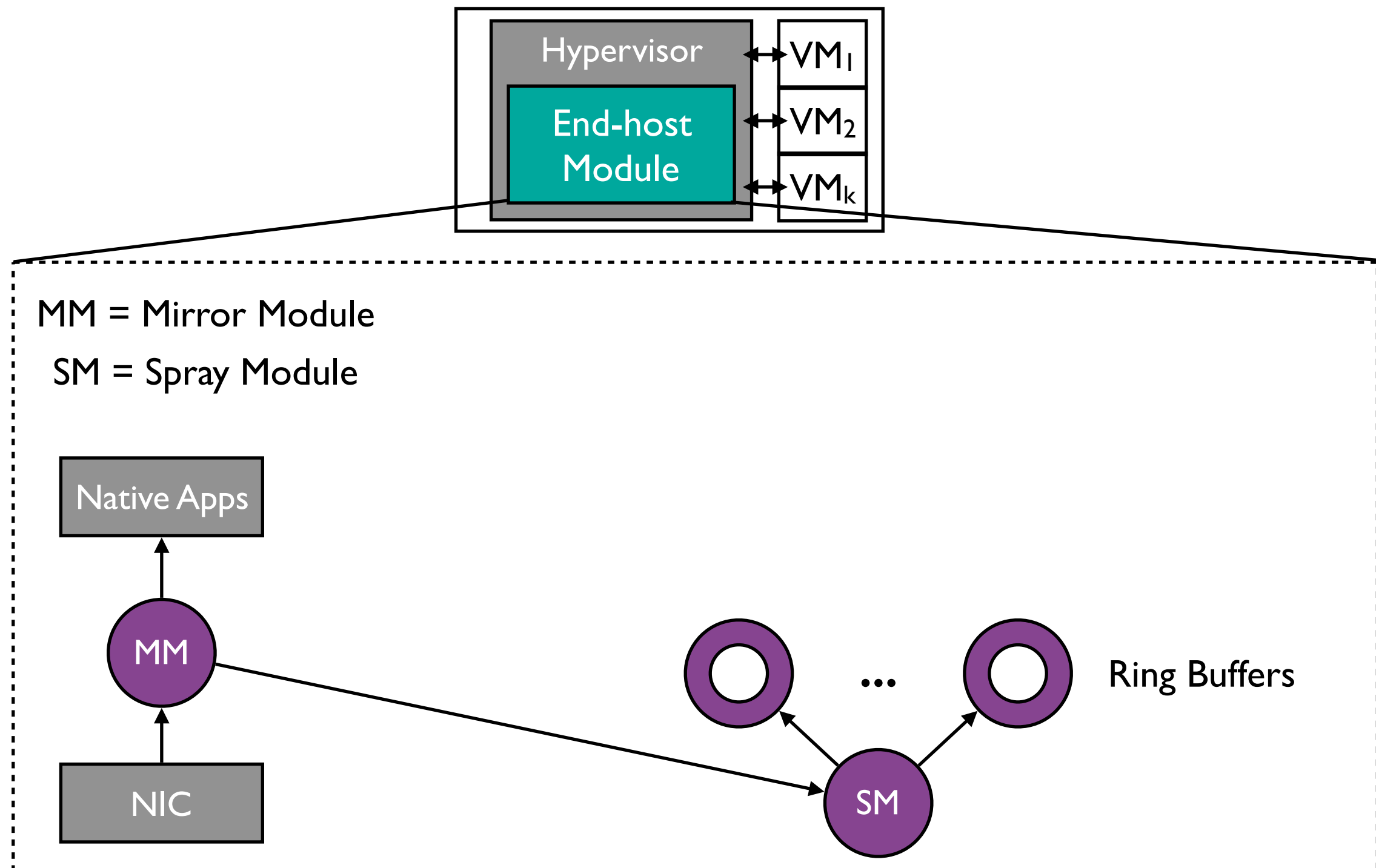
Confluo End-host Architecture



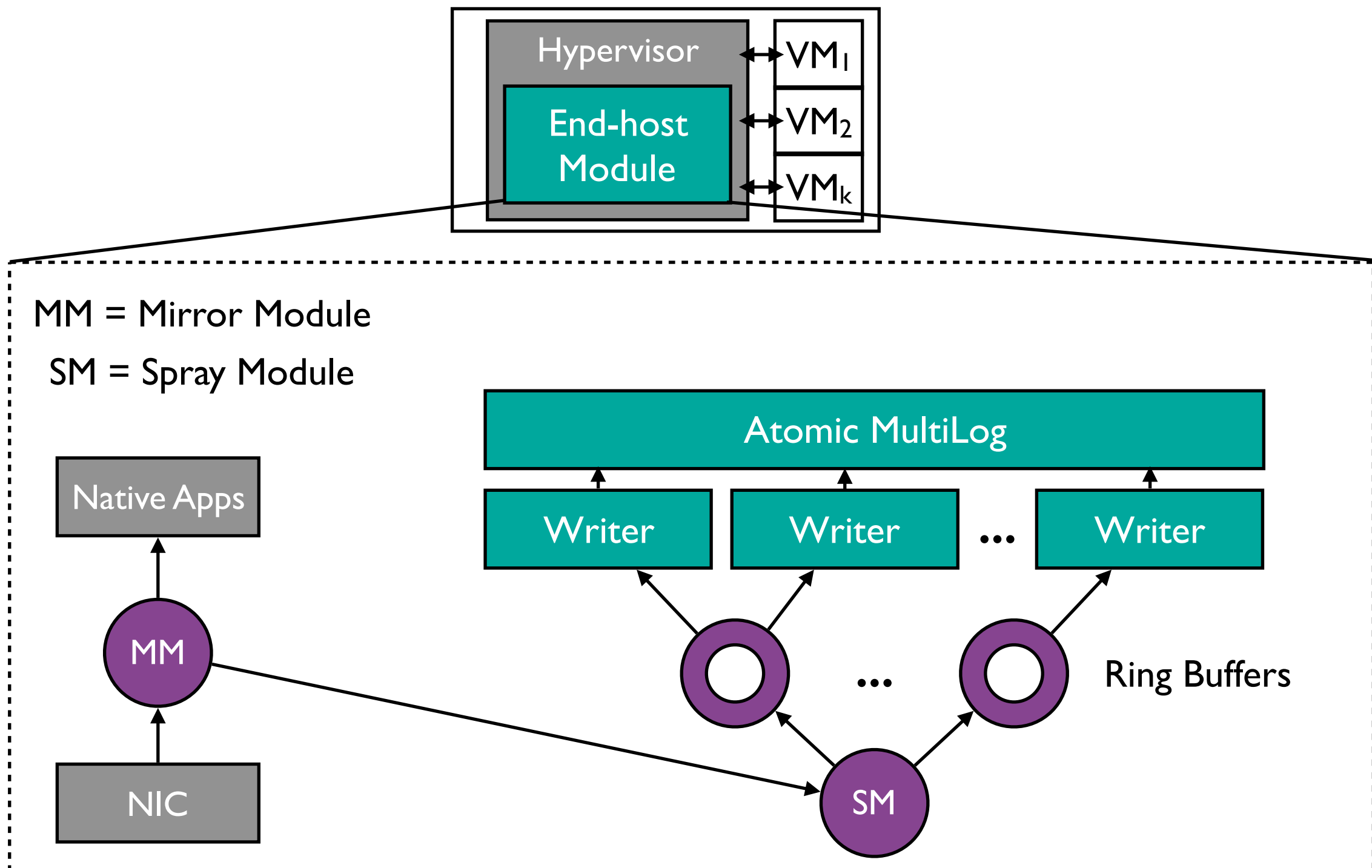
Confluo End-host Architecture



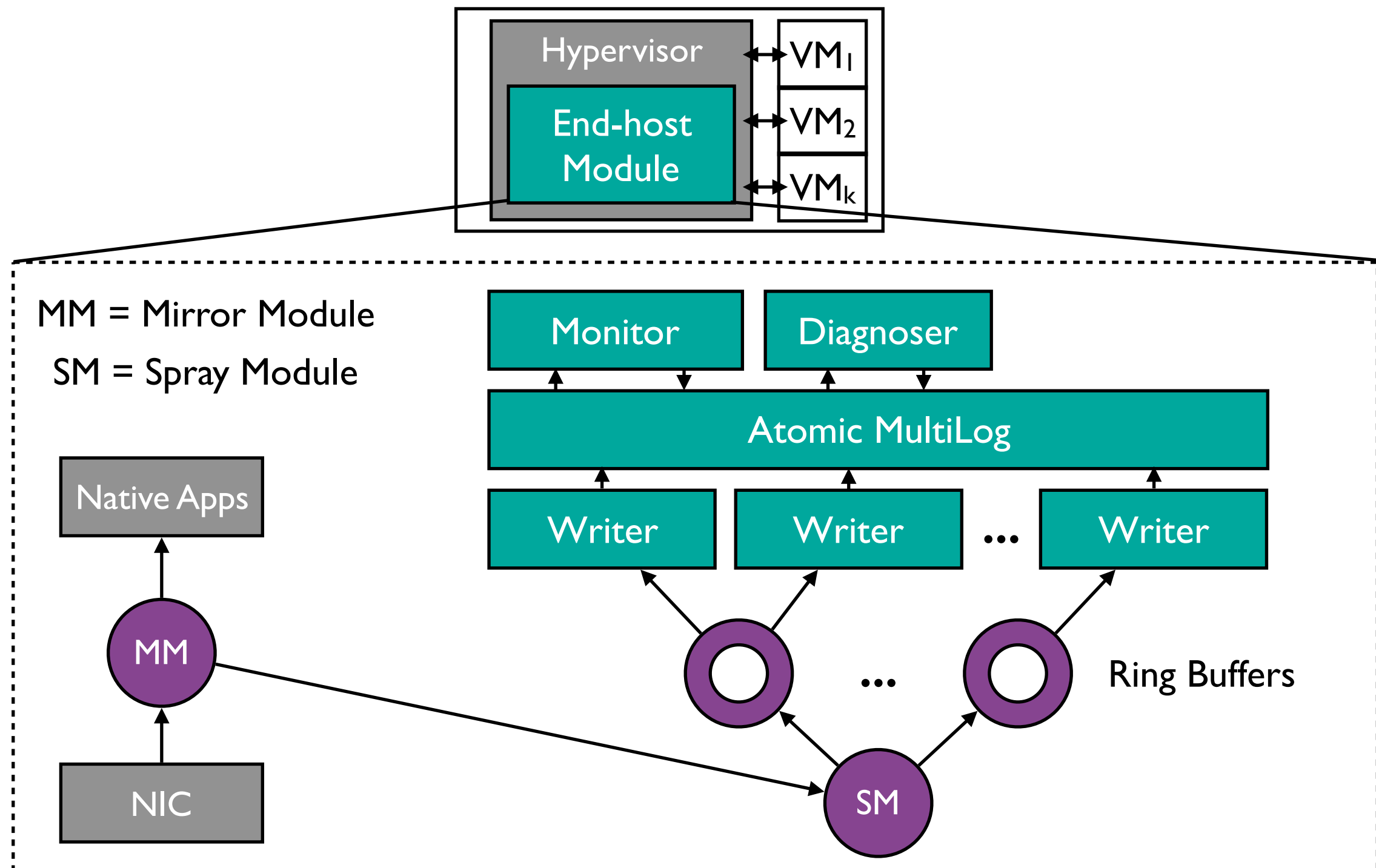
Confluo End-host Architecture



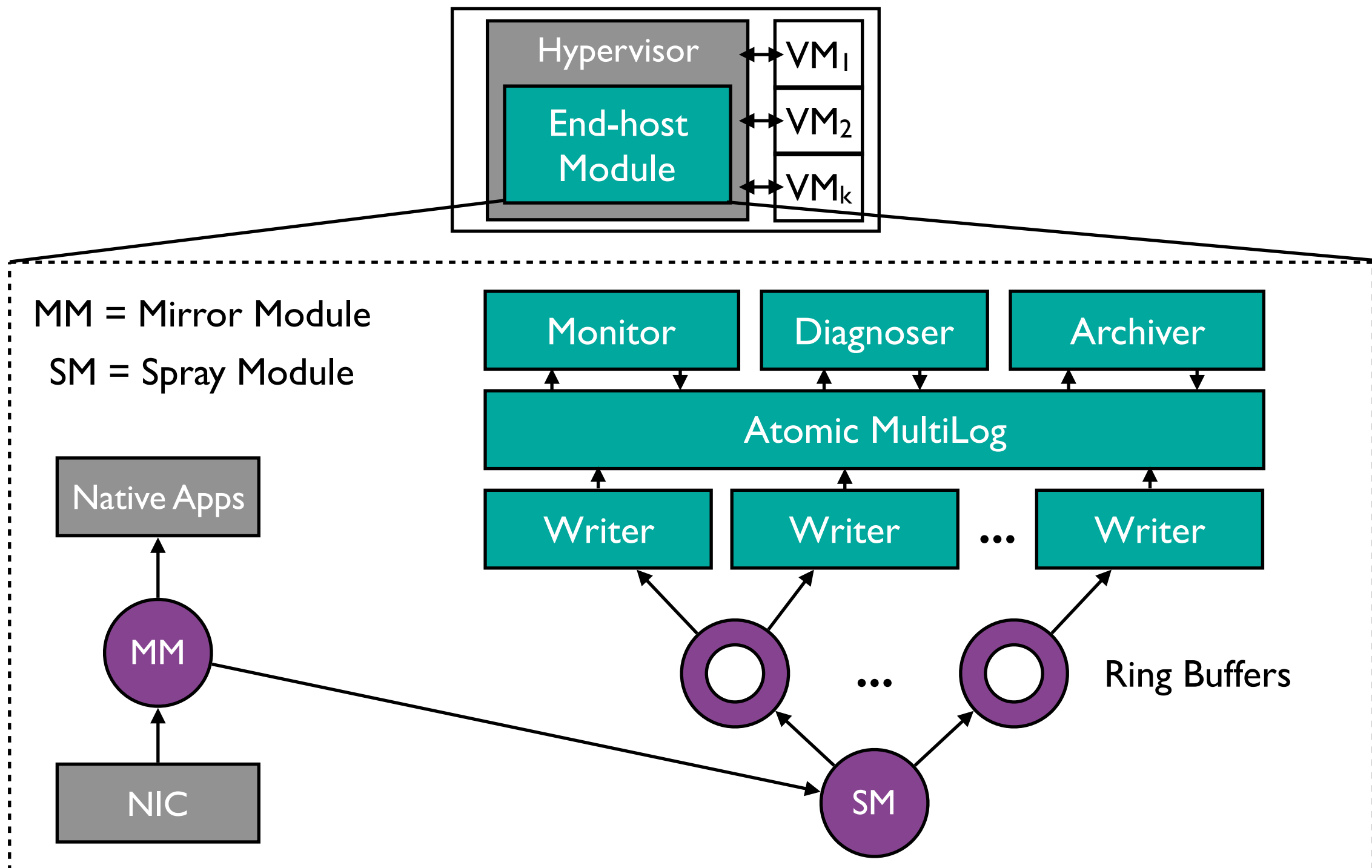
Confluo End-host Architecture



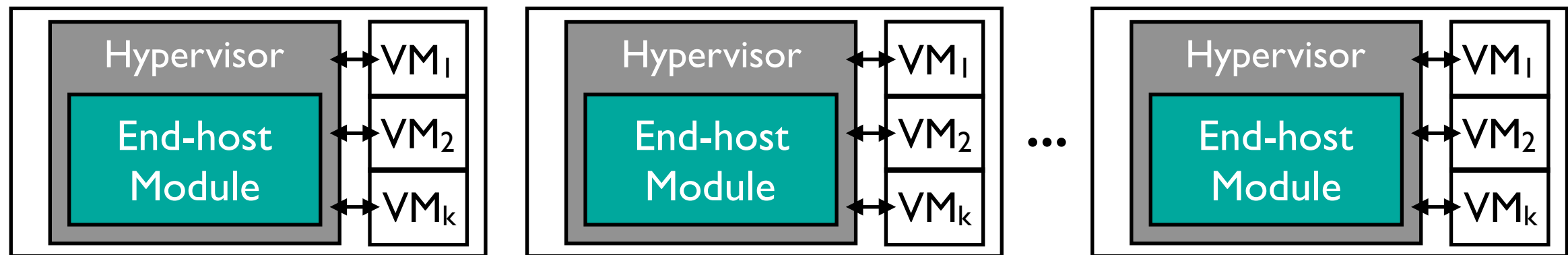
Confluo End-host Architecture



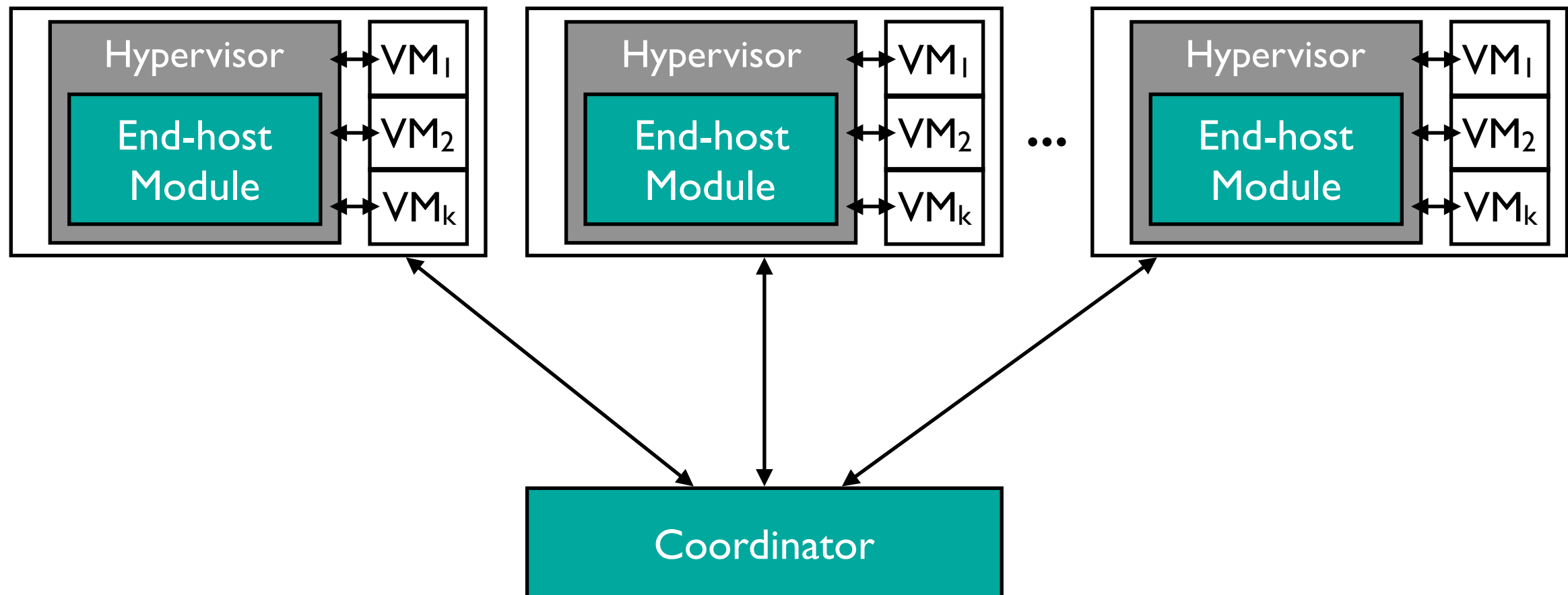
Confluo End-host Architecture



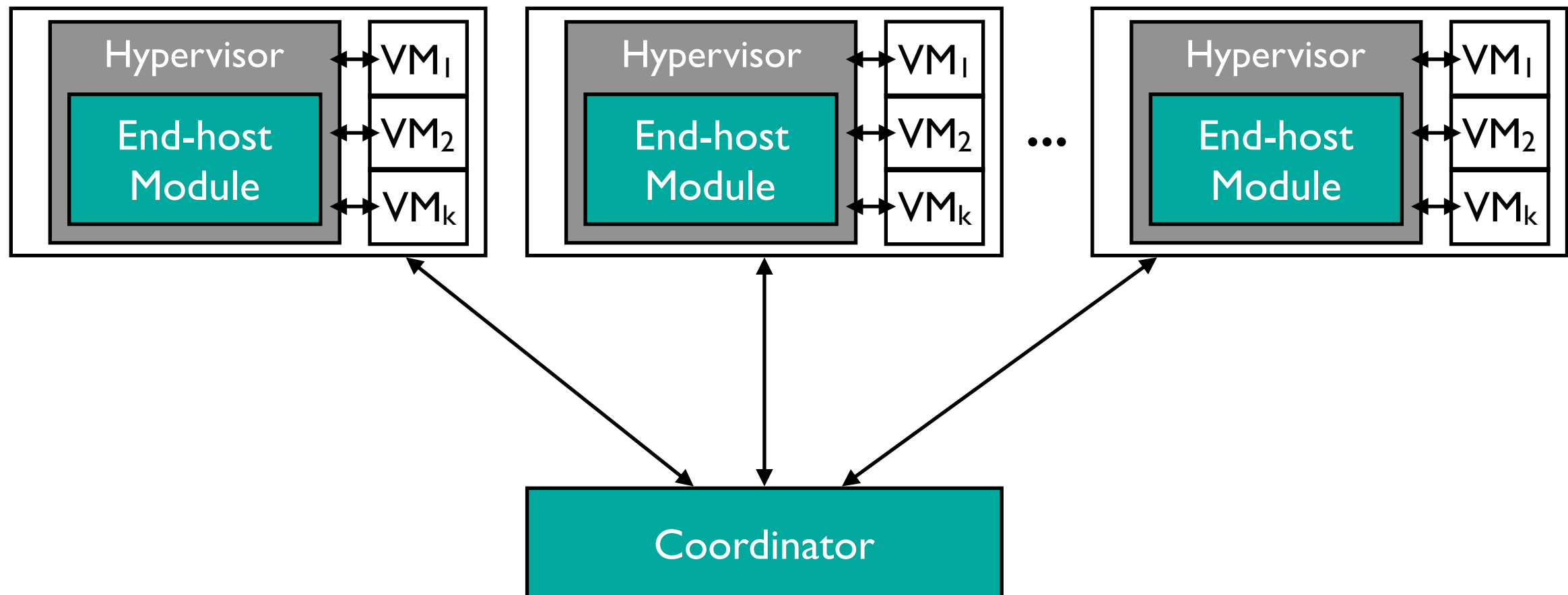
Confluo End-host Architecture



Confluo End-host Architecture



Confluo End-host Architecture



Consistent analysis of network-wide events using Linearizable Snapshots

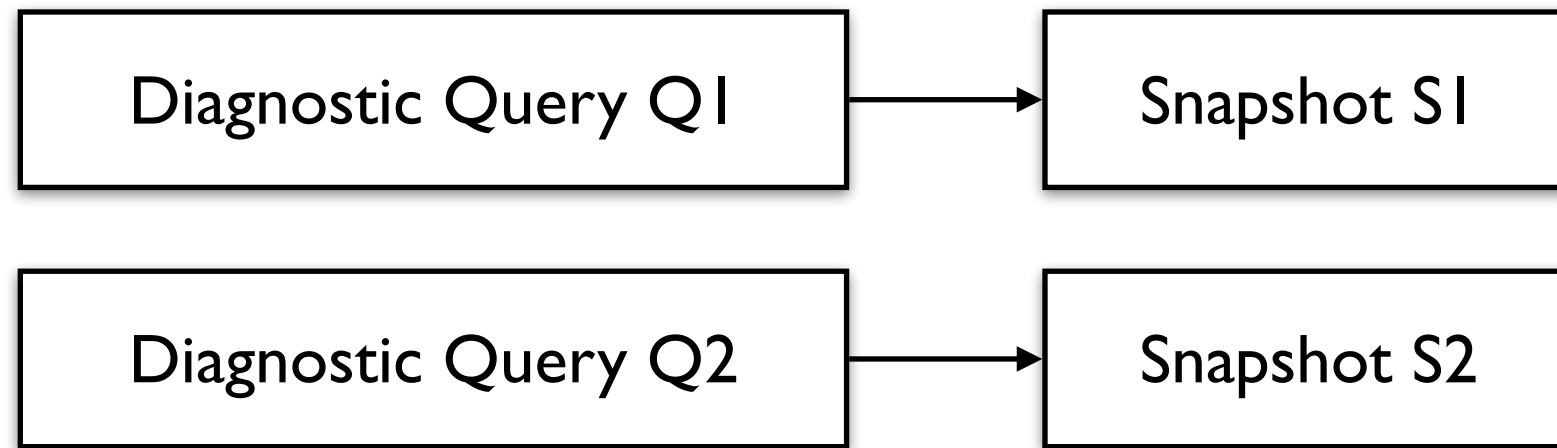
Consistency in Distributed Diagnosis

Consistency in Distributed Diagnosis

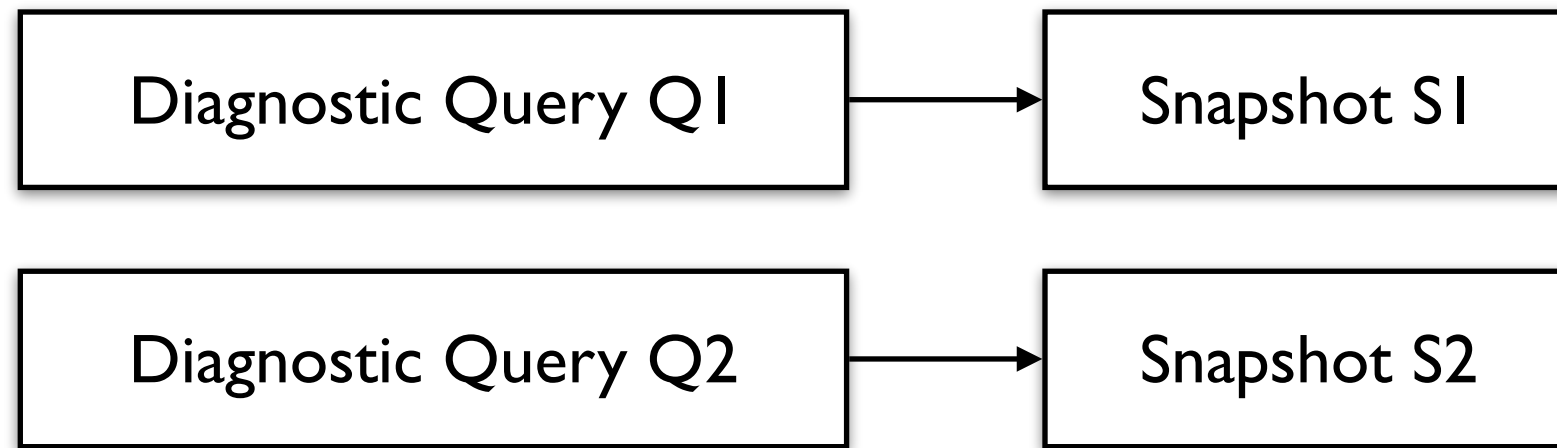
Diagnostic Query Q1

Diagnostic Query Q2

Consistency in Distributed Diagnosis

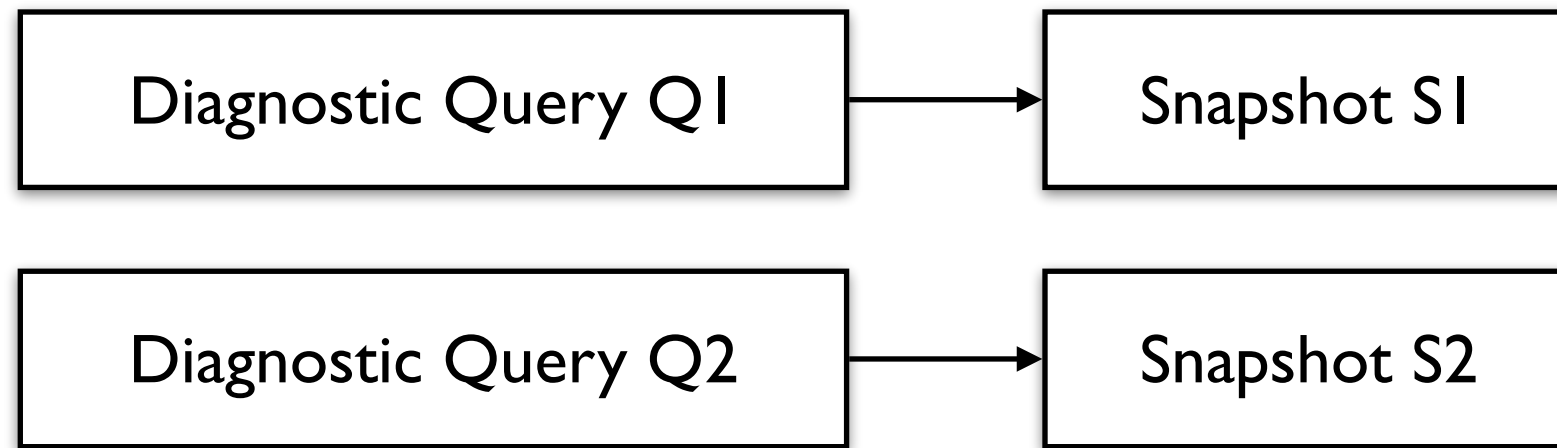


Consistency in Distributed Diagnosis



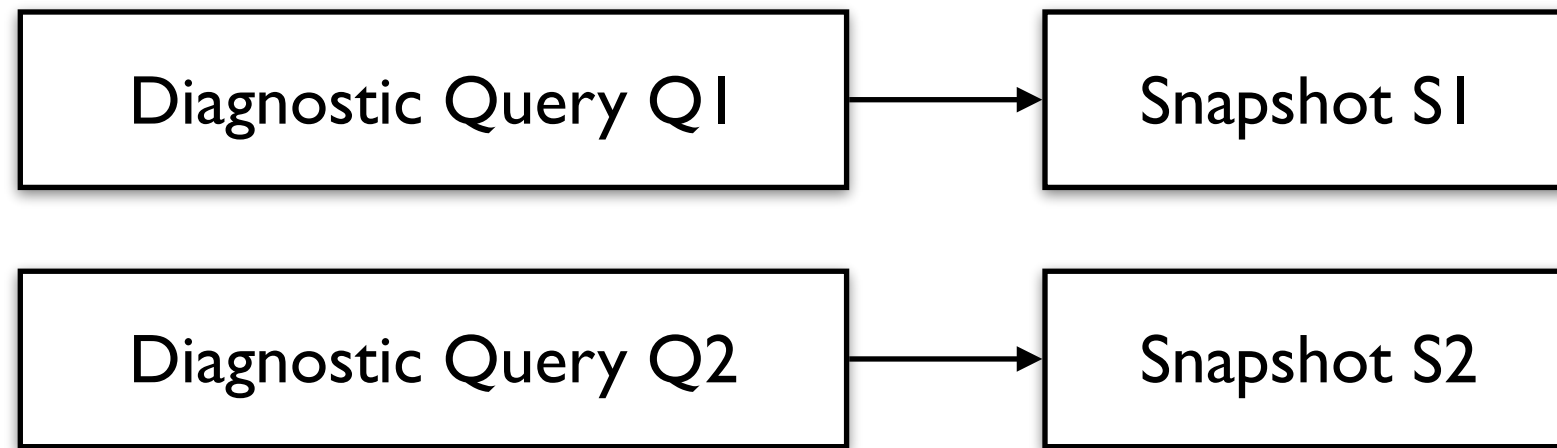
- Confluo provides linearizable snapshots, i.e.,
 - Each snapshot is atomic
 - Snapshots are totally ordered, i.e.,
 - if S1 “happens before” S2, S2 must contain all the changes in S1

Consistency in Distributed Diagnosis



- Confluo provides linearizable snapshots, i.e.,
 - Each snapshot is atomic
 - Snapshots are totally ordered, i.e.,
 - if S1 “happens before” S2, S2 must contain all the changes in S1
- **Limitation:** Does not consider stack delays in ordering packets *across* end-hosts

Consistency in Distributed Diagnosis



- Confluo provides linearizable snapshots, i.e.,
 - Each snapshot is atomic
 - Snapshots are totally ordered, i.e.,
 - if S1 “happens before” S2, S2 must contain all the changes in S1
- **Limitation:** Does not consider stack delays in ordering packets *across* end-hosts

Please see our paper for details on snapshot algorithm!

Evaluation

- **Setup:**
 - ▶ **Servers:** 12 core 2.3 GHz Xeon CPUs, 252GB RAM
 - ▶ **Network:** 10Gbps links, Pica8 P-3297 switches

Evaluation

- **Setup:**
 - ▶ **Servers:** 12 core 2.3 GHz Xeon CPUs, 252GB RAM
 - ▶ **Network:** 10Gbps links, Pica8 P-3297 switches
- **Summary of Results:**
 - ▶ Capture packet headers **at line rate > 10Gbps** while evaluating **1000s of triggers & 10s of filters** with **minimal CPU %**

Evaluation

- **Setup:**
 - ▶ **Servers:** 12 core 2.3 GHz Xeon CPUs, 252GB RAM
 - ▶ **Network:** 10Gbps links, Pica8 P-3297 switches
- **Summary of Results:**
 - ▶ Capture packet headers **at line rate > 10Gbps** while evaluating **1000s of triggers & 10s of filters** with **minimal CPU %**
 - ▶ Exploit rich telemetry data in packet headers to enable **large class of network monitoring and diagnosis applications**

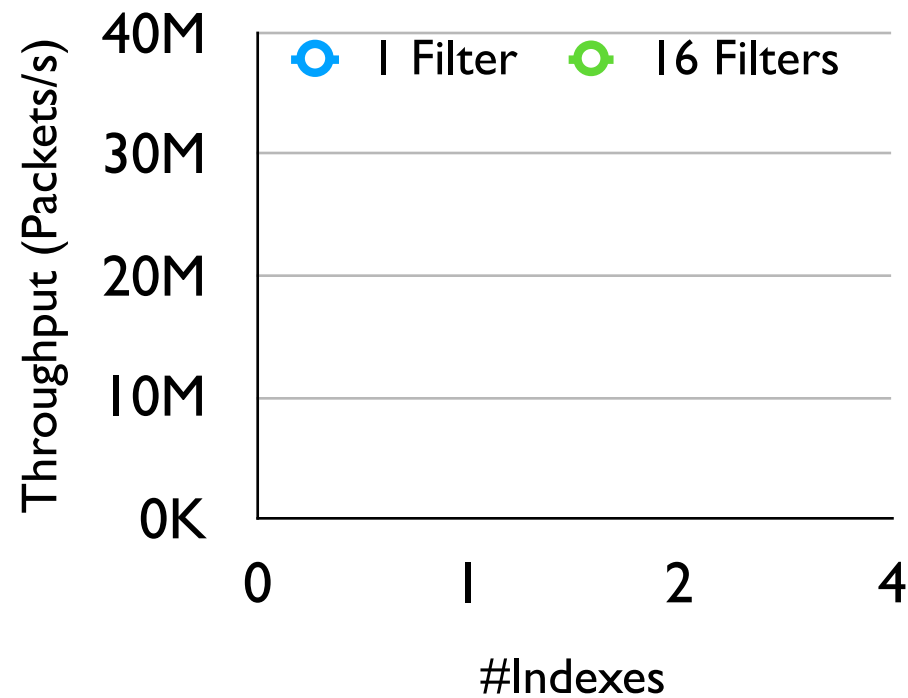
Evaluation

- **Setup:**
 - ▶ **Servers:** 12 core 2.3 GHz Xeon CPUs, 252GB RAM
 - ▶ **Network:** 10Gbps links, Pica8 P-3297 switches
- **Summary of Results:**
 - ▶ Capture packet headers **at line rate > 10Gbps** while evaluating **1000s of triggers & 10s of filters** with **minimal CPU %**
 - ▶ Exploit rich telemetry data in packet headers to enable **large class of network monitoring and diagnosis applications**

Please see our paper for detailed evaluation!

Atomic MultiLog Performance

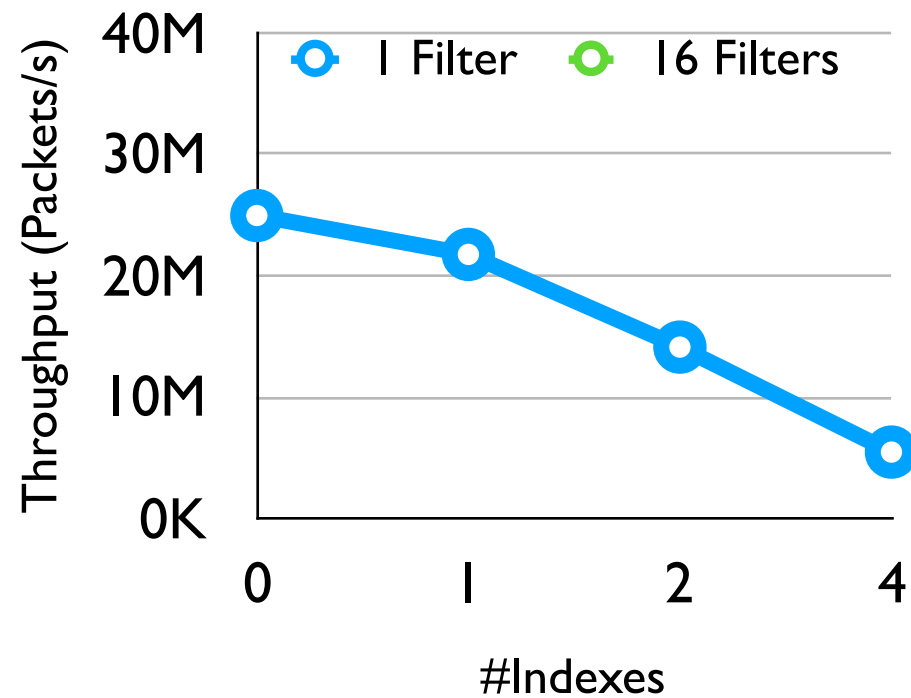
Atomic MultiLog Performance



Indexes: srcIP, srcPort, dstIP, dstPort, timestamp

Filter Templates: (f1) packets from VM A to VM B; (f2) packets to VM A; (f3) packets from VM A on destination port P; (f4) packets between (IP1, P1) and (IP2, P2); and (f5) packets to or from VM A.

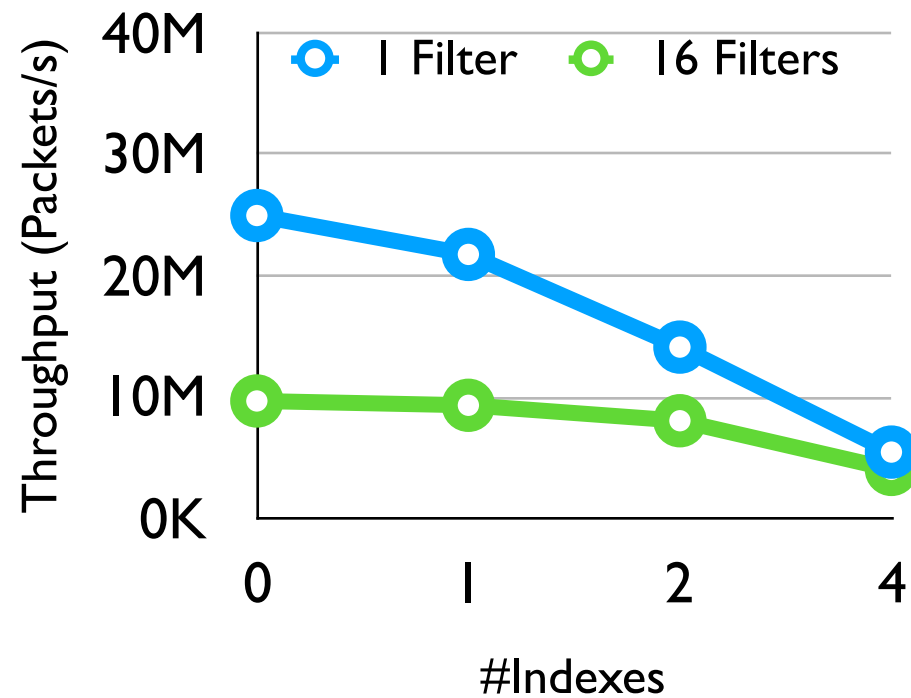
Atomic MultiLog Performance



Indexes: srcIP, srcPort, dstIP, dstPort, timestamp

Filter Templates: (f1) packets from VM A to VM B; (f2) packets to VM A; (f3) packets from VM A on destination port P; (f4) packets between (IP1, P1) and (IP2, P2); and (f5) packets to or from VM A.

Atomic MultiLog Performance

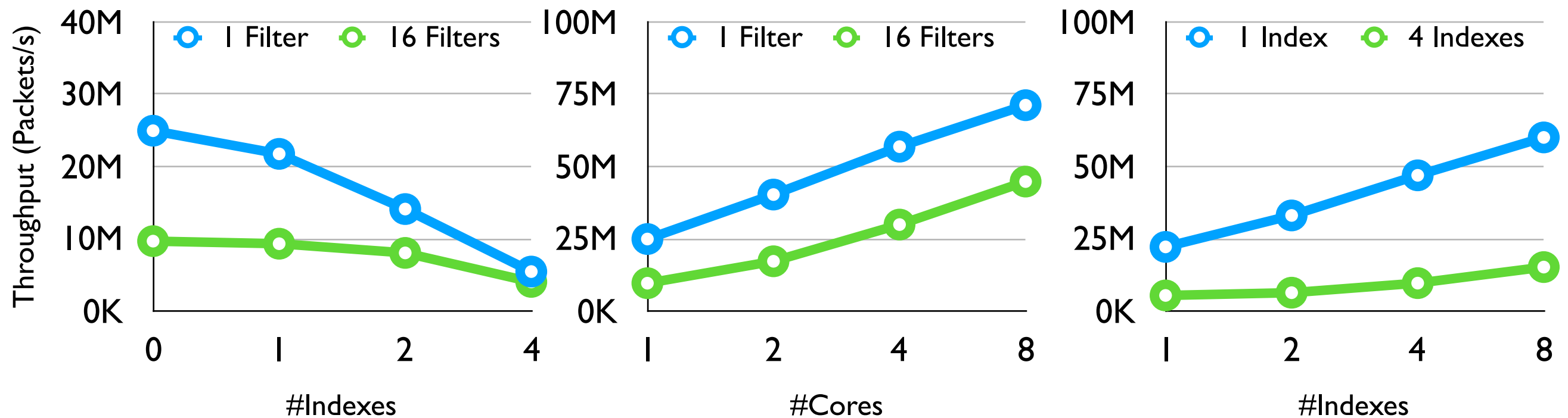


Indexes: srcIP, srcPort, dstIP, dstPort, timestamp

Filter Templates: (f1) packets from VM A to VM B; (f2) packets to VM A; (f3) packets from VM A on destination port P; (f4) packets between (IP1, P1) and (IP2, P2); and (f5) packets to or from VM A.

Takeaway: Packet write-rate degrades gracefully on adding more filters and indexes

Atomic MultiLog Performance



Indexes: srcIP, srcPort, dstIP, dstPort, timestamp

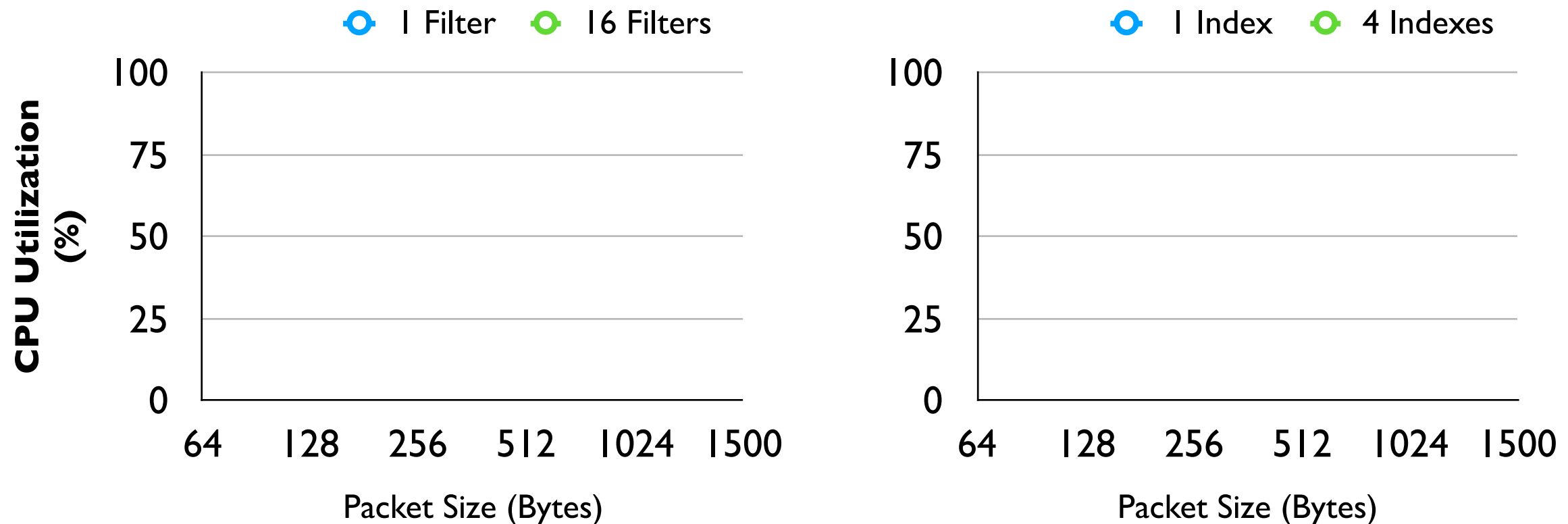
Filter Templates: (f1) packets from VM A to VM B; (f2) packets to VM A; (f3) packets from VM A on destination port P; (f4) packets between (IP₁, P₁) and (IP₂, P₂); and (f5) packets to or from VM A.

Takeaway: Packet write-rate degrades gracefully on adding more filters and indexes

Takeaway: Confluo's write throughput scales well with #cores due to inexpensive concurrency control

Atomic MultiLog Performance

CPU Utilization for processing packets at line rate on a 10Gbps link

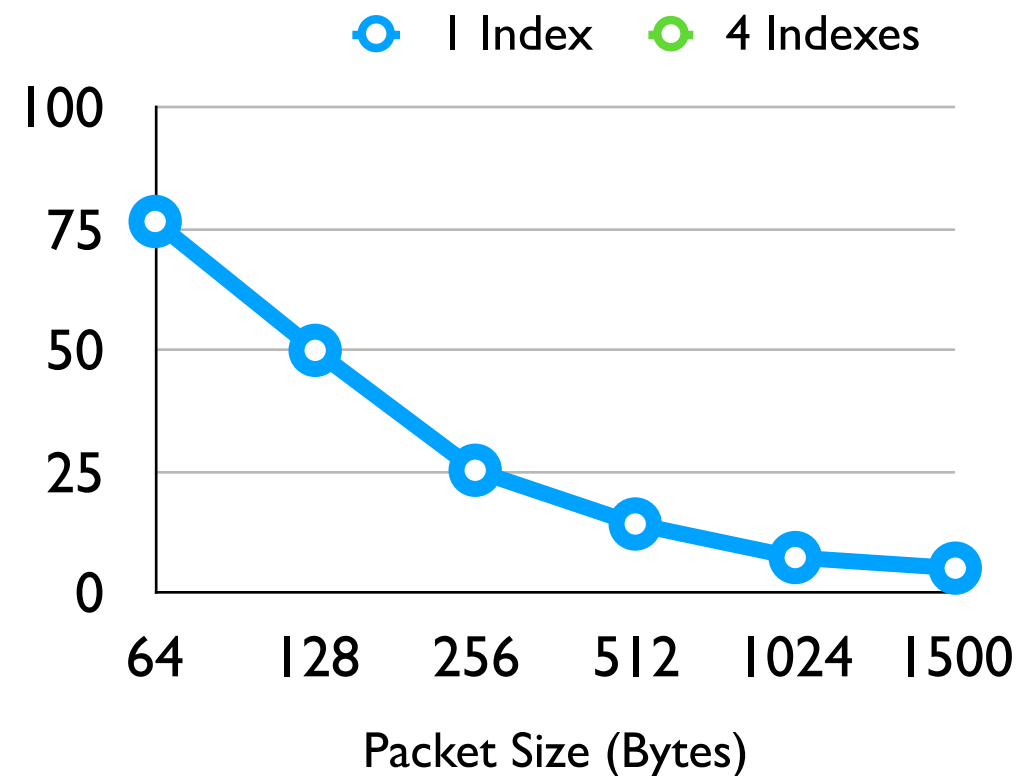
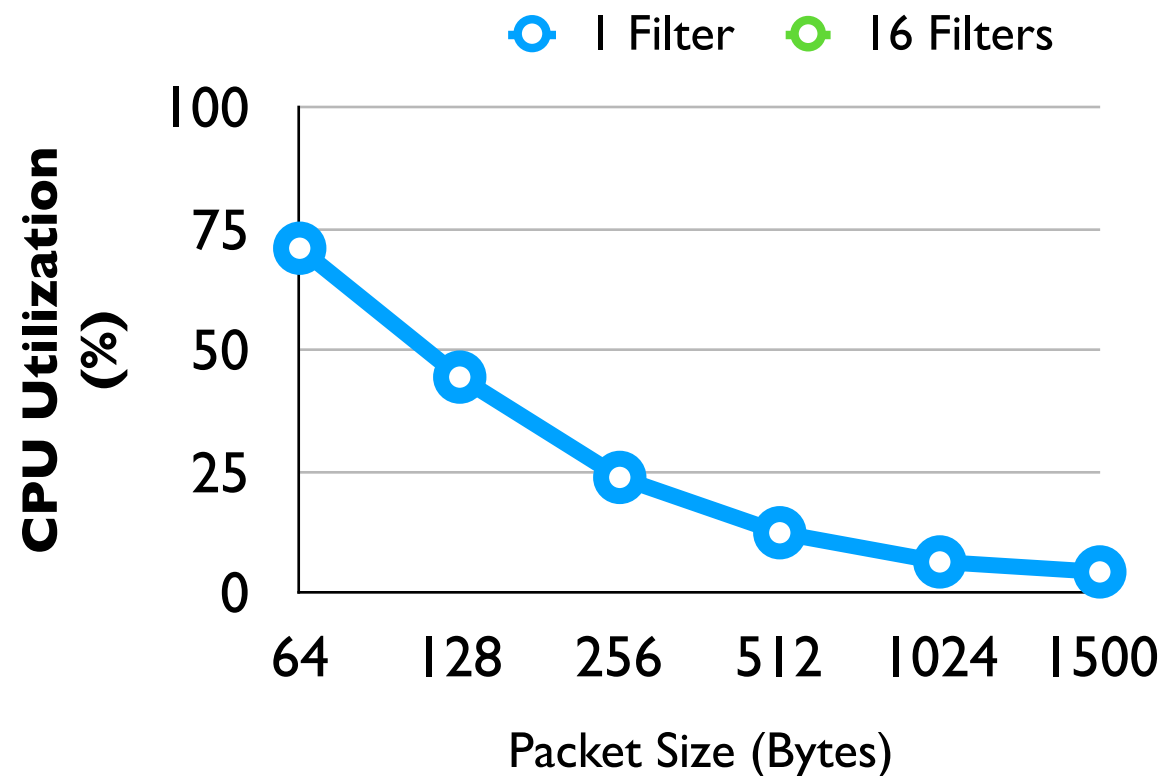


Indexes: srcIP, srcPort, dstIP, dstPort, timestamp

Filter Templates: (f1) packets from VM A to VM B; (f2) packets to VM A; (f3) packets from VM A on destination port P; (f4) packets between (IP1, P1) and (IP2, P2); and (f5) packets to or from VM A.

Atomic MultiLog Performance

CPU Utilization for processing packets at line rate on a 10Gbps link

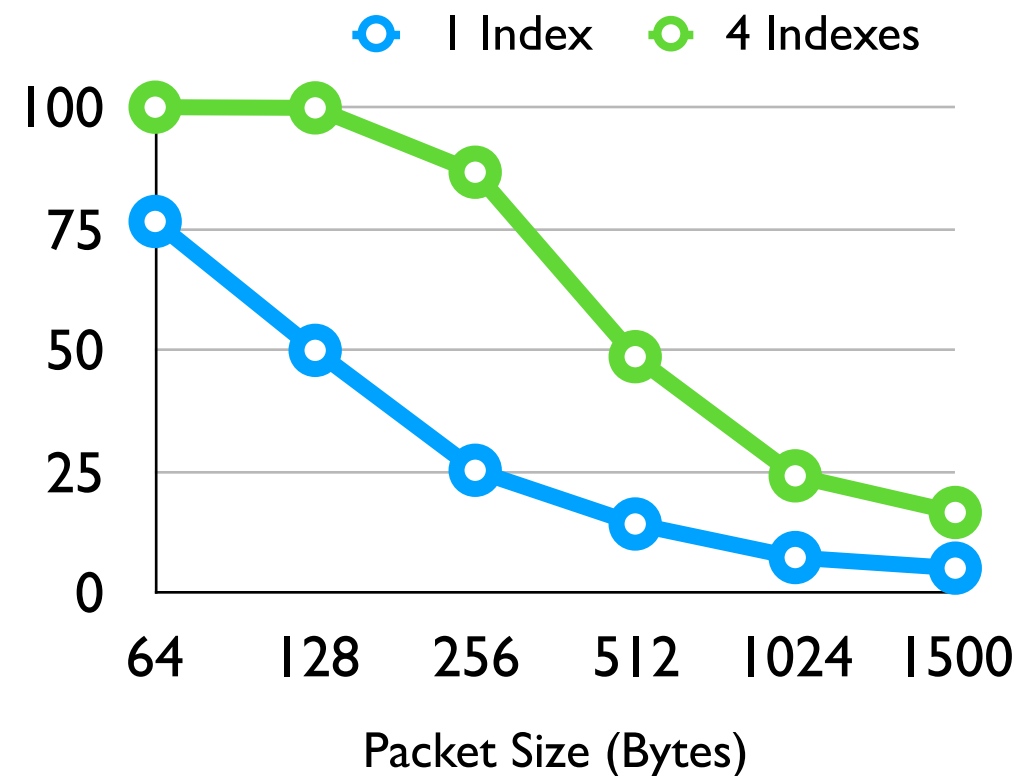
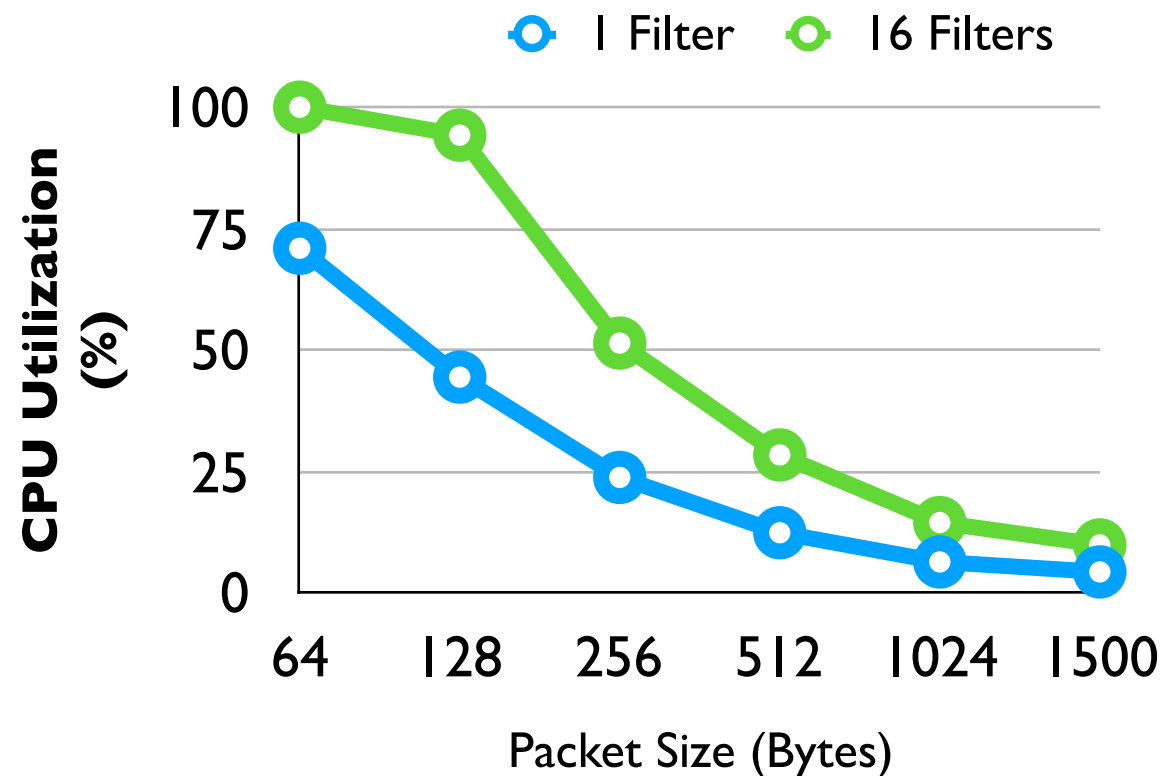


Indexes: srcIP, srcPort, dstIP, dstPort, timestamp

Filter Templates: (f1) packets from VM A to VM B; (f2) packets to VM A; (f3) packets from VM A on destination port P; (f4) packets between (IP1, P1) and (IP2, P2); and (f5) packets to or from VM A.

Atomic MultiLog Performance

CPU Utilization for processing packets at line rate on a 10Gbps link

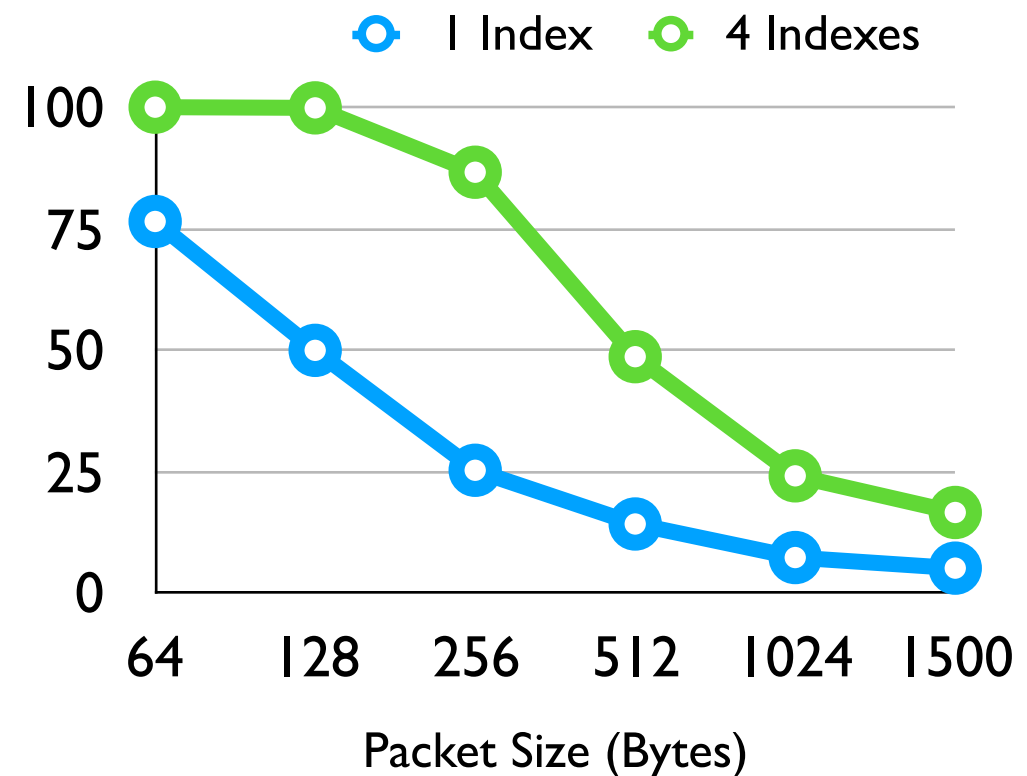
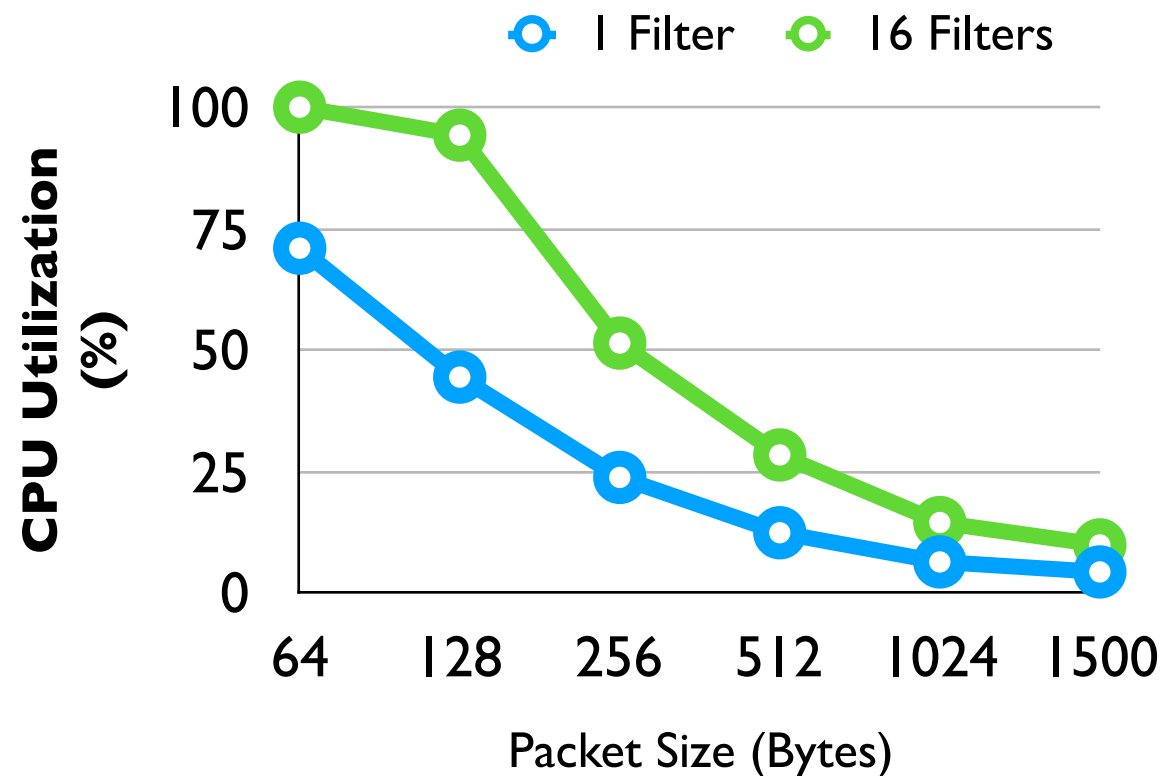


Indexes: srcIP, srcPort, dstIP, dstPort, timestamp

Filter Templates: (f1) packets from VM A to VM B; (f2) packets to VM A; (f3) packets from VM A on destination port P; (f4) packets between (IP1, P1) and (IP2, P2); and (f5) packets to or from VM A.

Atomic MultiLog Performance

CPU Utilization for processing packets at line rate on a 10Gbps link



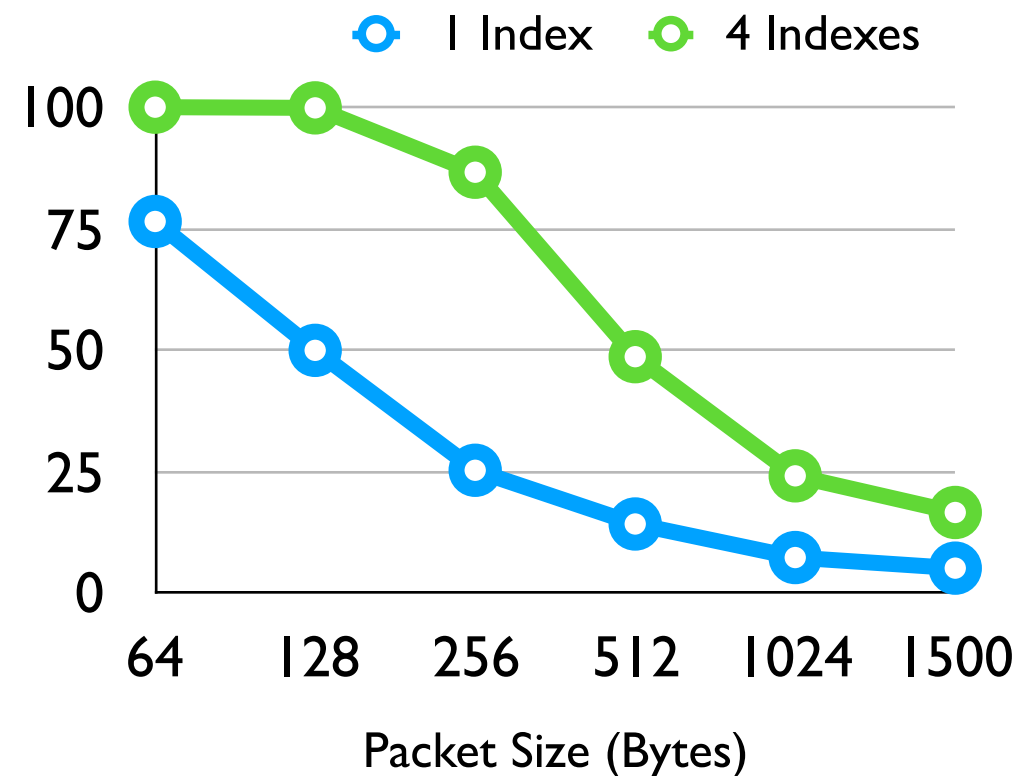
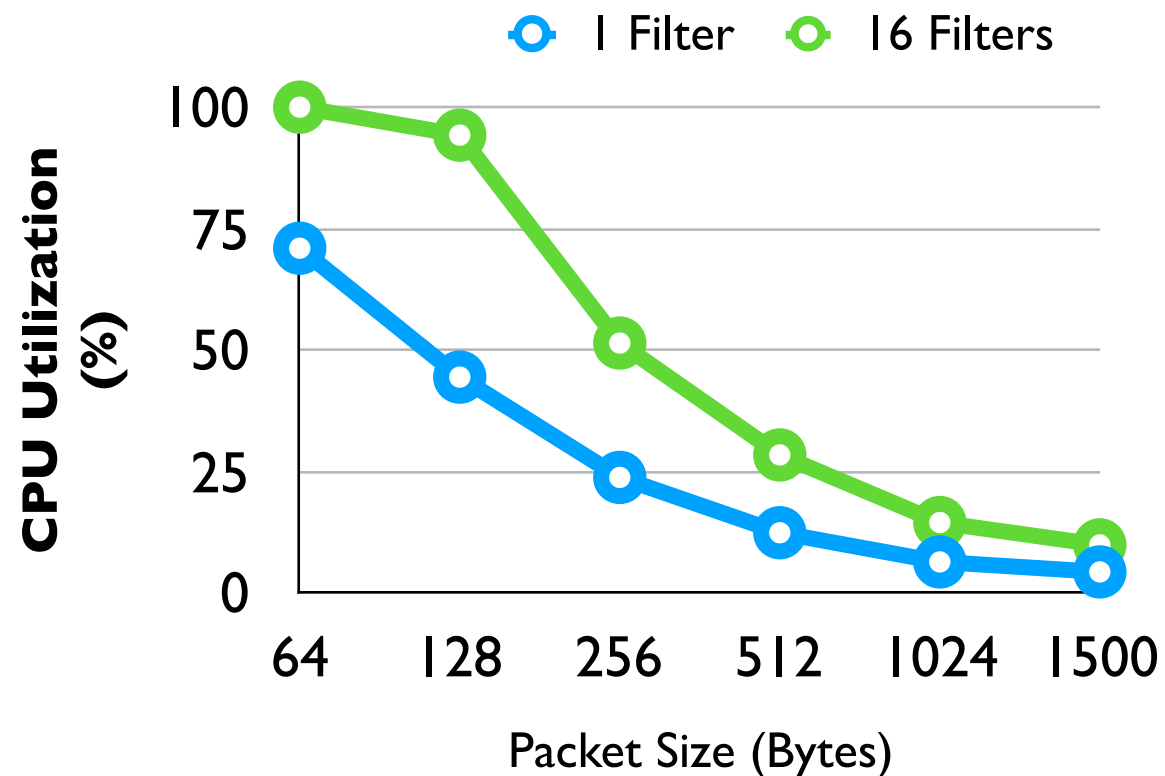
Indexes: srcIP, srcPort, dstIP, dstPort, timestamp

Filter Templates: (f1) packets from VM A to VM B; (f2) packets to VM A; (f3) packets from VM A on destination port P; (f4) packets between (IP1, P1) and (IP2, P2); and (f5) packets to or from VM A.

Takeaway: Confluo can capture packets at line rate for 10Gbps links for a wide range of packet sizes using a single CPU core.

Atomic MultiLog Performance

CPU Utilization for processing packets at line rate on a 10Gbps link



Indexes: srcIP, srcPort, dstIP, dstPort, timestamp

Filter Templates: (f1) packets from VM A to VM B; (f2) packets to VM A; (f3) packets from VM A on destination port P; (f4) packets between (IP1, P1) and (IP2, P2); and (f5) packets to or from VM A.

Takeaway: Confluo can capture packets at line rate for 10Gbps links for a wide range of packet sizes using a single CPU core.

Many more results in the paper...

General Applicability and Impact

General Applicability and Impact

- Confluo exploits three properties: fixed-sized attributes, append-only, non-transactional

General Applicability and Impact

- Confluo exploits three properties: fixed-sized attributes, append-only, non-transactional
- Many other applications exhibit similar properties:
 - Distributed messaging, e.g., Apache Kafka, Amazon Kinesis, etc.
 - Time-series databases, e.g., OpenTSDB, InfluxDB, etc.

General Applicability and Impact

- Confluo exploits three properties: fixed-sized attributes, append-only, non-transactional
- Many other applications exhibit similar properties:
 - Distributed messaging, e.g., Apache Kafka, Amazon Kinesis, etc.
 - Time-series databases, e.g., OpenTSDB, InfluxDB, etc.
- We are actively exploring Confluo applicability beyond network monitoring and debugging

General Applicability and Impact

- Confluo exploits three properties: fixed-sized attributes, append-only, non-transactional
- Many other applications exhibit similar properties:
 - Distributed messaging, e.g., Apache Kafka, Amazon Kinesis, etc.
 - Time-series databases, e.g., OpenTSDB, InfluxDB, etc.
- We are actively exploring Confluo applicability beyond network monitoring and debugging

Open Source: <https://www.github.com/ucbrise/confluo>



Unwatch ▼

103



Unstar

1,324



Fork

182

Confluo Summary

Confluo Summary

Distributed monitoring and diagnosis stack for high-speed networks

Introduces a new data structure: **Atomic MultiLog**

Exploits **structure in network telemetry data** to support:

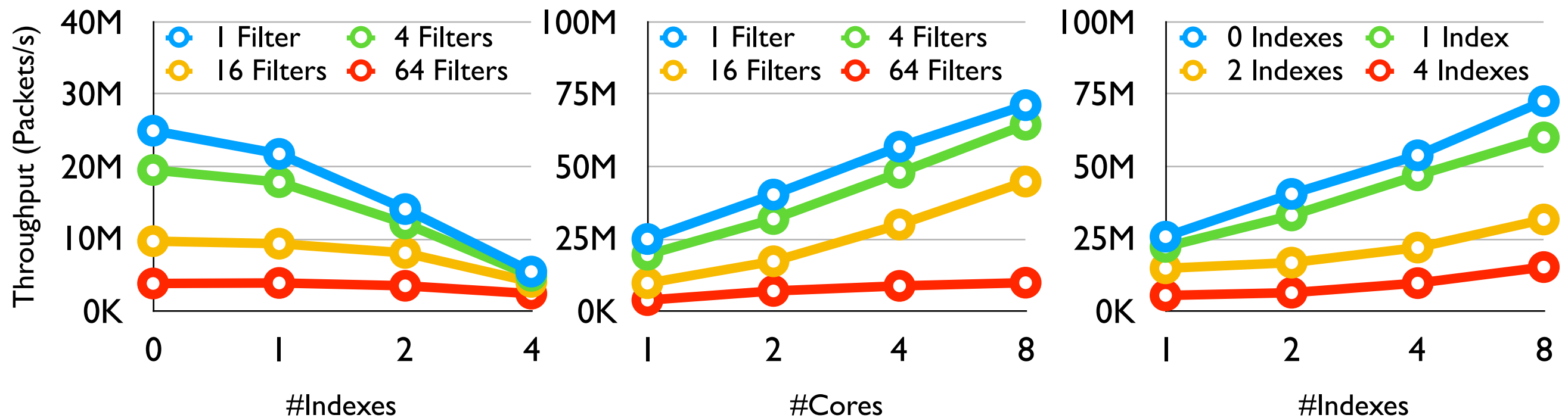
- Rich monitoring
- Low-overhead diagnosis and,
- High-concurrency reads and writes

<https://www.github.com/ucbrise/confluo>

THANK YOU!

Backup Slides

Atomic MultiLog Performance



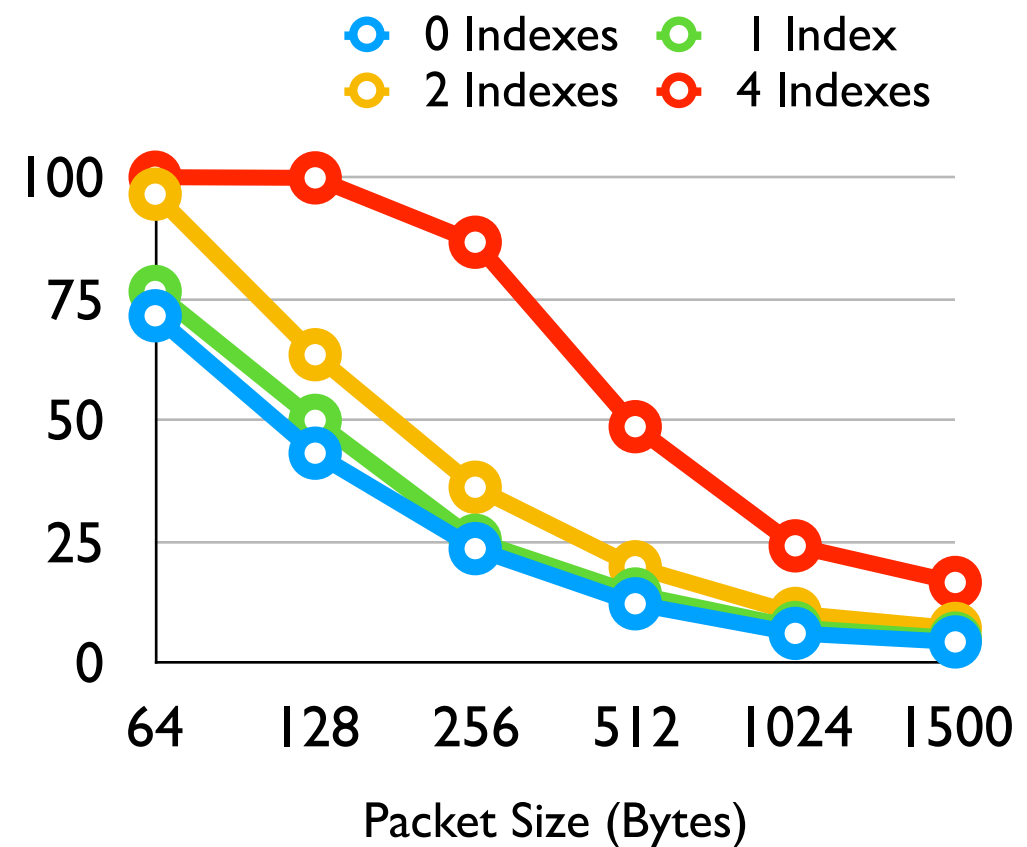
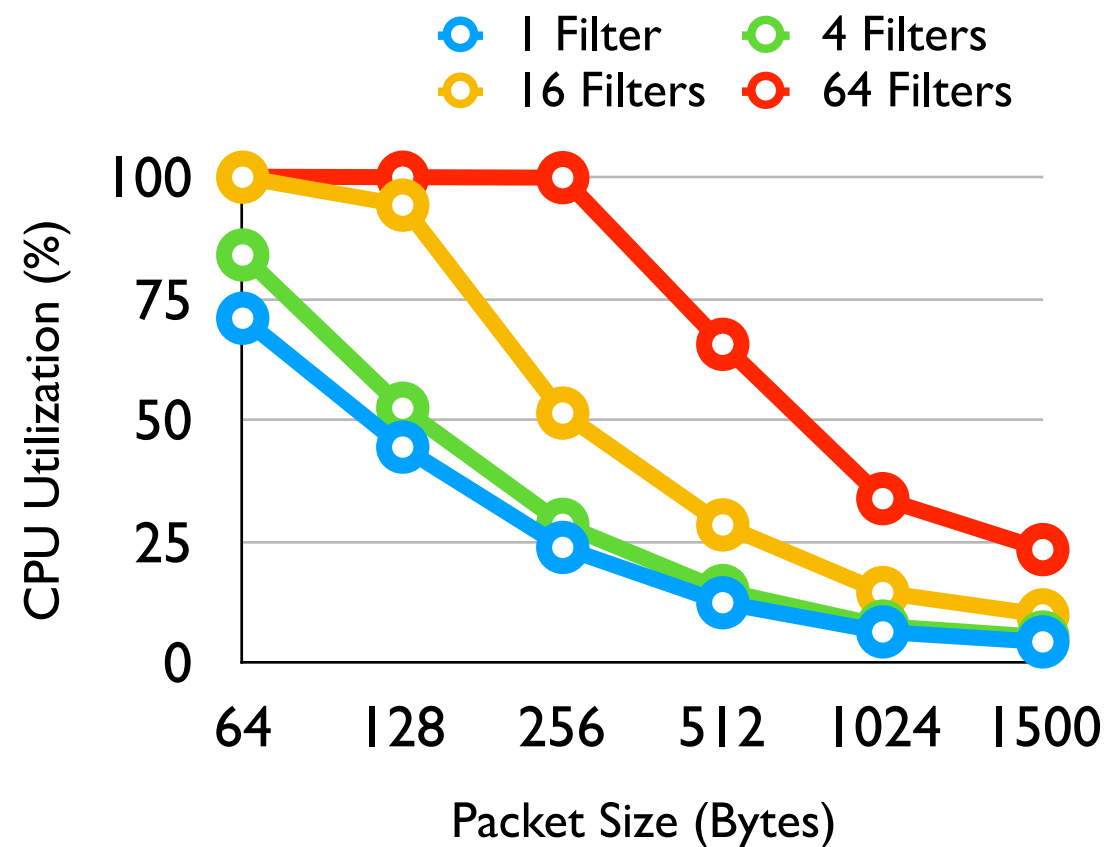
Indexes: srcIP, srcPort, dstIP, dstPort, timestamp

Filter Templates: (f1) packets from VM A to VM B; (f2) packets to VM A; (f3) packets from VM A on destination port P; (f4) packets between (IP₁, P₁) and (IP₂, P₂); and (f5) packets to or from VM A.

Takeaway: Packet write-rate degrades gracefully on adding more filters and indexes

Takeaway: Confluo's write throughput scales well with #cores owing to its inexpensive lock-free concurrency

Atomic MultiLog Performance

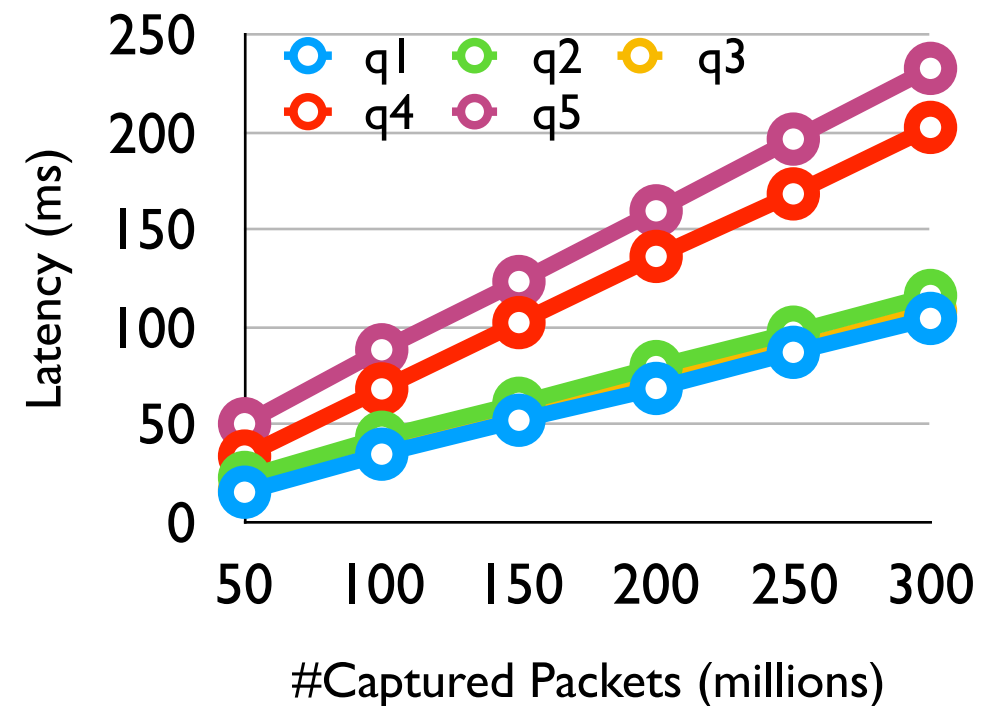
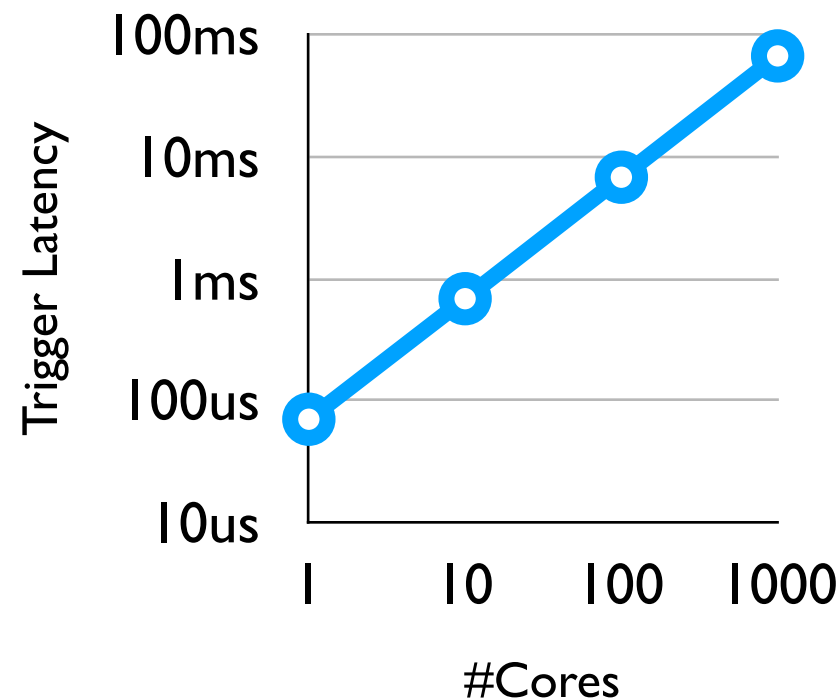
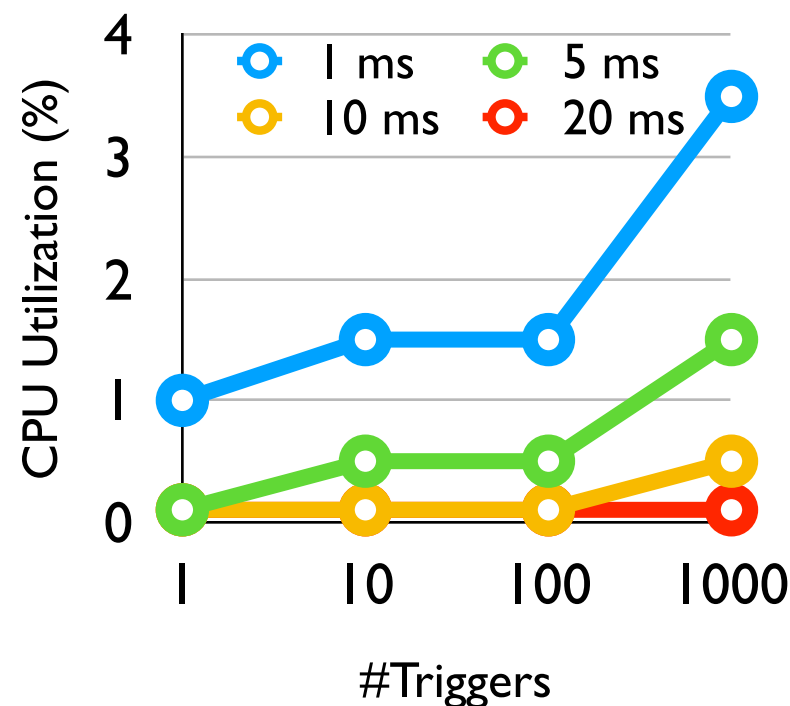


Indexes: srcIP, srcPort, dstIP, dstPort, timestamp

Filter Templates: (f1) packets from VM A to VM B; (f2) packets to VM A; (f3) packets from VM A on destination port P; (f4) packets between (IP₁, P₁) and (IP₂, P₂); and (f5) packets to or from VM A.

Takeaway: At line rate of 10Gbps, Confluo can handle average packet size as small as 128B with 16 filters and 2 indexes on a single core.

Atomic MultiLog Performance



Trigger Templates: `aggregate > threshold, aggregate in {sum(pktSize), min(priority), max(CWND), count(pkts), ...}`

Query Templates: (q1) packets from VM A to VM B; (q2) packets to VM A; (q3) packets from VM A on destination port P; (q4) packets between (IP1, P1) and (IP2, P2); and (q5) packets to or from VM A.

Takeaway: Confluo can evaluate 1000s of trigger queries with less than 4% CPU utilization at 1ms intervals, and with latency less than 70μs.

Takeaway: Diagnostic query latency in Confluo increases linearly with number of captured packets in Confluo.

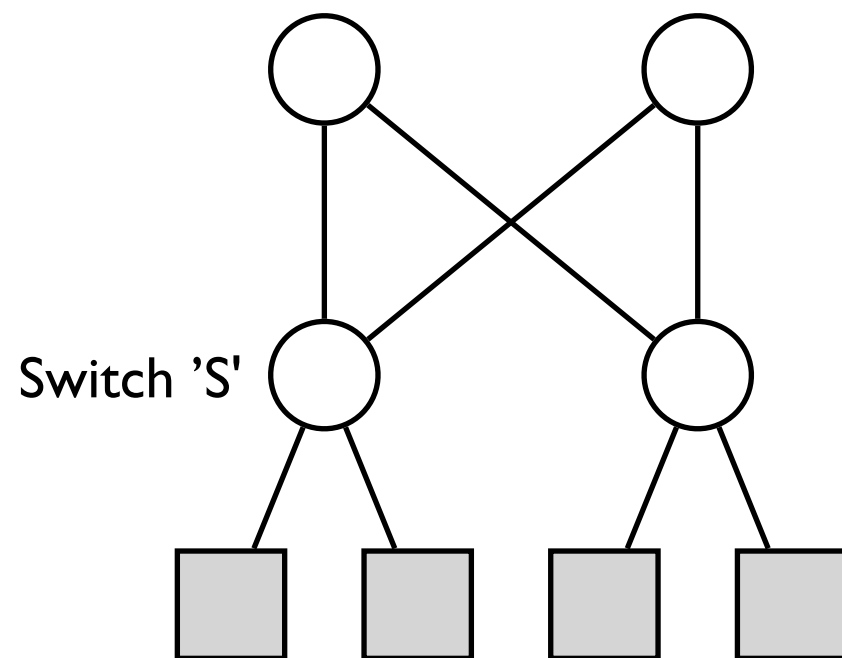
Monitoring & Diagnosis Scenario

Monitoring & Diagnosis Scenario

“Detect TCP packet losses, determine if it is due to difference in flow priorities.”

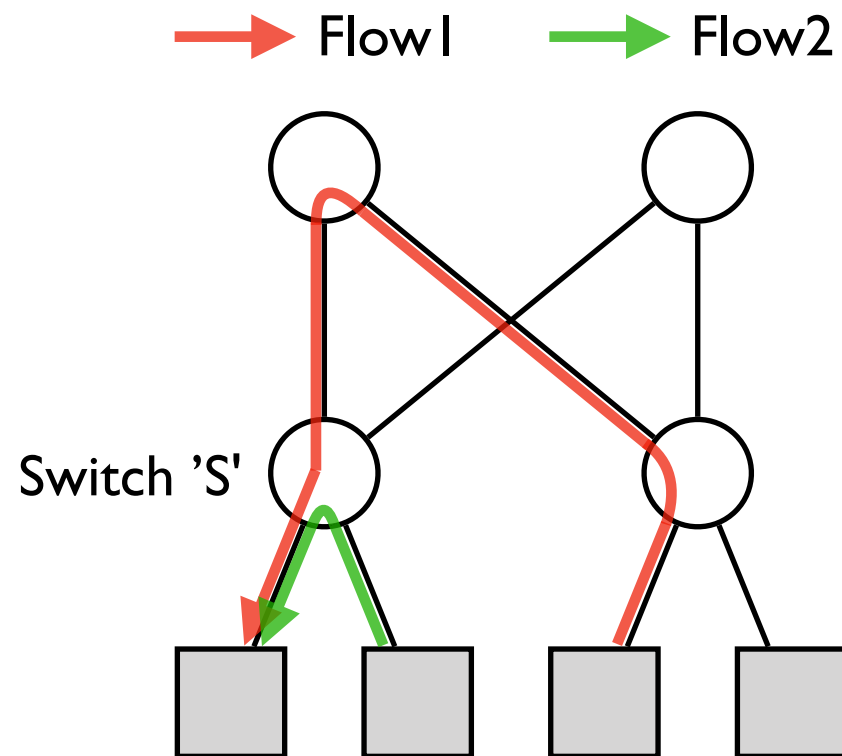
Monitoring & Diagnosis Scenario

“Detect TCP packet losses, determine if it is due to difference in flow priorities.”



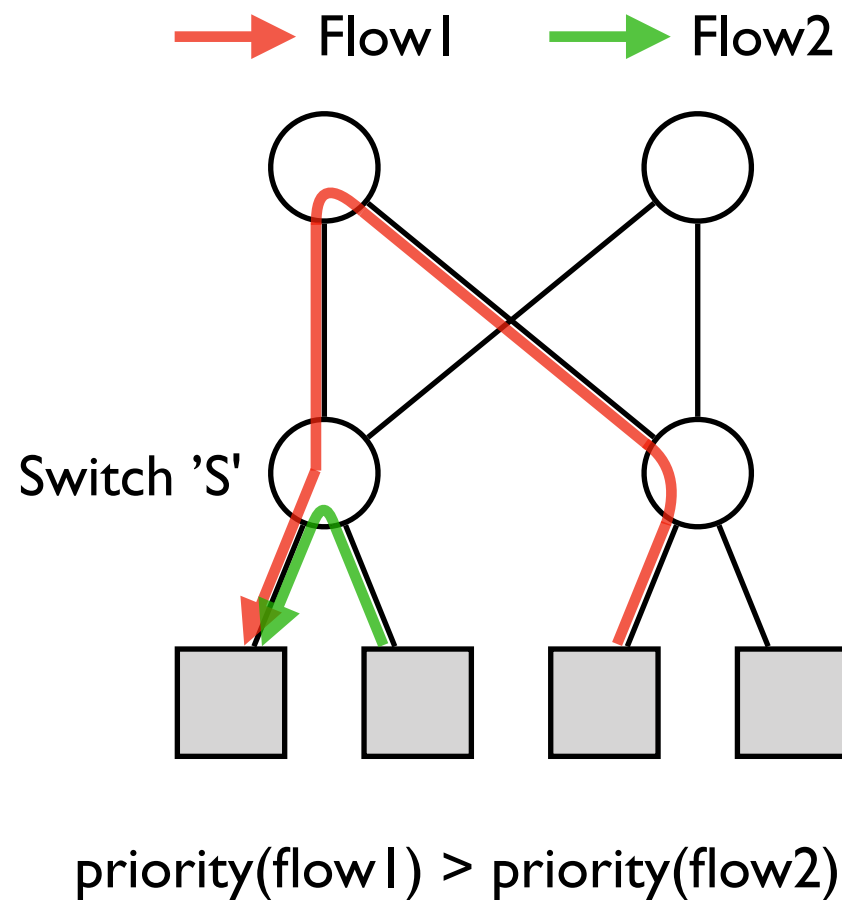
Monitoring & Diagnosis Scenario

“Detect TCP packet losses, determine if it is due to difference in flow priorities.”



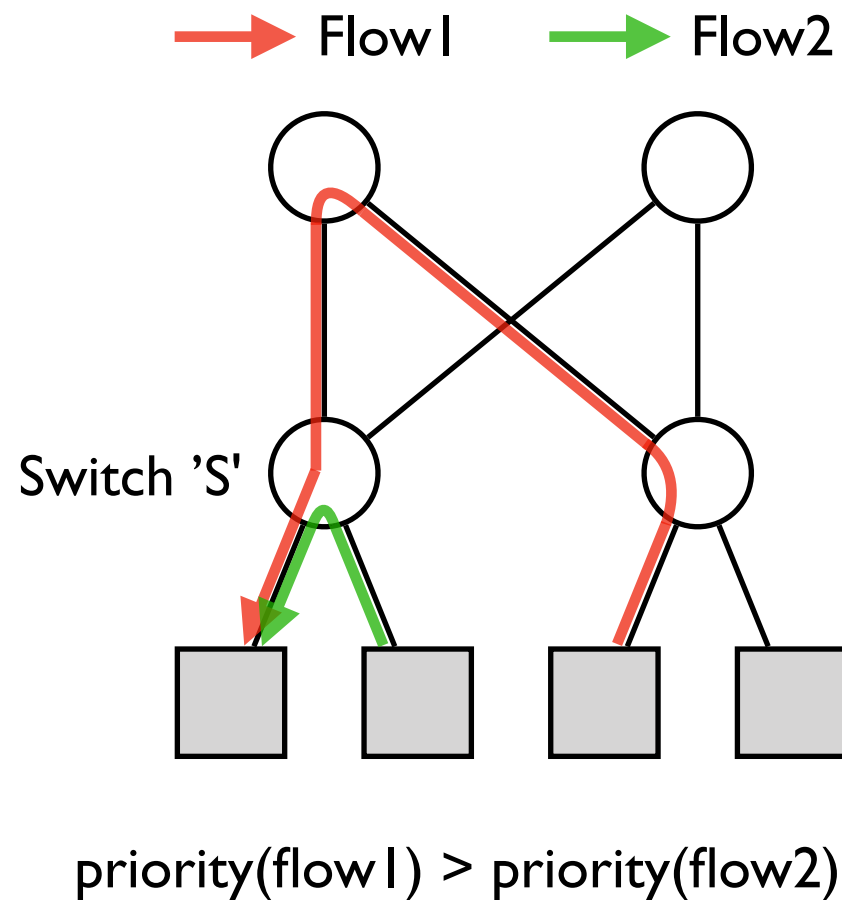
Monitoring & Diagnosis Scenario

“Detect TCP packet losses, determine if it is due to difference in flow priorities.”



Monitoring & Diagnosis Scenario

“Detect TCP packet losses, determine if it is due to difference in flow priorities.”

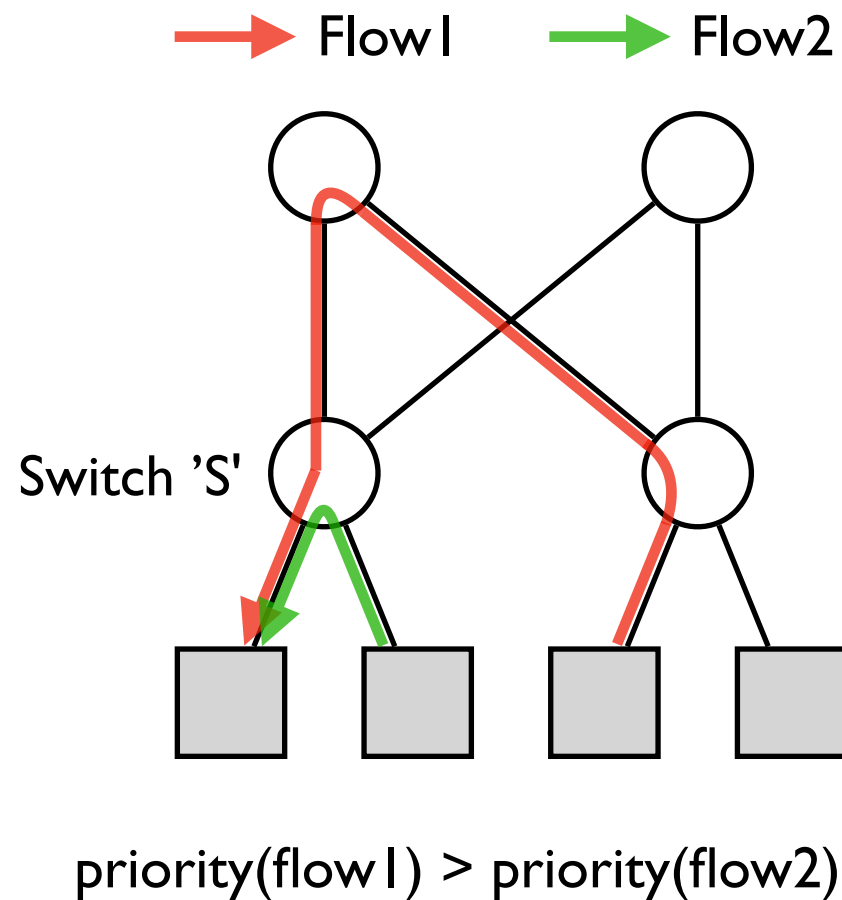


Monitoring:

- **Filter** TCP retransmissions as **pkt_drops**

Monitoring & Diagnosis Scenario

“Detect TCP packet losses, determine if it is due to difference in flow priorities.”

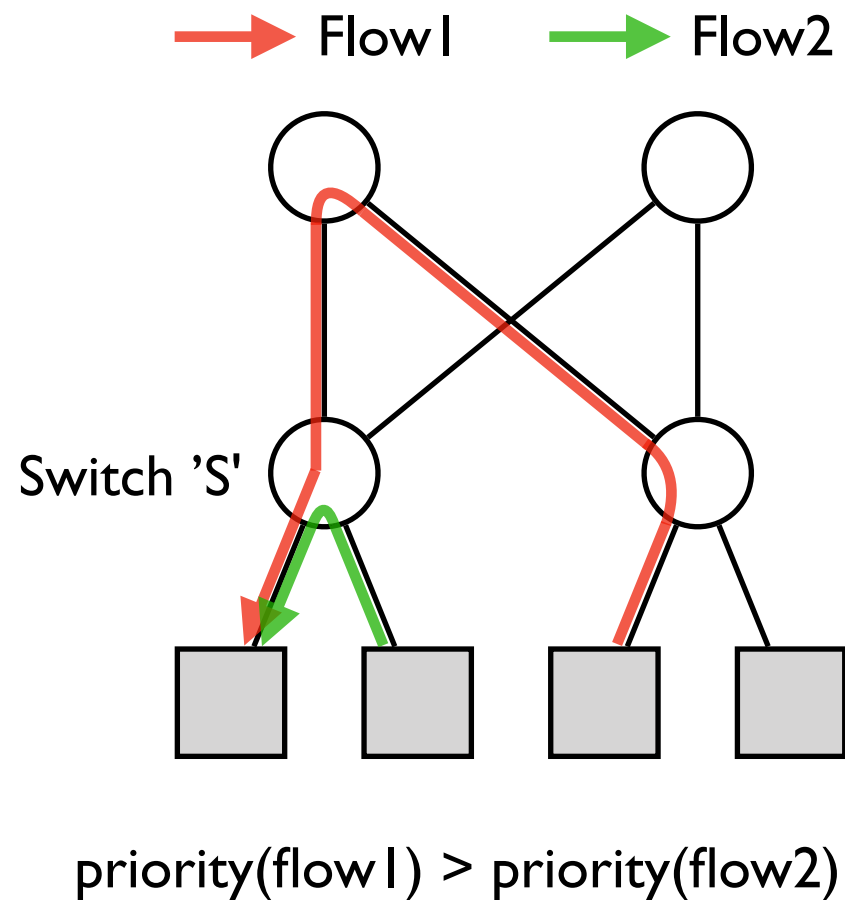


Monitoring:

- Filter TCP retransmissions as **pkt_drops**
- Aggregate **drop_count** on **pkt_drops**
- Trigger alert if **drop_count** $> T$ in **1ms** interval

Monitoring & Diagnosis Scenario

“Detect TCP packet losses, determine if it is due to difference in flow priorities.”



Monitoring:

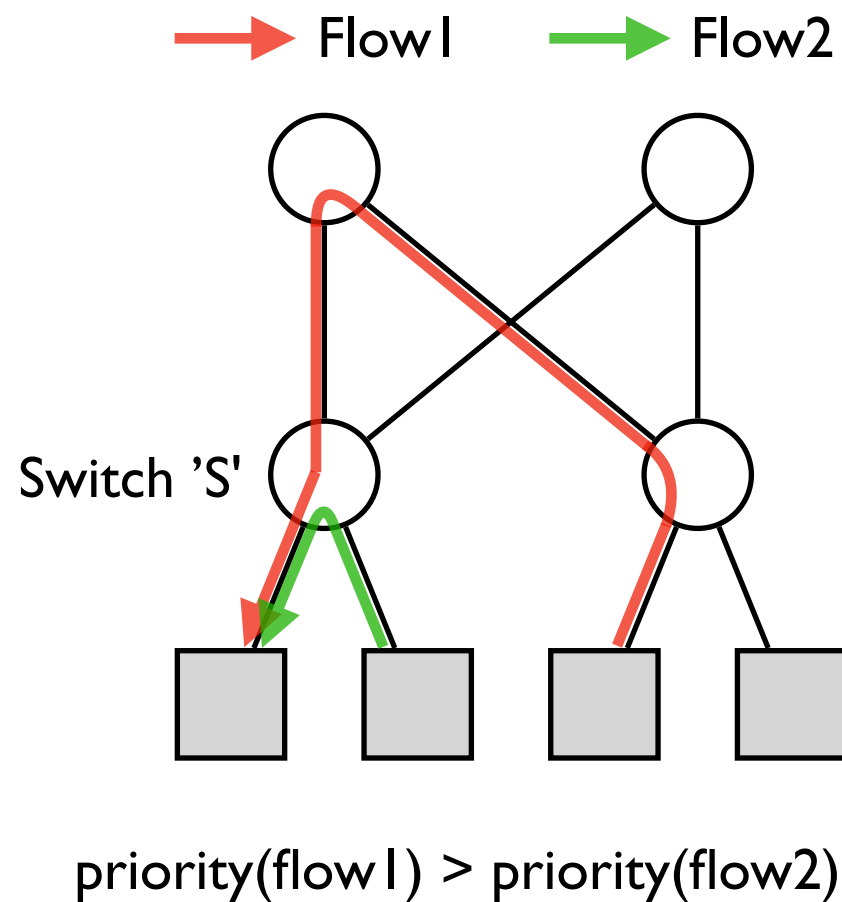
- Filter TCP retransmissions as **pkt_drops**
- Aggregate **drop_count** on **pkt_drops**
- Trigger alert if **drop_count > T** in **1ms** interval

Diagnosis: Check if:

- $\text{drop_count}(\text{flow1}) < \text{drop_count}(\text{flow2})$
- $\text{priority}(\text{flow1}) > \text{priority}(\text{flow2})$

Diagnosing Losses due to Flow Priorities

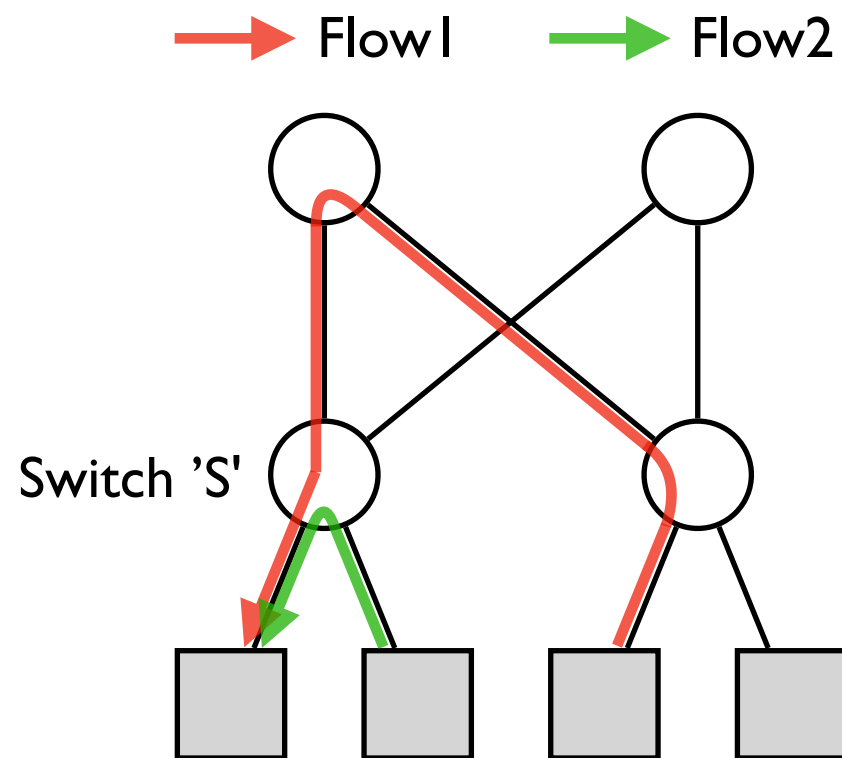
“Detect TCP packet losses, determine if it is due to difference in flow priorities.”



Diagnosing Losses due to Flow Priorities

“Detect TCP packet losses, determine if it is due to difference in flow priorities.”

Setup: 15 low priority flows & 1 high priority flow w/ 10Gbps links

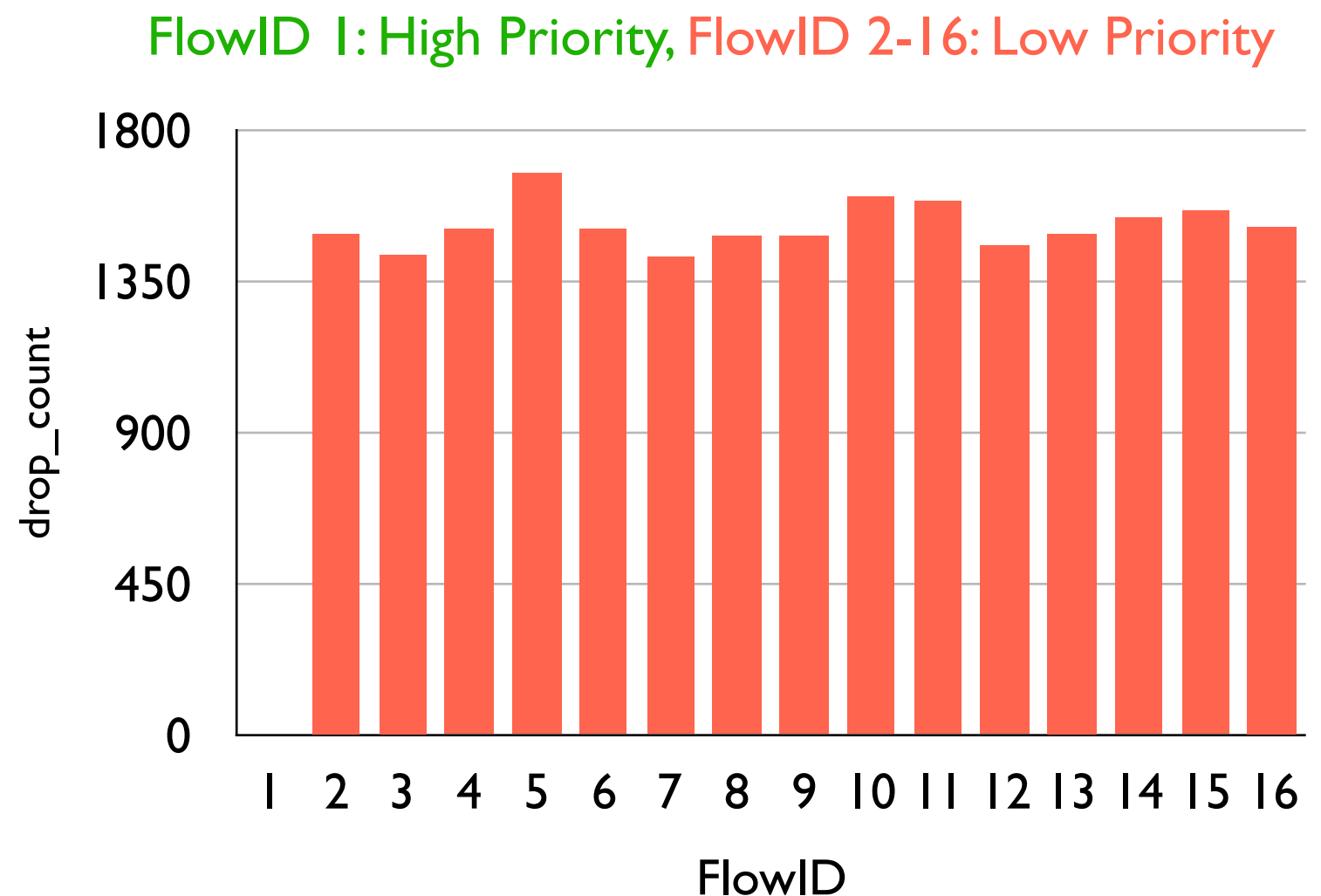
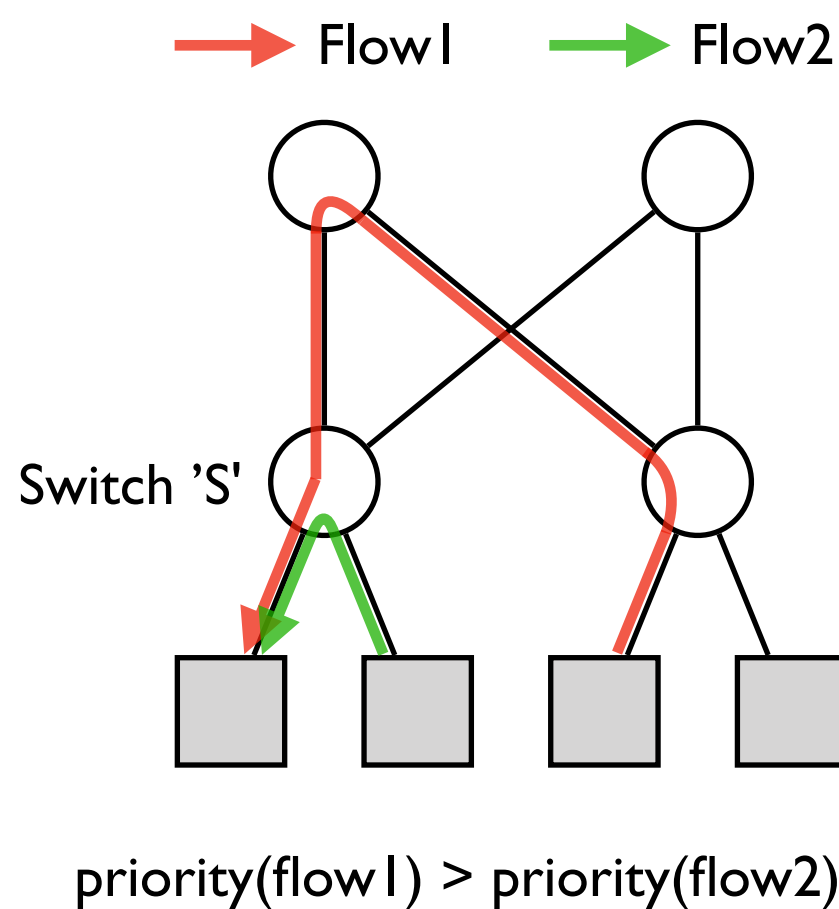


priority(flow 1) > priority(flow 2)

Diagnosing Losses due to Flow Priorities

“Detect TCP packet losses, determine if it is due to difference in flow priorities.”

Setup: 15 low priority flows & 1 high priority flow w/ 10Gbps links



Takeaway: Confluo is able to diagnose packet drops due to flow priorities.

Diagnosing Losses due to Flow Priorities

“Detect TCP packet losses, determine if it is due to difference in flow priorities.”

Diagnosing Losses due to Flow Priorities

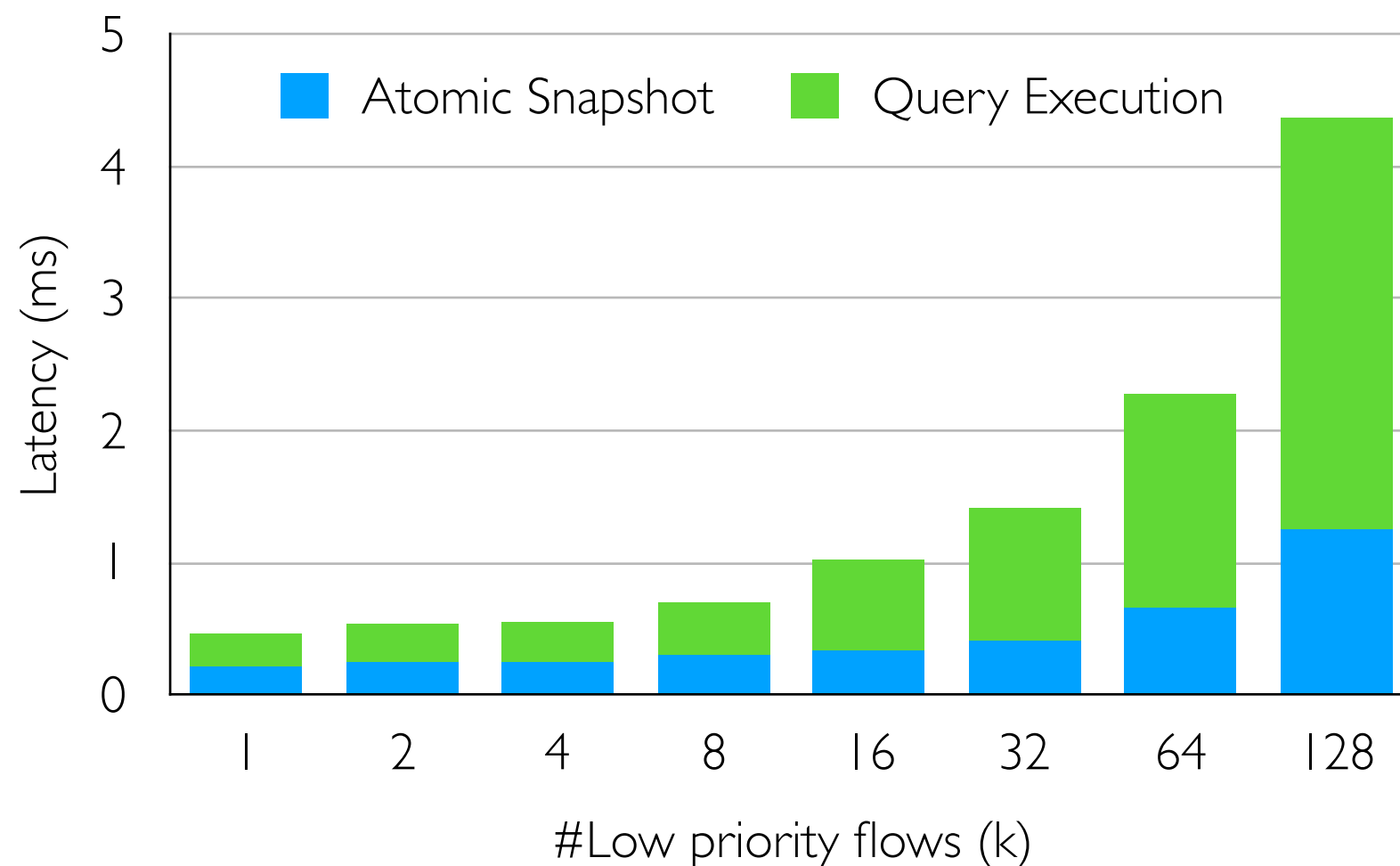
“Detect TCP packet losses, determine if it is due to difference in flow priorities.”

Setup: k low priority flows & 1 high priority flow w/ 10Gbps links

Diagnosing Losses due to Flow Priorities

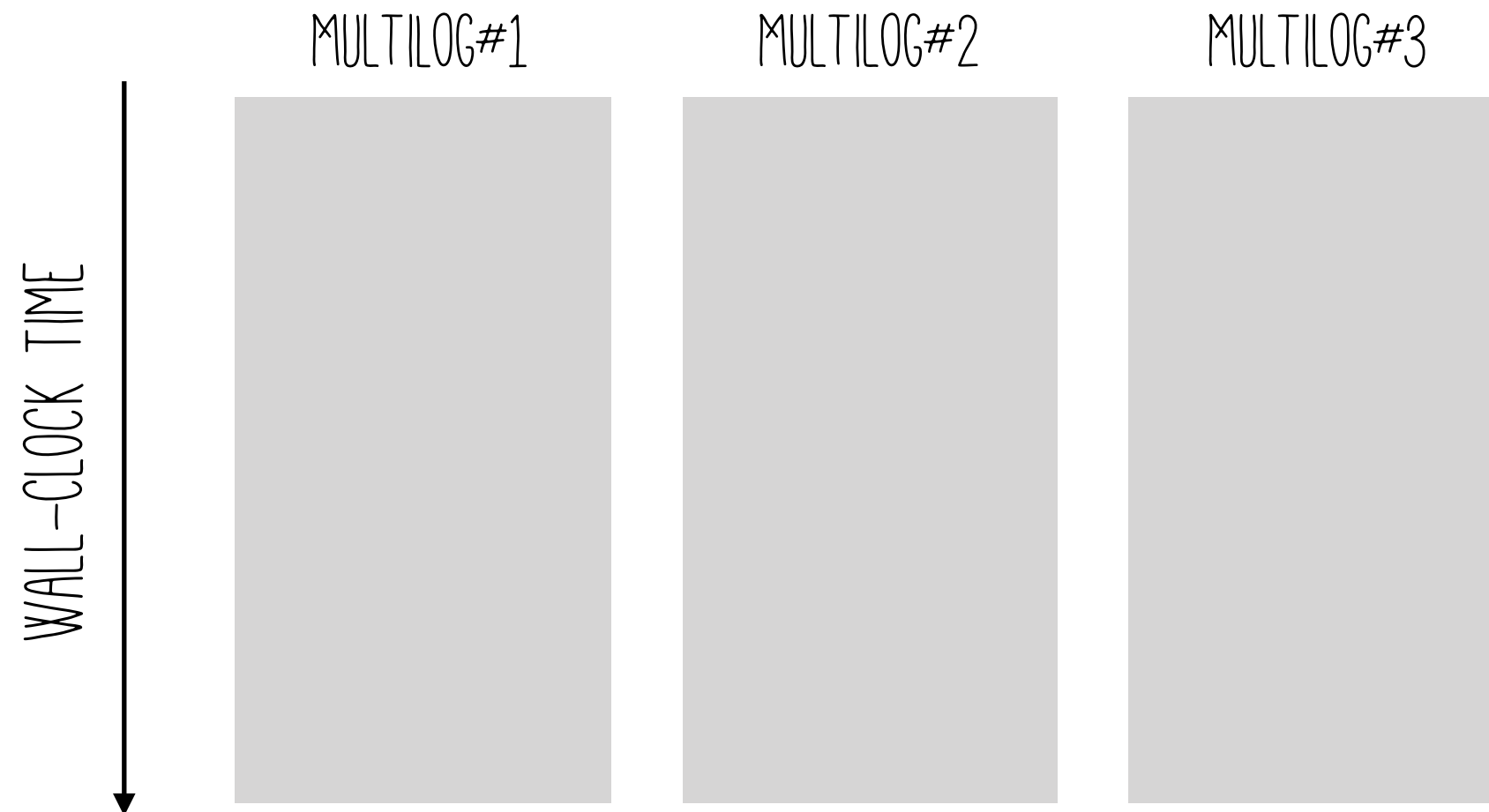
“Detect TCP packet losses, determine if it is due to difference in flow priorities.”

Setup: k low priority flows & 1 high priority flow w/ 10Gbps links



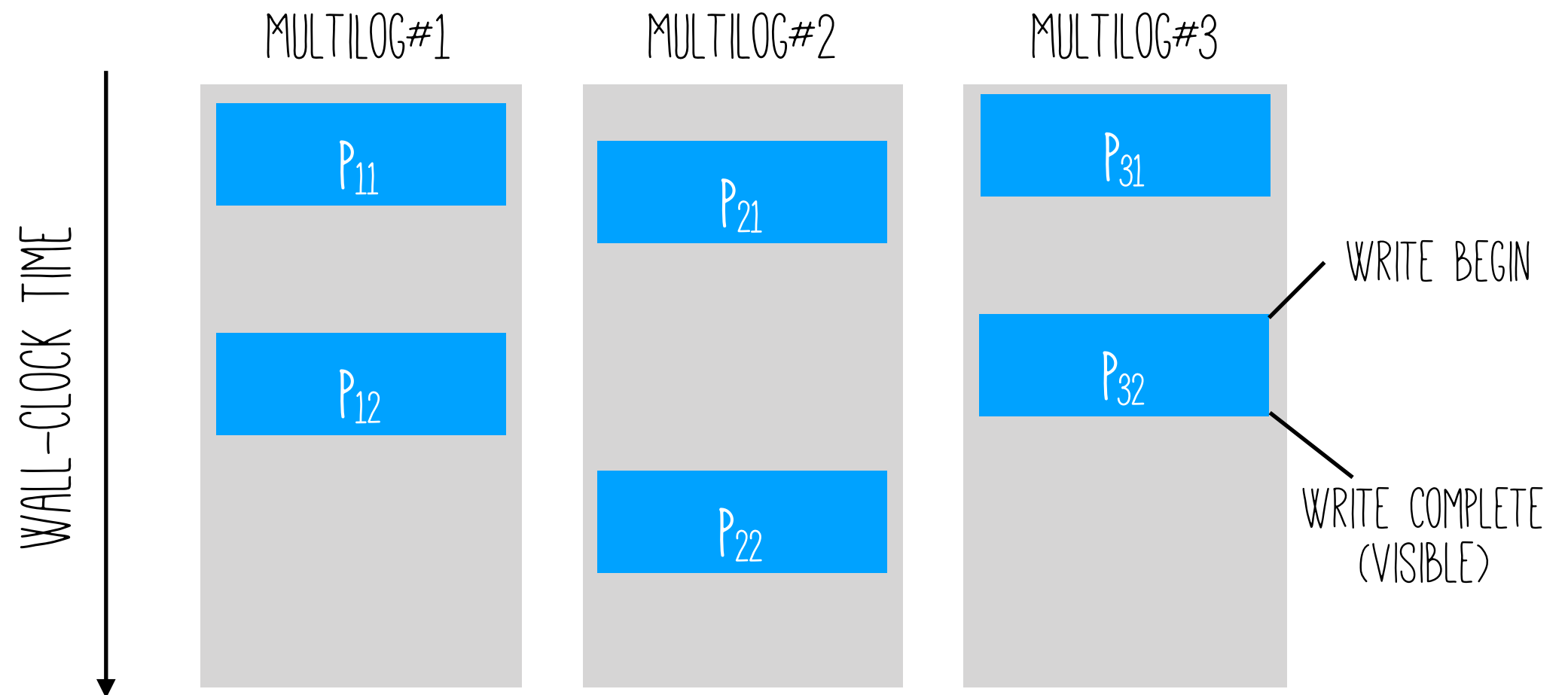
Takeaway: Confluo can diagnose issues across 100s of VMs in a few ms

Consistency in Distributed Diagnosis



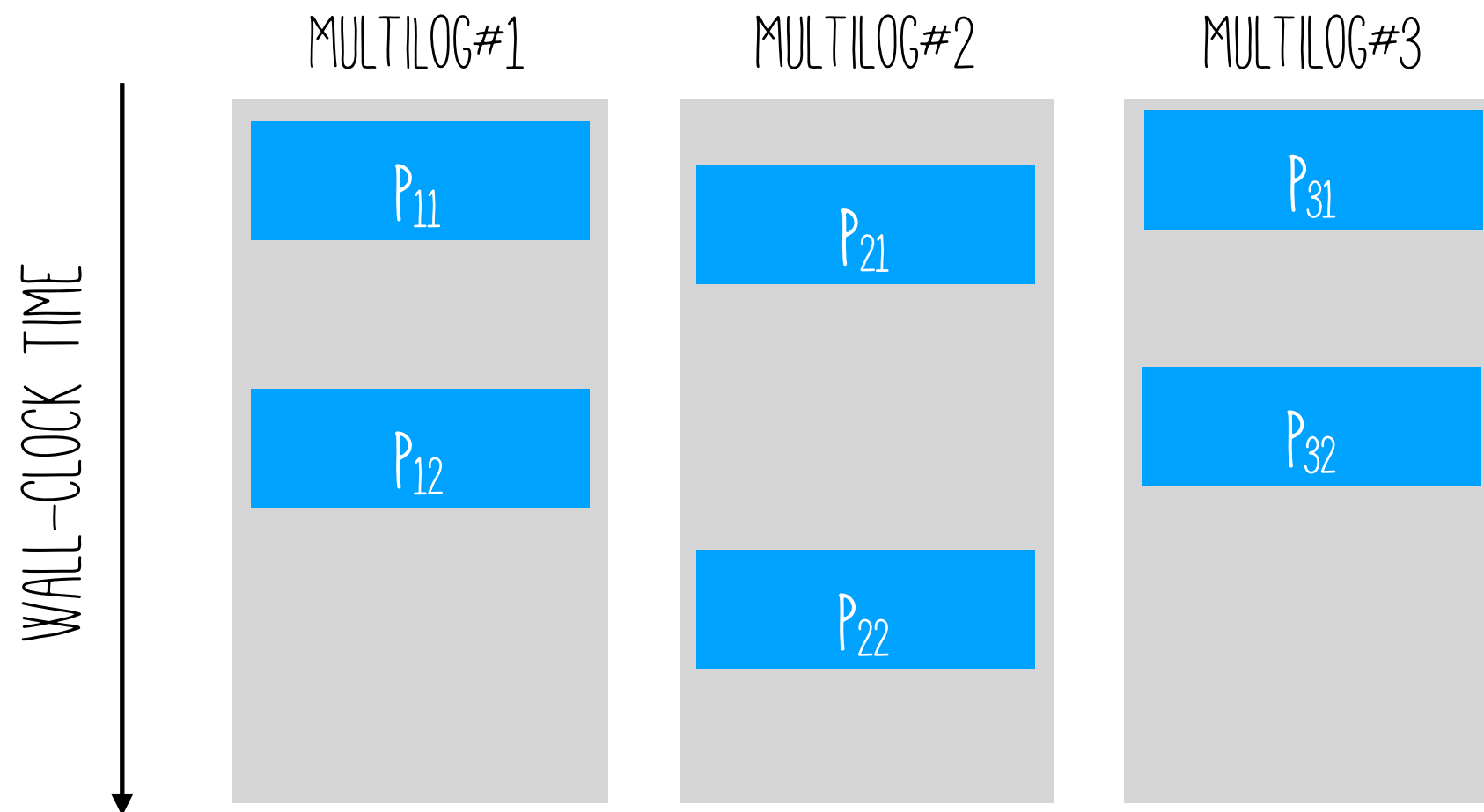
Consistency in Distributed Diagnosis

P_i = PACKET WRITES



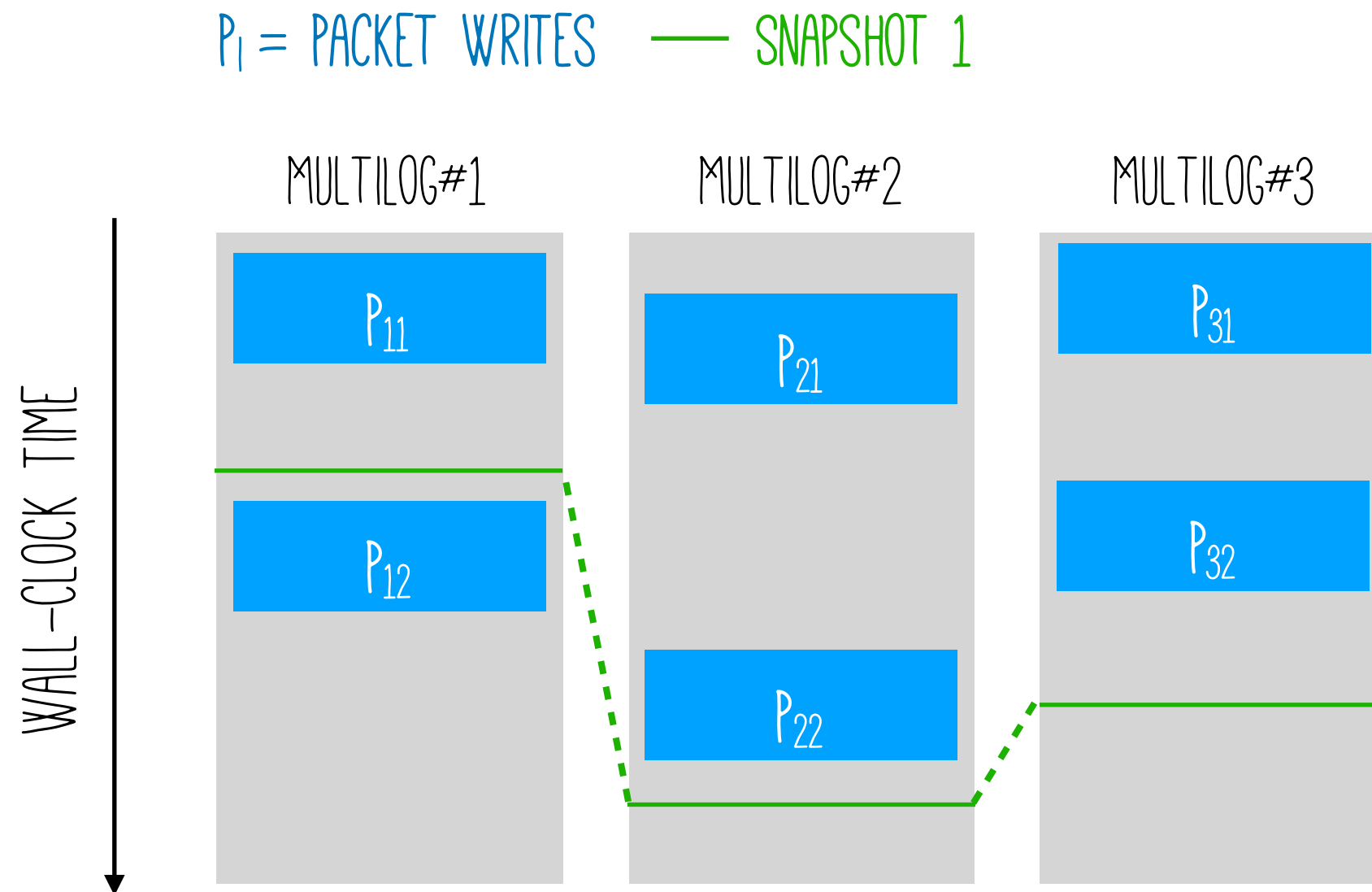
Consistency in Distributed Diagnosis

P_i = PACKET WRITES



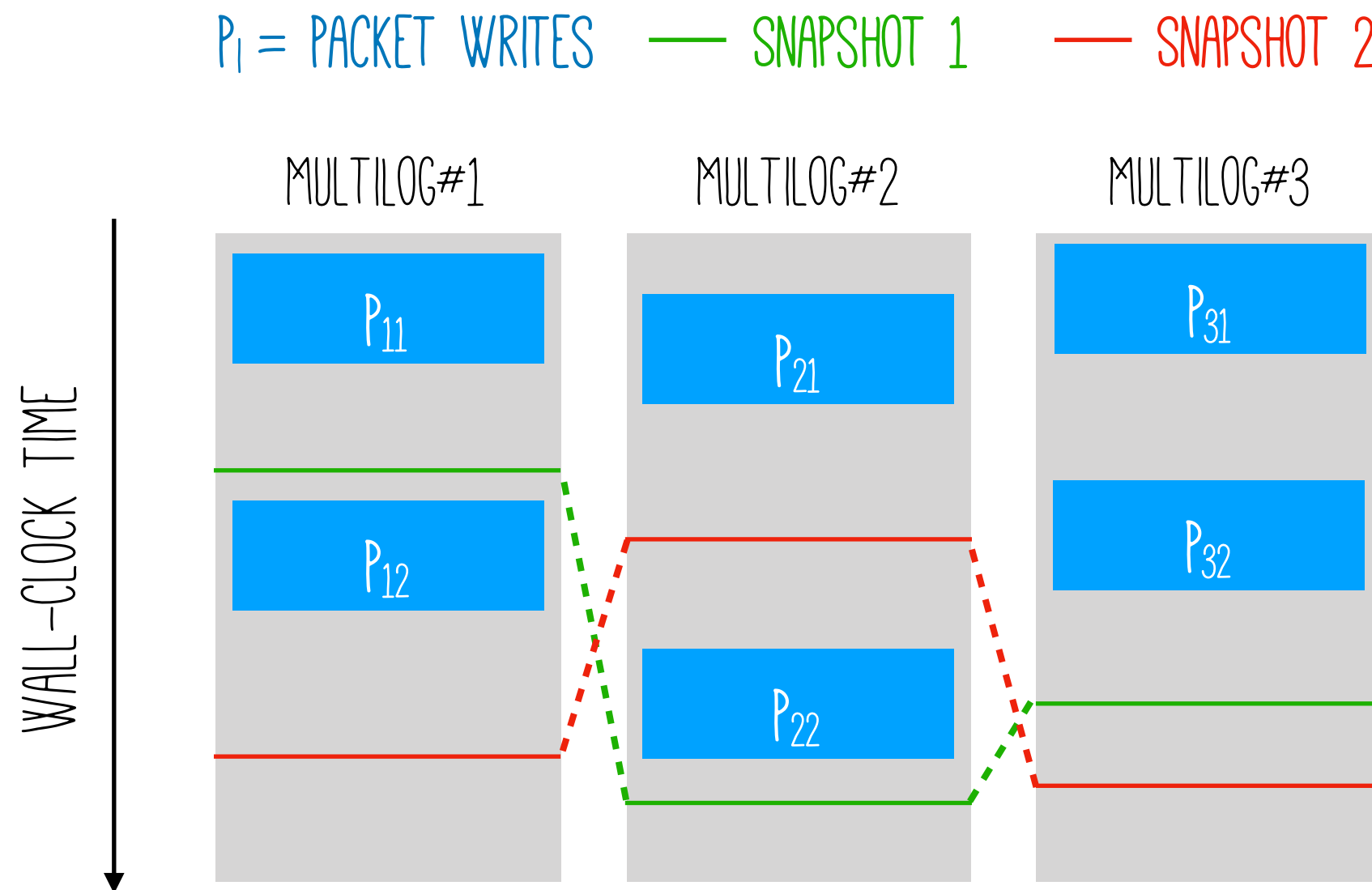
Naive approach to snapshot: obtain globalReadTails for all MultiLogs

Consistency in Distributed Diagnosis



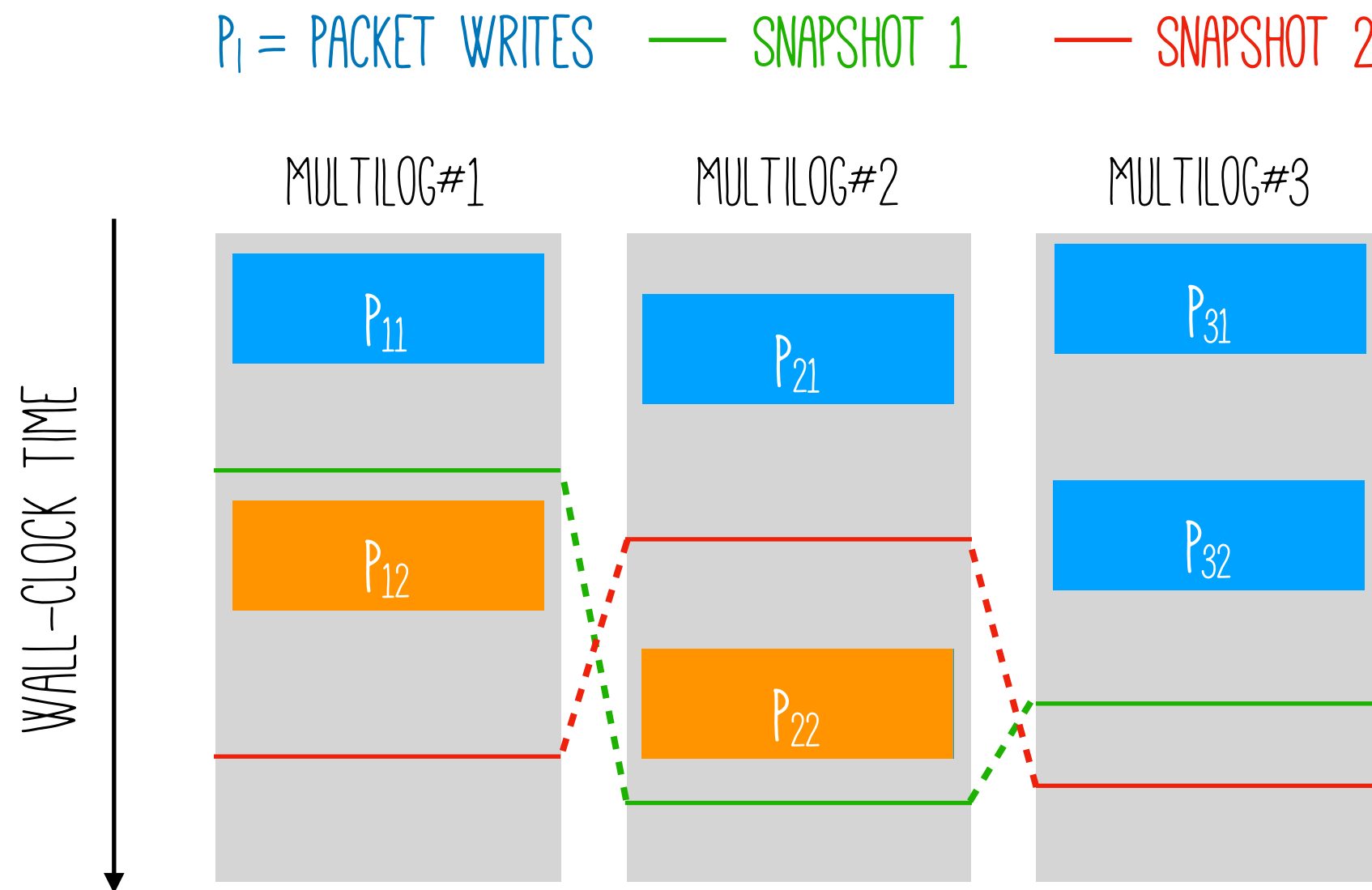
Naive approach to snapshot: obtain globalReadTails for all MultiLogs

Consistency in Distributed Diagnosis



Naive approach to snapshot: obtain globalReadTails for all MultiLogs

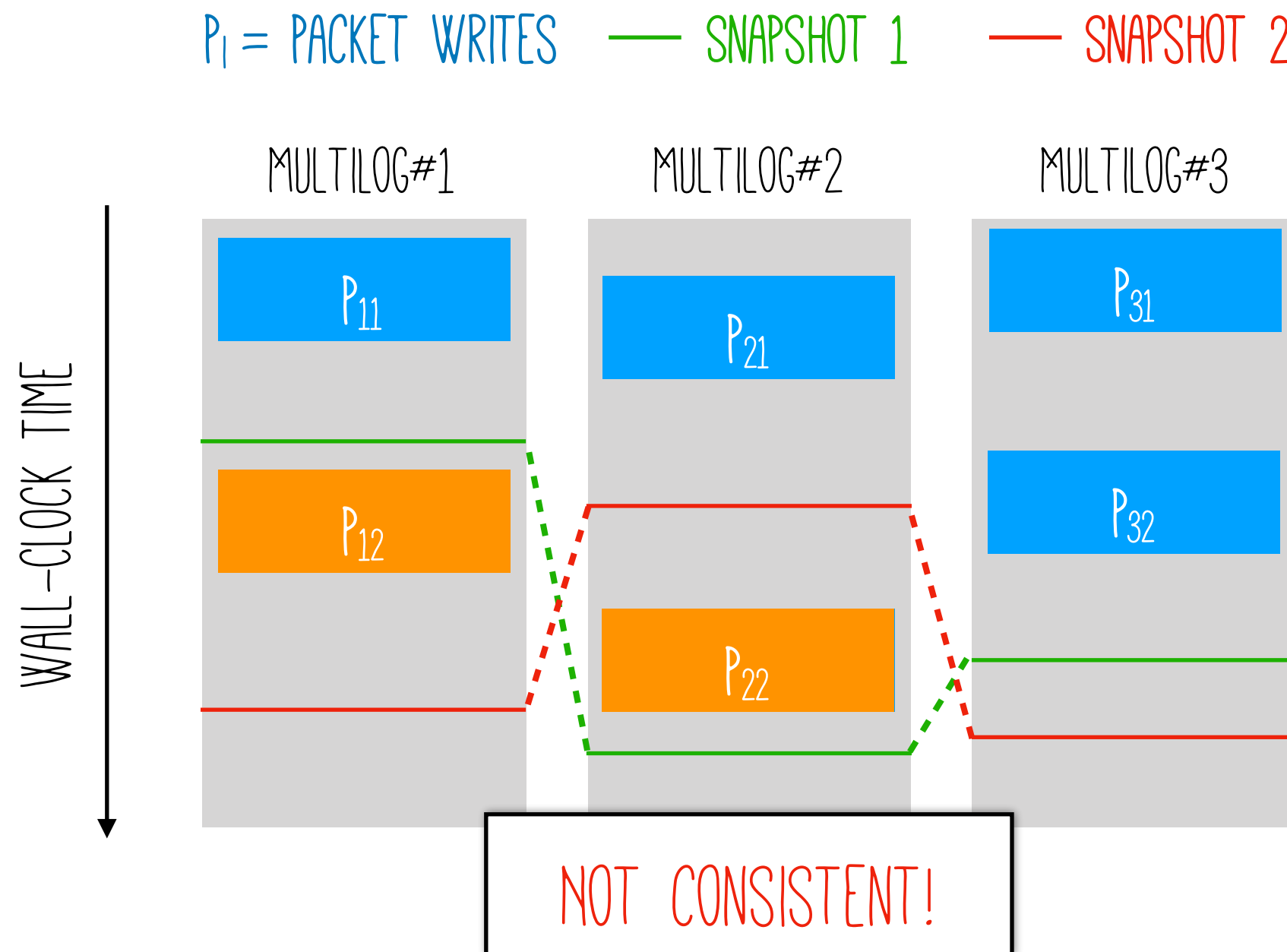
Consistency in Distributed Diagnosis



Naive approach to snapshot: obtain globalReadTails for all MultiLogs

Snapshot 1 contains P₂₂ but not P₁₂, Snapshot 2 contains P₁₂ but not P₂₂

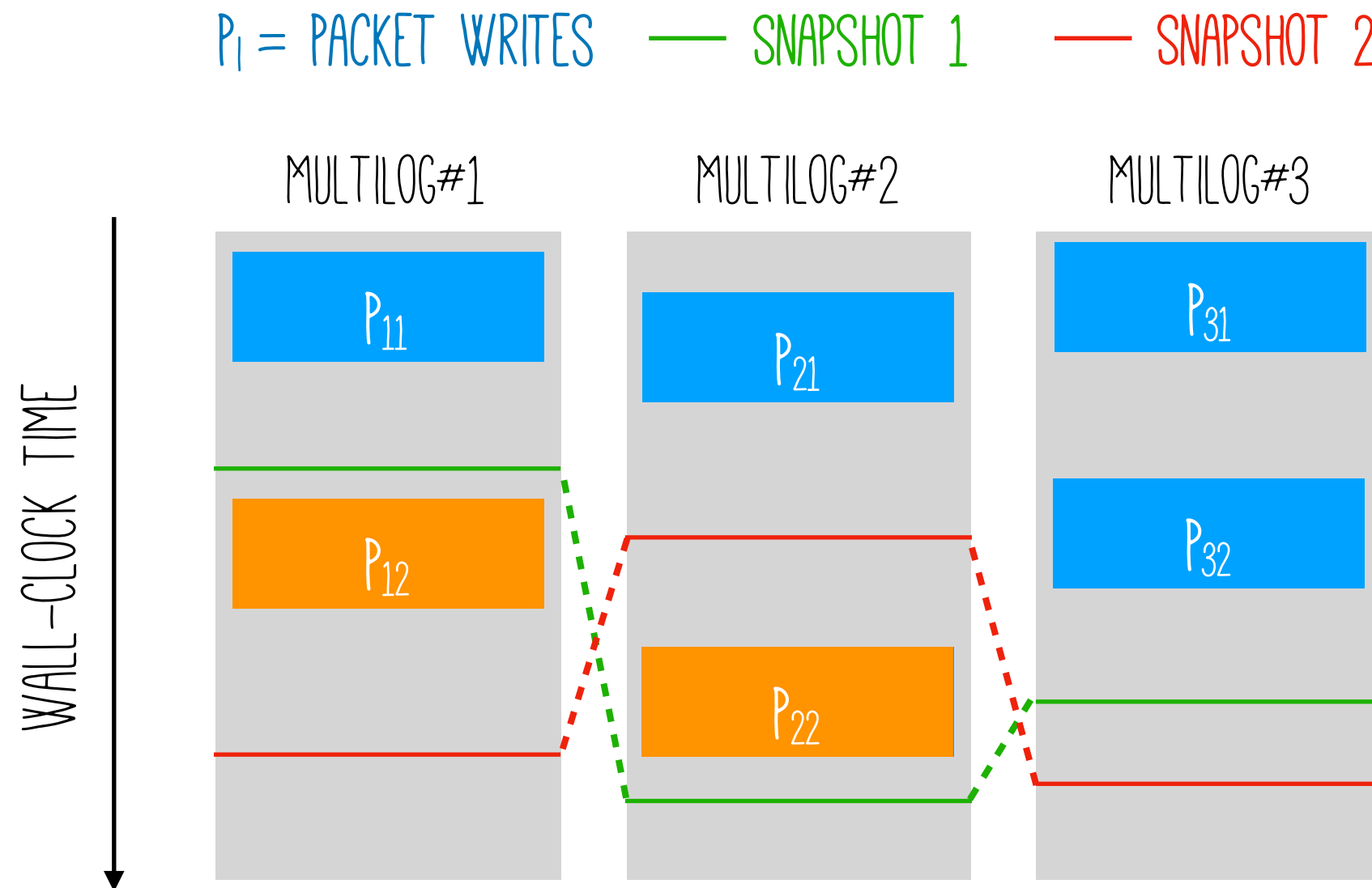
Consistency in Distributed Diagnosis



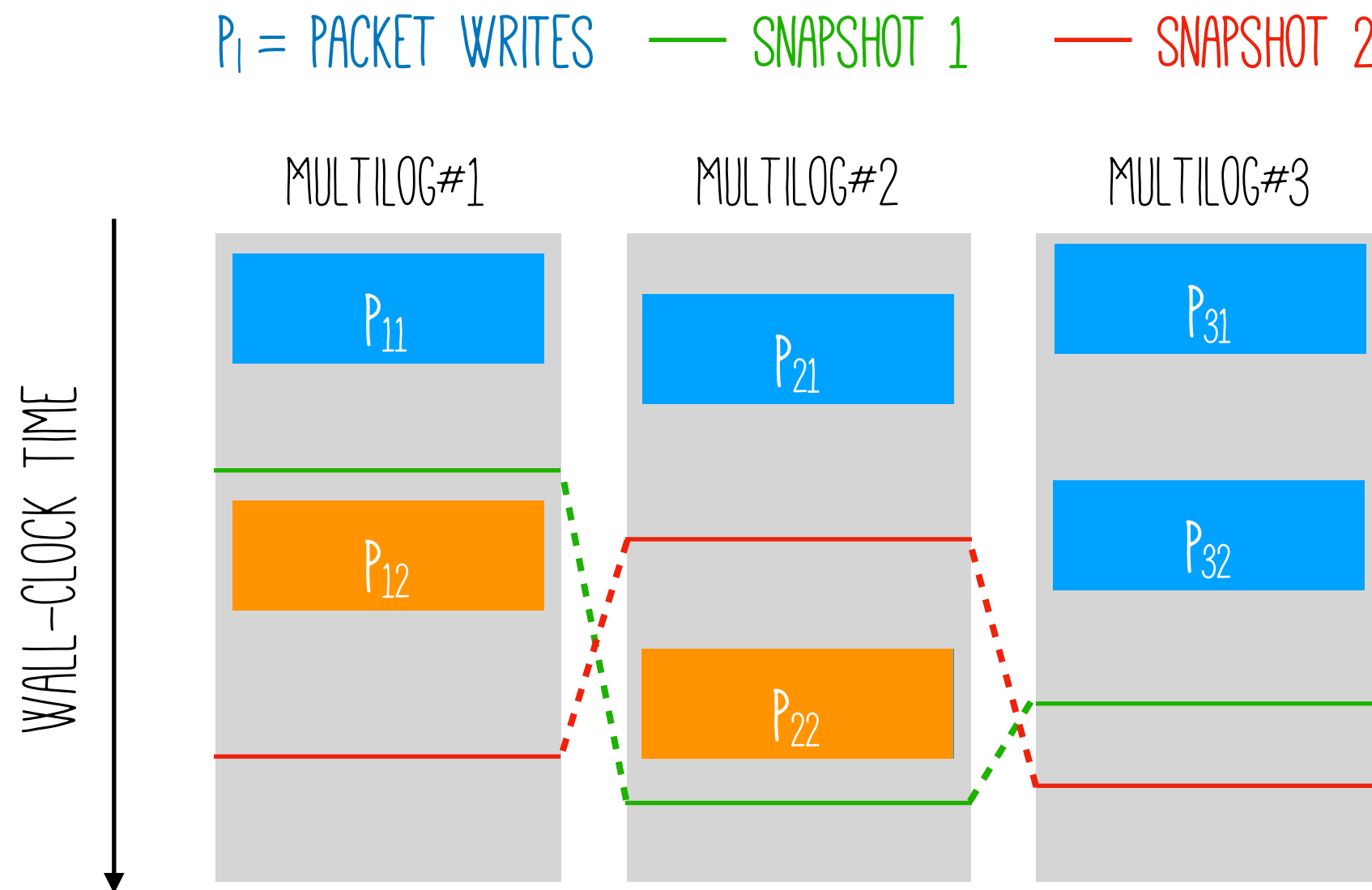
Naive approach to snapshot: obtain globalReadTails for all MultiLogs

Snapshot 1 contains P_{22} but not P_{12} , Snapshot 2 contains P_{12} but not P_{22}

Consistency in Distributed Diagnosis



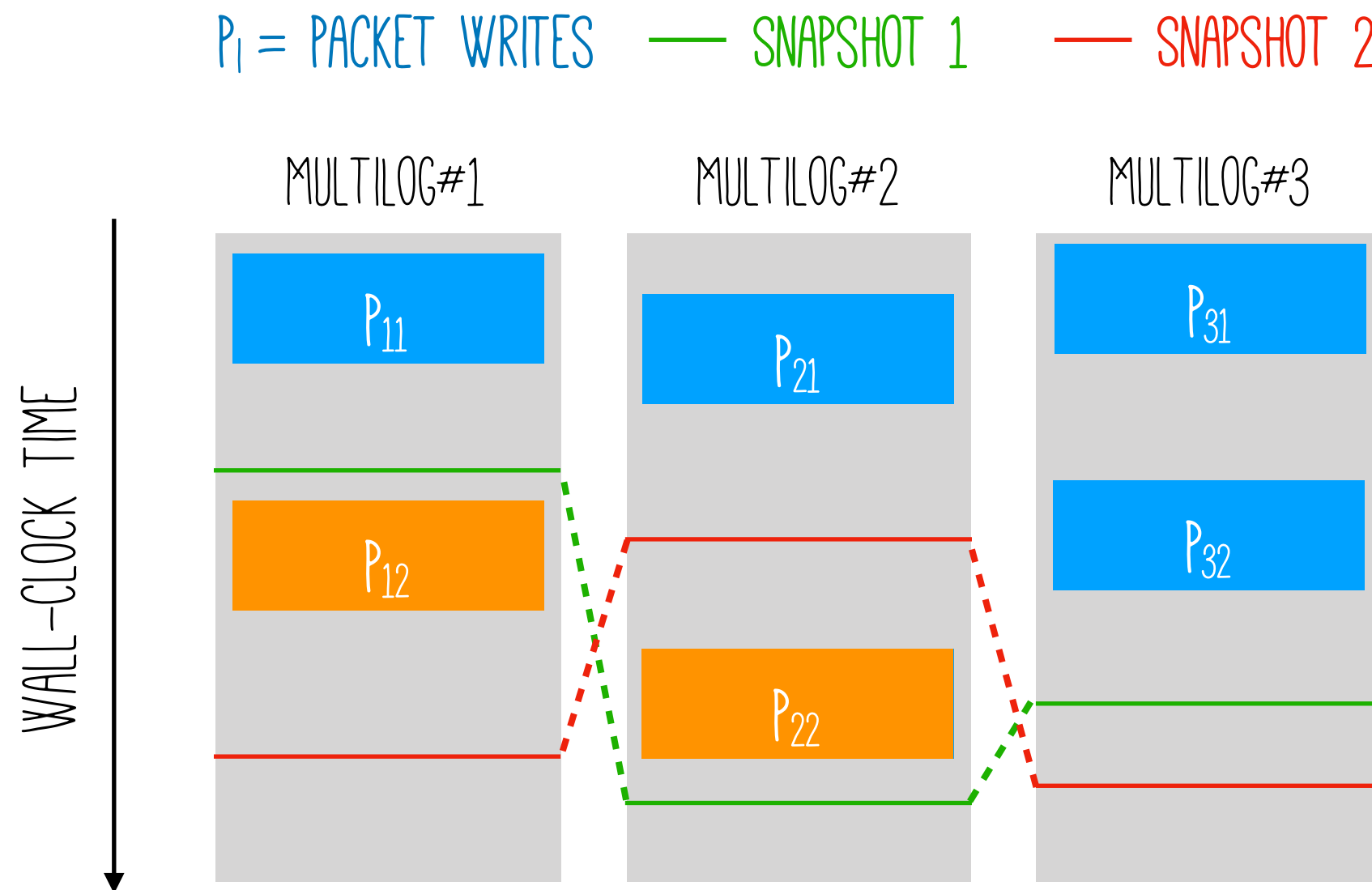
Consistency in Distributed Diagnosis



Existing
Approaches

- Centralized sequencer orders all writes to system (e.g., DBMS)

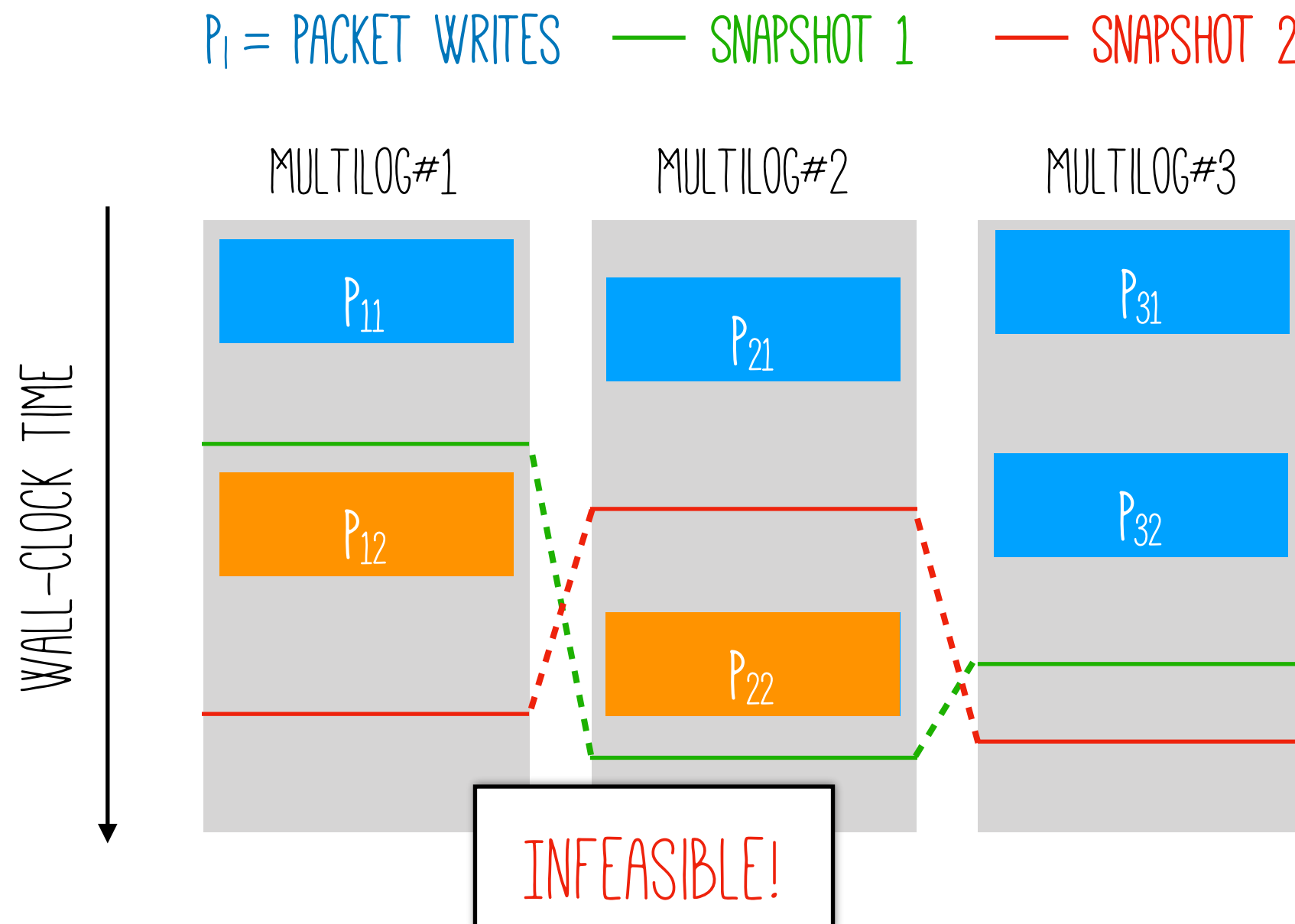
Consistency in Distributed Diagnosis



Existing Approaches

- Centralized sequencer orders all writes to system (e.g., DBMS)
- Algorithms with weaker consistency (e.g., Chandy Lamport)

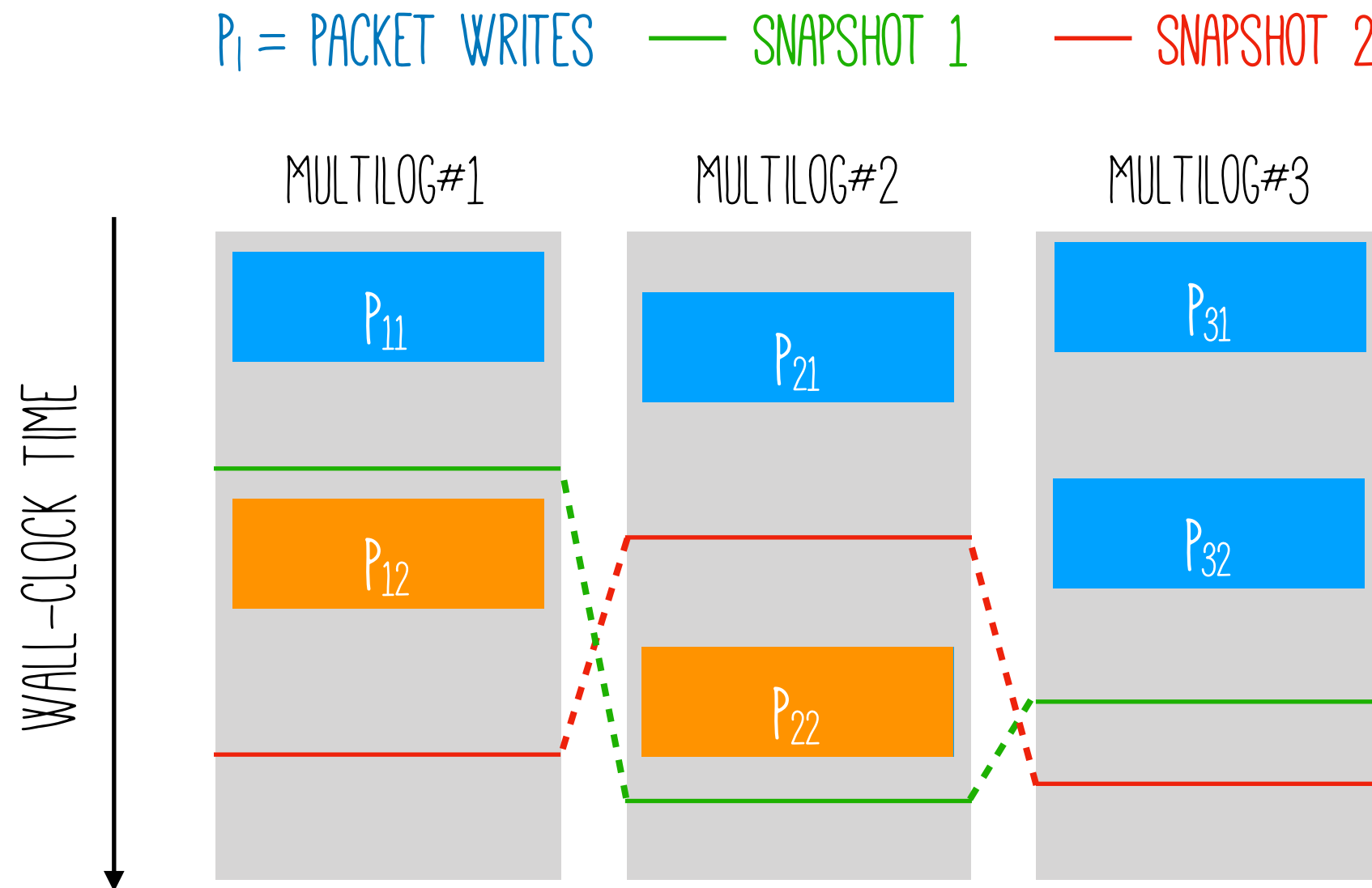
Consistency in Distributed Diagnosis



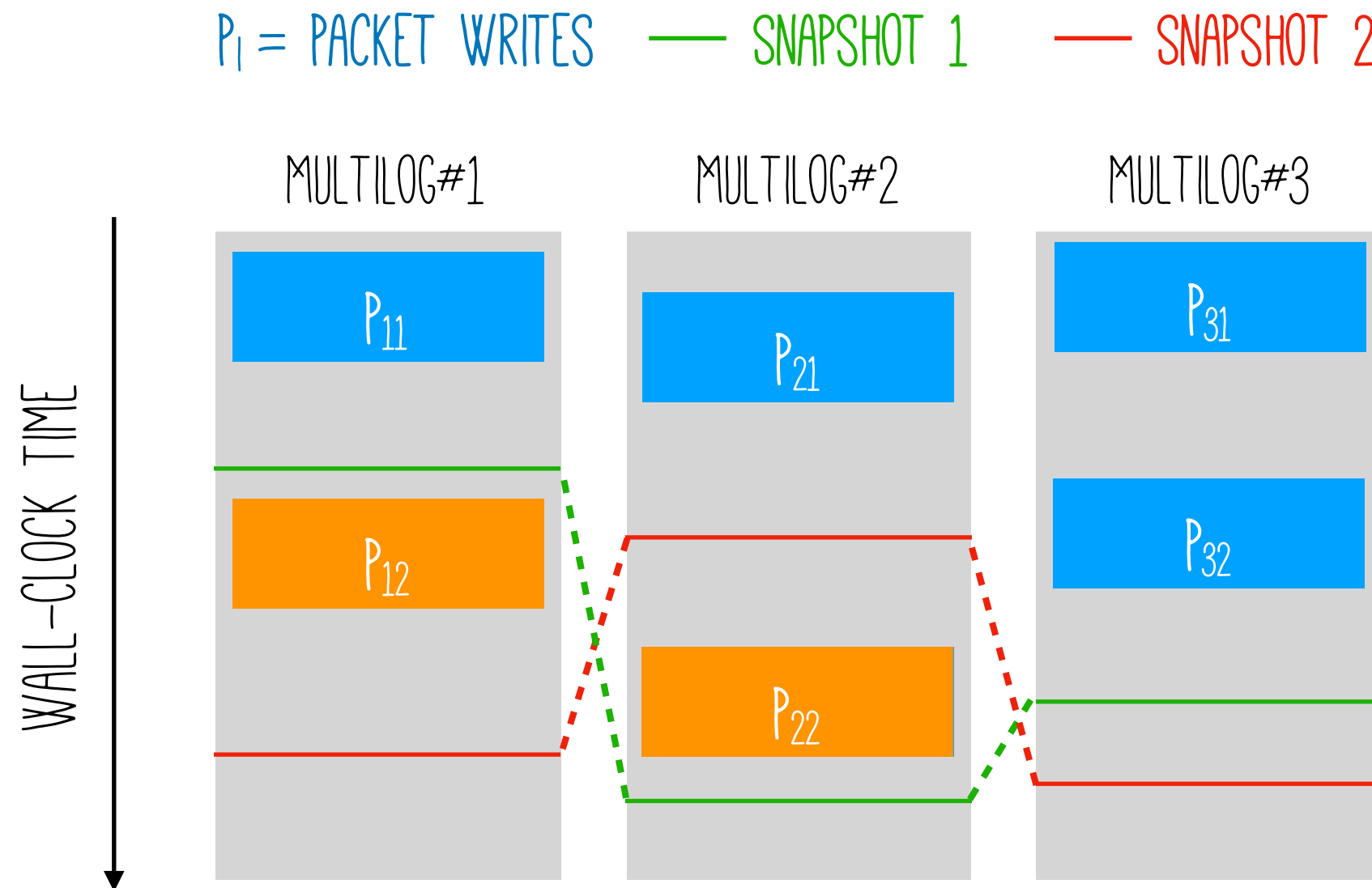
Existing
Approaches

- Centralized sequencer orders all writes to system (e.g., DBMS)
- Algorithms with weaker consistency (e.g., Chandy Lamport)

Linearizable Snapshots in Confluo

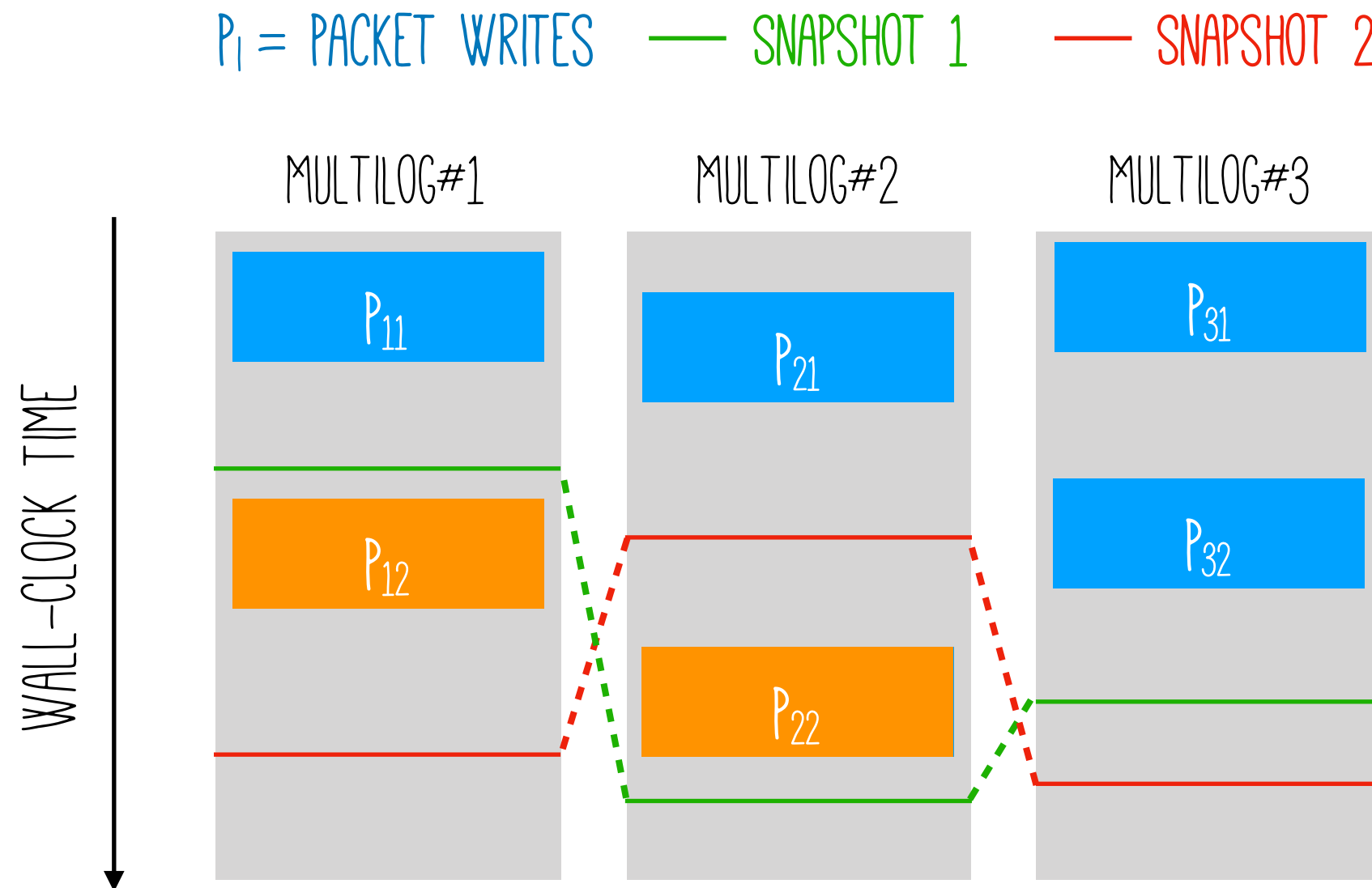


Linearizable Snapshots in Confluo



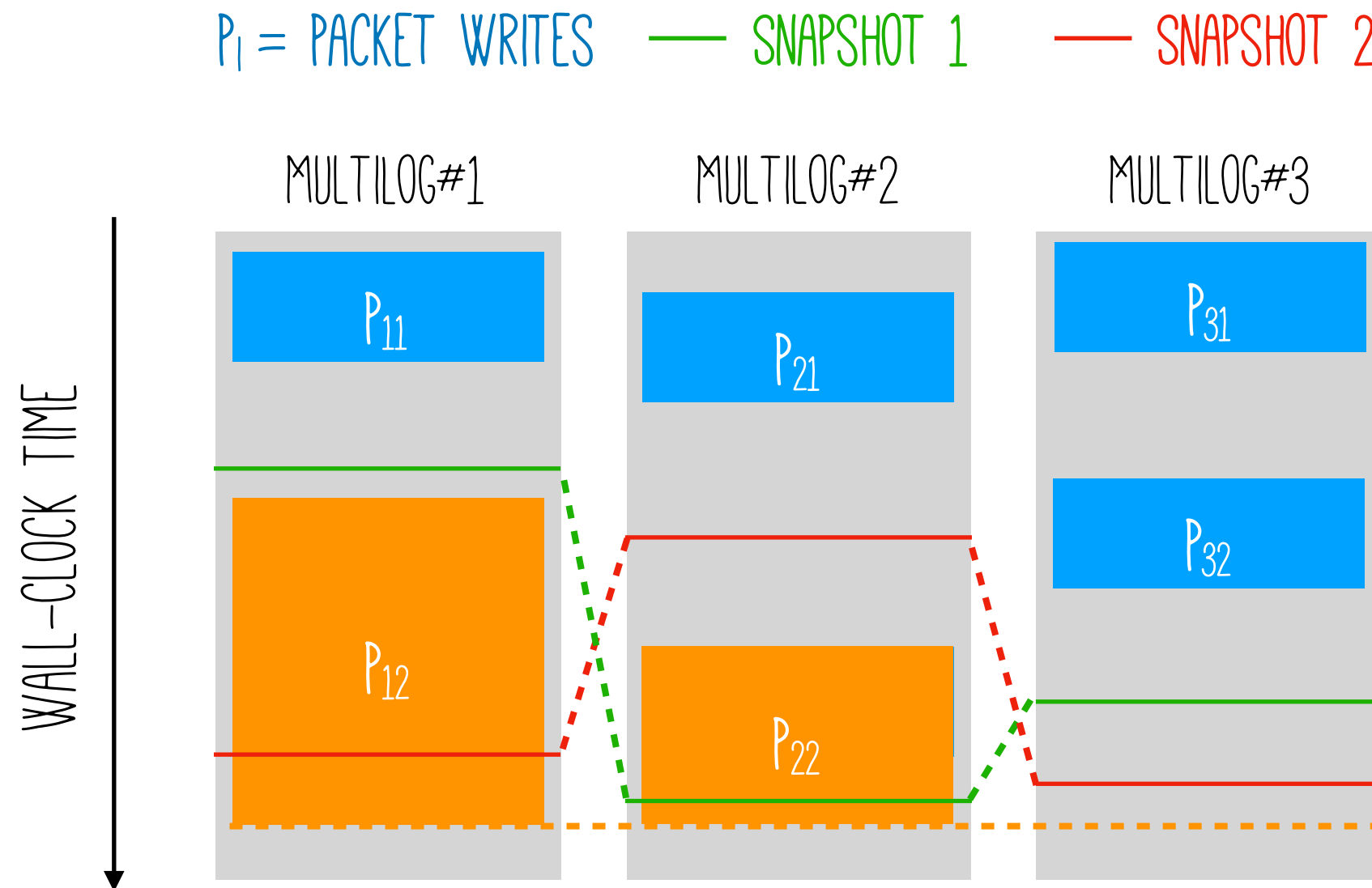
- Impose order on some writes **during query execution** rather than during writes

Linearizable Snapshots in Confluo



- Impose order on some writes **during query execution** rather than during writes
- How do we make the naive snapshots consistent?

Linearizable Snapshots in Confluo



- Impose order on some writes **during query execution** rather than during writes
- **Key insight:** Delay visibility of P_{11} , P_{22} to all queries until after the snapshots
- P_{11} , P_{12} now excluded from both snapshots

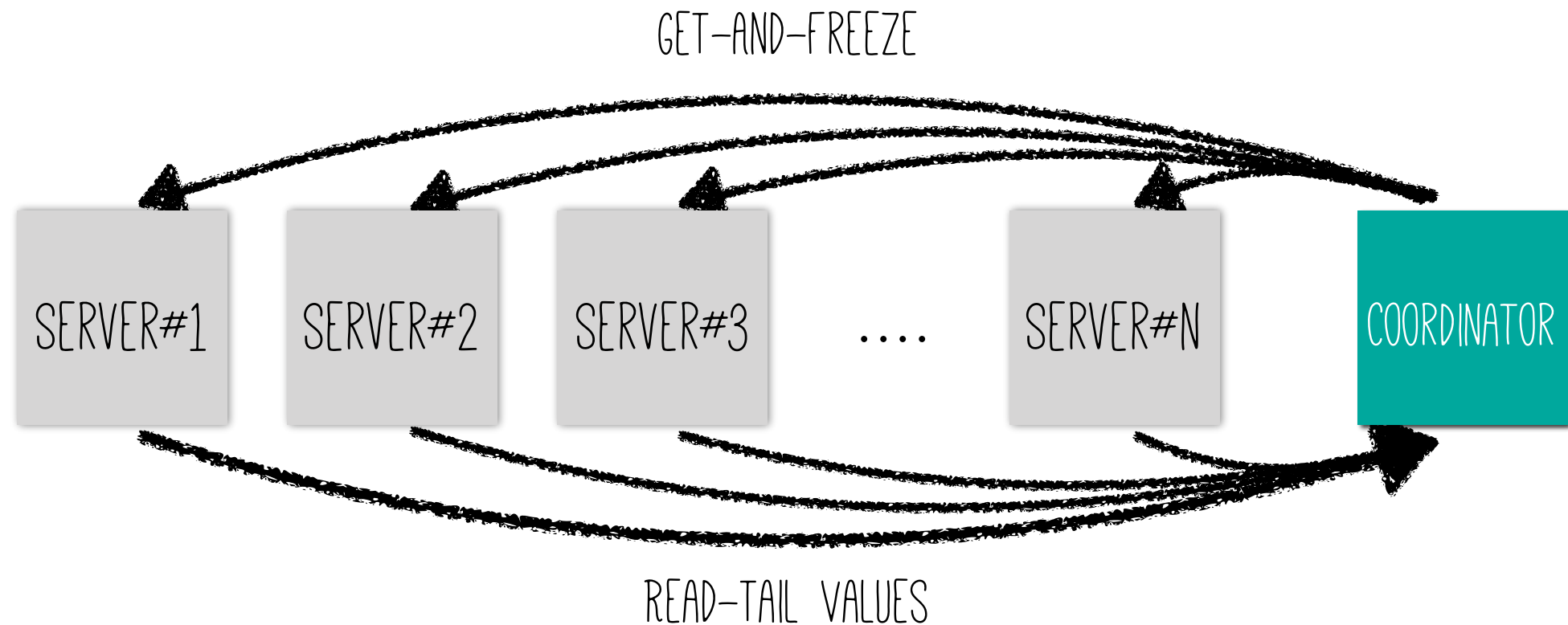
Linearizable Snapshots in Confluo

Delay packet writes that happen during a snapshot operation



Linearizable Snapshots in Confluo

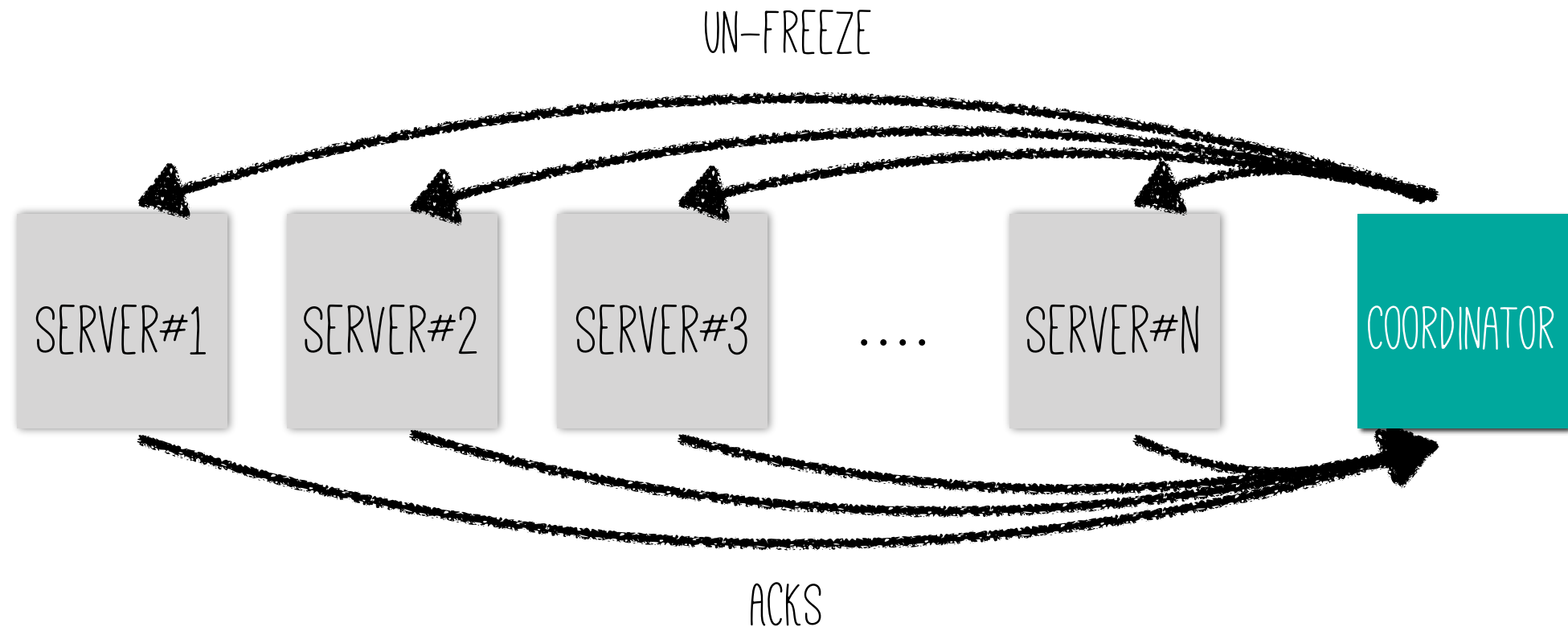
Delay packet writes that happen during a snapshot operation



1. Broadcast FreezeAndGet request to all servers
 - Each server *atomically* freezes and gets the value of readTail
2. Receive the readTail values from all servers

Linearizable Snapshots in Confluo

Delay packet writes that happen during a snapshot operation

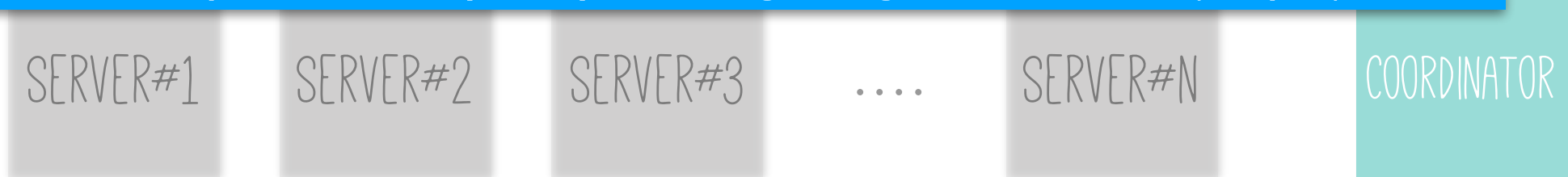


1. Broadcast FreezeAndGet request to all servers
 - Each server *atomically* freezes and gets the value of readTail
2. Receive the readTail values from all servers
3. Send Unfreeze request to all servers
 - If no other snapshot in progress, *atomically* un-freeze readTail
4. Receive ACKs from all servers

Linearizable Snapshots in Confluo

Delay packet writes that happen during a snapshot operation

The visibility of any packet write that would have completed during the snapshot is delayed by freezing the globalReadTail (Step 1)



1. Broadcast FreezeAndGet request to all servers
 - Each server *atomically* freezes and gets the value of readTail
2. Receive the readTail values from all servers
3. Send Unfreeze request to all servers
 - If no other snapshot in progress, *atomically* un-freeze readTail
4. Receive ACKs from all servers

Linearizable Snapshots in Confluo

Delay packet writes that happen during a snapshot operation

The visibility of any packet write that would have completed during the snapshot is delayed by freezing the globalReadTail (Step 1)

The packet writes are only made visible (in Step 3) *after* snapshot(s) have been collected (in Step 2)

1. Broadcast FreezeAndGet request to all servers
 - Each server *atomically* freezes and gets the value of readTail
2. Receive the readTail values from all servers
3. Send Unfreeze request to all servers
 - If no other snapshot in progress, *atomically* un-freeze readTail
4. Receive ACKs from all servers